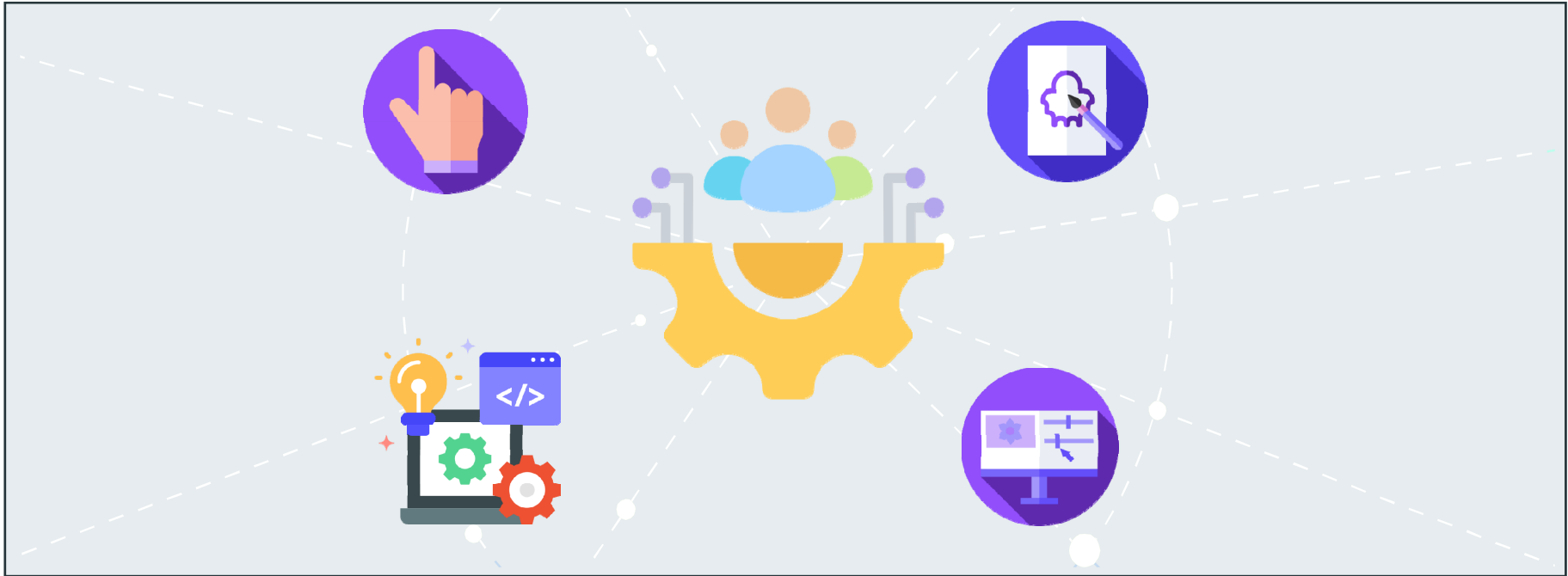
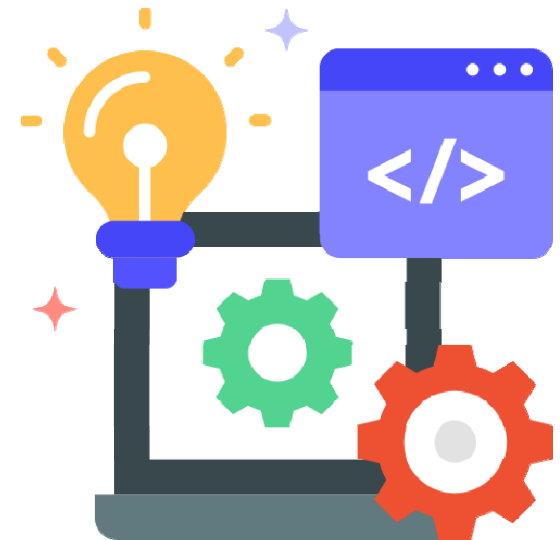




LLM 이해와 활용

목차

- Large Language Model 이해
- OpenAI API 주요 기능 활용
- RAG(Retrieval-Augmented Generation)



Large Language Model 이해



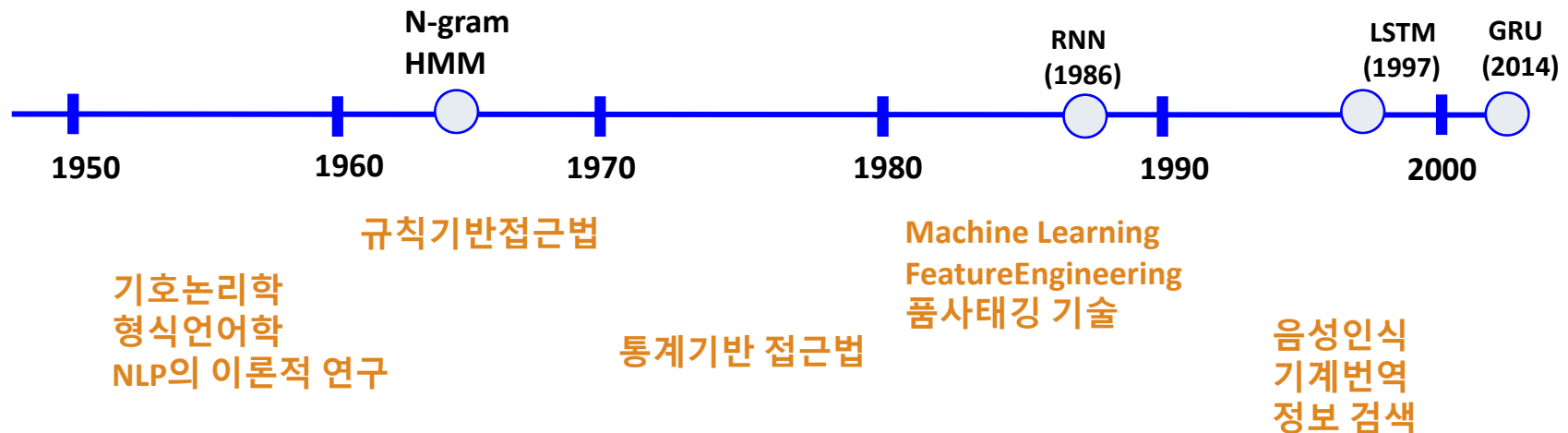
소목차

- 자연어처리(NLP) 기술의 발전
- 자연어 처리(NLP) Process
- LLM의 언어 처리 Task Model
- Large Language Model 이해

LLM(Large Language Model)

● 자연어처리(NLP) 기술의 발전

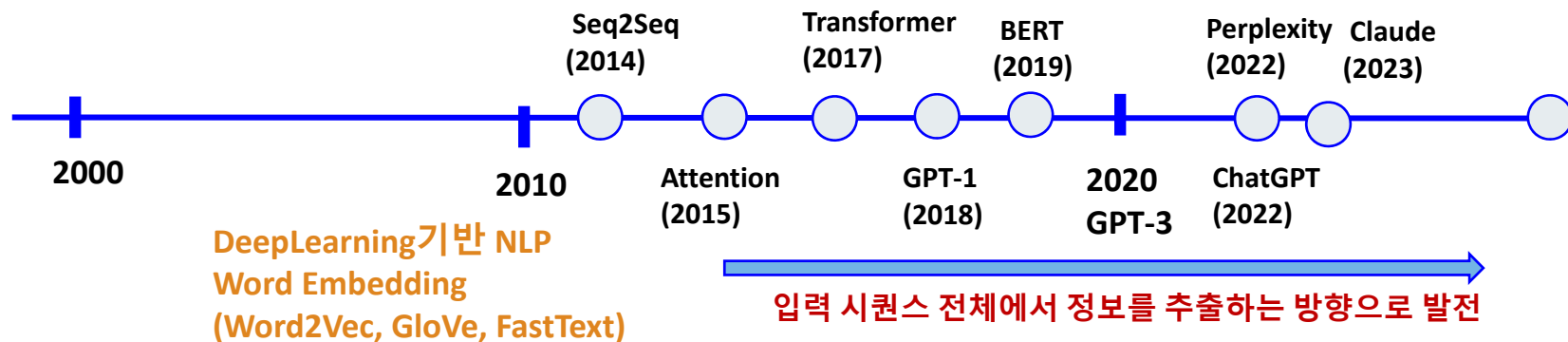
- **n-gram 모델** : 연속된 단어의 그룹을 기반으로 확률을 계산하는 모델
- **Hidden Markov Model (HMM)** : 관찰할 수 없는(숨겨진) 상태와 관찰 가능한 데이터 사이의 확률 관계를 모델링
- **Recurrent Neural Networks (RNN)** : 순차적 데이터(텍스트, 음성, 동영상 등)를 처리하기 위해 설계된 신경망 구조



LLM(Large Language Model)

● 자연어처리(NLP) 기술의 발전

- **Sequence-to-Sequence (Seq2Seq)** : Encoder(문맥 이해, 압축)-Decoder(문장 생성) 아키텍처 기반, 입력 시퀀스를 출력 시퀀스로 변환하는 모델 구조
- **Attention** : 입력 시퀀스 내의 중요한 부분에 집중하여, 시퀀스 처리 작업의 성능을 향상시키는 신경망 구조
- **Transformer** : 어텐션 메커니즘을 활용하여 순차적 계산을 최소화하고 병렬 처리를 극대화한 신경망 아키텍처



생성AI	특성
ChatGPT	텍스트 생성, 이미지 인식, 수치 분석, 그래프 해석, 음성 대화 등 다양한 작업을 수행
Claude	자연스러운 대화 능력, 빠른 응답 https://claude.ai
Perplexity	AI기반 검색 엔진 https://www.perplexity.ai

LLM(Large Language Model)

● 자연어 처리(NLP) Process

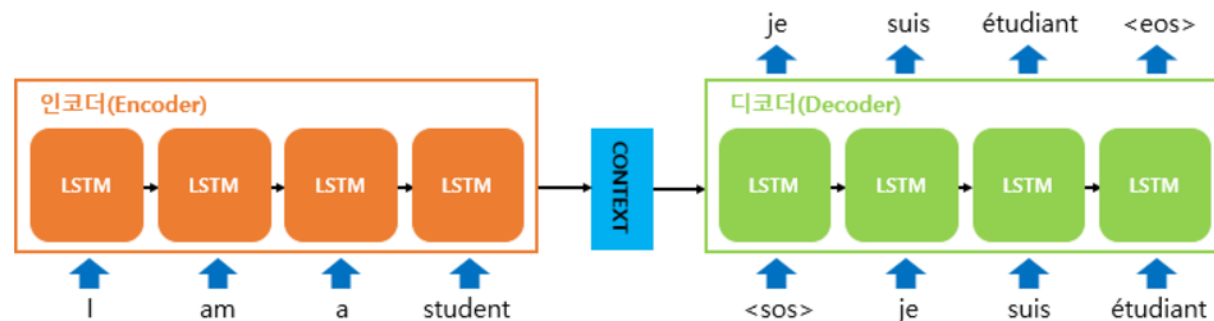
1. **Cleaning** : 불필요한 데이터 제거(예: HTML 태그, 특수 문자), 노이즈 감소
 - 데이터의 품질을 보장하고 다음 단계의 처리를 용이하게 합니다.
2. **Tokenization** : 텍스트를 작은 단위(토큰)로 나누는 과정
3. **정규화 (Normalization)** : 데이터의 일관성을 높이고, 처리 과정의 복잡성을 줄이고 텍스트 데이터의 변형을 최소화
 - 어간 추출 (Stemming) : 단어에서 접사를 제거하여 기본 형태(어간)를 찾는 과정
 - 표제어 추출 (Lemmatization): 단어를 사전에 등록된 형태(표제어)로 변환.
 - 형태소 분석 (Morphological Analysis): 단어를 형태소로 분석하여 그 구조를 이해하는 과정.
4. **품사 태깅 (Part-of-Speech Tagging)**: 각 토큰에 품사(명사, 동사 등)를 할당하는 과정
 - 구문 분석이나 의미 분석에 중요한 정보를 제공
5. **임베딩 (Embedding)** : 토큰을 벡터 공간에 매핑하여 컴퓨터가 처리할 수 있는 수치적 형태로 변환.
6. **학습 (Training)** : 주어진 태스크(예: 분류, 번역)에 맞게 모델을 학습시키는 과정
 - 데이터를 기반으로 패턴을 학습하고 모델 파라미터를 최적화
7. **생성 및 분류 (Generation and Classification)**

LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ Sequence-to-Sequence (Seq2Seq)

- ▶ 하나의 입력 시퀀스를 받아 다른 출력 시퀀스로 변환하는 신경망 구조
- ▶ Encoder(문맥이해, 압축)-Decoder(문장 생성) 아키텍처 기반
- ▶ 컨텍스트 벡터를 기반으로 다음 단어를 예측
- ▶ 두 개의 RNN(인코더와 디코더)를 연결하여 구현
- ▶ 입력과 출력 시퀀스의 길이가 달라도 처리 가능
- ▶ 초기 Seq2Seq 모델에서는 모든 입력 정보를 고정 길이 벡터 C로 압축하므로, 긴 시퀀스에서는 정보 손실이 발생 (장기 의존성 문제)
- ▶ 정보 손실을 해결하기 위해 Attention 메커니즘 도입



LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ Sequence-to-Sequence (Seq2Seq)

▶ Encoder

- 입력 시퀀스를 고정된 길이의 벡터(컨텍스트 벡터)로 변환
- 입력 데이터를 단계별로 처리하여 입력 시퀀스의 정보를 함축적으로 표현
- RNN, LSTM, GRU 등의 순환 신경망이 사용됨

▶ Decoder

- Encoder에서 생성된 컨텍스트 벡터를 사용하여 출력 시퀀스를 생성
- RNN 계열 모델로 구성
- 생성된 단어(또는 토큰)와 컨텍스트 벡터를 기반으로 Decoder가 생성한 단어들을 하나씩 결합하면서 다음 단어를 예측하며, 생성해야 할 단어의 수만큼 예측 반복

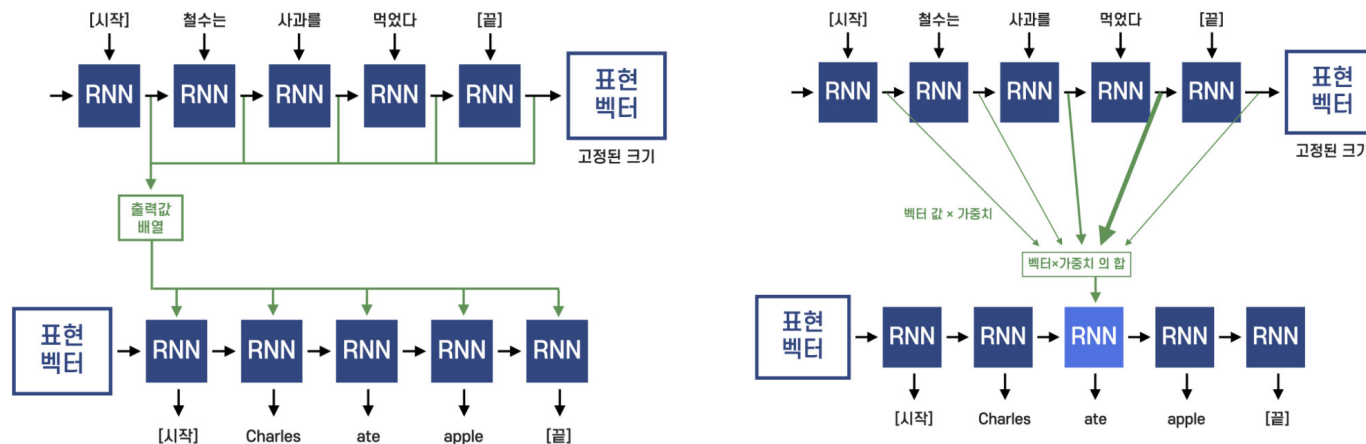
▶ 제약 : 문맥 정보가 인코더의 마지막 벡터 하나에 집중

LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ Attention Mechanism

- ▶ Seq2Seq 모델에서 Decoder가 입력 시퀀스의 모든 단어를 동적으로 참조할 수 있도록 도와주는 방법
- ▶ Seq2Seq 모델에서 발생하는 정보 손실 문제를 해결하기 위해 도입
- ▶ Decoder 가 출력 단어를 생성할 때 입력의 특정 부분에 더 집중(Attention) 하도록 함
- ▶ Context Vector 는 첫 단어의 예측에 가장 많은 영향을 미치는 단어에 대한 정보가 담겨 있다 (단어별로 계산된 Attention Score에 Softmax를 적용한 가중치의 표현)
- ▶ 모든 디코더의 RNN이 인코더의 각 타임 스텝마다 RNN의 결과 값인 출력을 참고하는 방식



출처 : <https://deepdaiv.oopy.io/55b8a52b-4904-4455-ae32-e5985e9914d9>

자연어 처리 기초

● LLM의 언어 처리 Task Model

→ Attention Mechanism

Attention	특성
Supervised Attention	학습 데이터에서 주어진 주석(어노테이션)이나 레이블을 기반으로 입력 데이터 내 특정 부분에 주의를 집중하는 방법을 학습합니다.
Multi-headed Attention	입력 데이터의 다양한 부분에서 정보를 동시에 추출할 수 있도록 설계된 복수의 attention 메커니즘(헤드)을 사용합니다. 각 헤드는 입력 시퀀스의 다른 측면에 초점을 맞추어 독립적으로 attention 점수를 계산하고, 결과적으로 모델은 입력 데이터의 다양한 표현을 동시에 학습할 수 있습니다.
Self-Attention	입력 시퀀스 내에서 각 위치가 다른 모든 위치와 어떻게 상호 작용하는지를 모델링하는 방식입니다. 더 풍부한 데이터 표현을 생성할 수 있습니다.

자연어 처리 기초

● LLM의 언어 처리 Task Model

→ Transformer

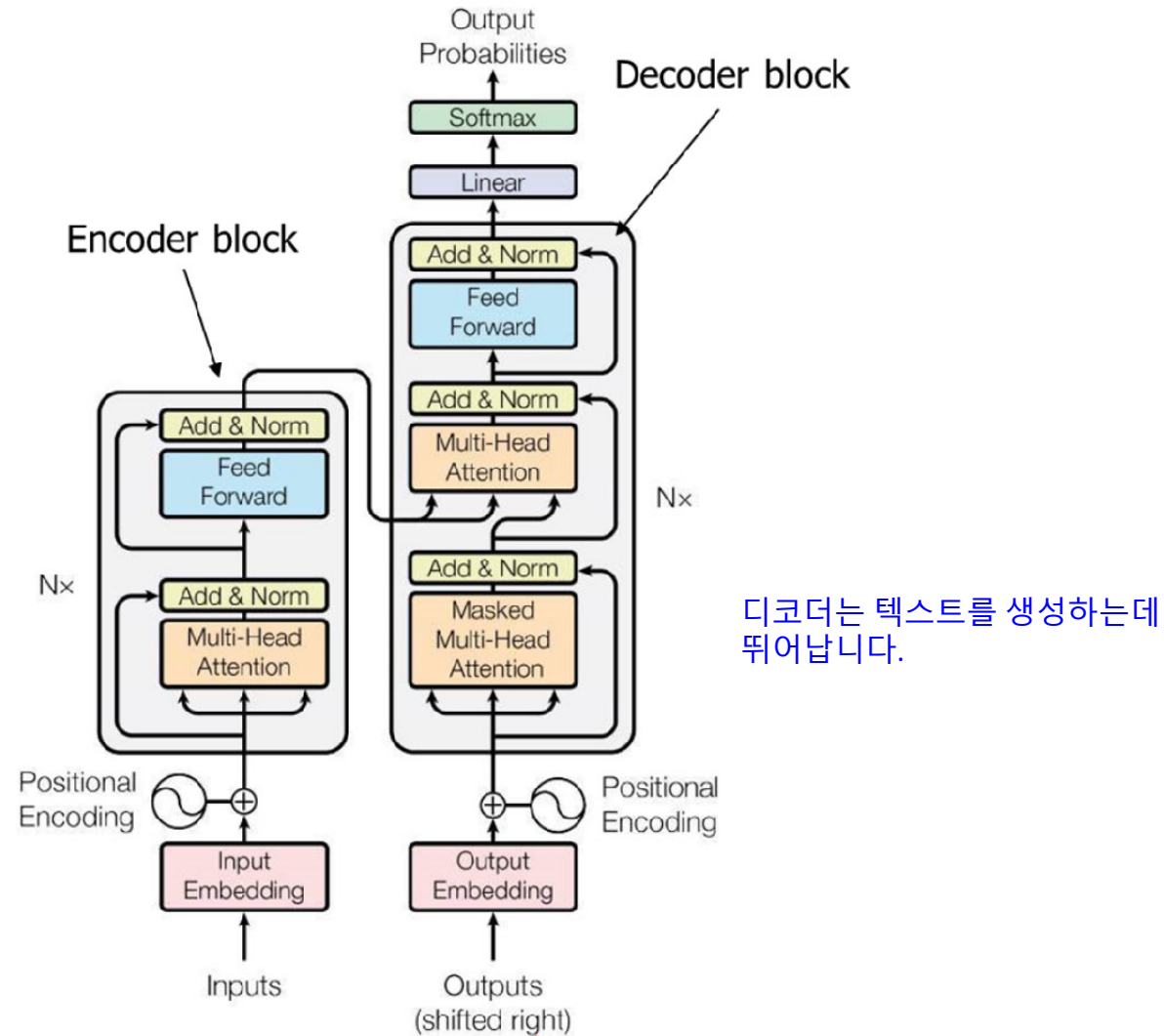
- ▶ 각 토큰을 시퀀스에 존재하는 다른 토큰들과 연관시킨다
- ▶ 순차적인 데이터 처리의 필요성을 최소화하고 전체 시퀀스를 한 번에 처리할 수 있게 합니다.
- ▶ Self-Attention Mechanism
 - 입력 데이터의 모든 부분이 서로 상호 작용할 수 있도록 하여, 문맥 이해를 극대화합니다.
 - 각 단어가 문장 내 다른 단어와의 관계를 동시에 고려하여, 풍부하고 상세한 텍스트 표현을 생성합니다.
- ▶ Multi-Head Attention
 - 여러 개의 어텐션 메커니즘(헤드)을 동시에 사용하여 다양한 부분에서 정보를 추출하고, 입력 시퀀스의 다양한 표현을 학습합니다.
 - 모델이 더 넓은 범위의 정보를 포착하고, 다양한 관점에서 데이터를 해석할 수 있도록 돕습니다.
- ▶ Positional Encoding
 - 시퀀스의 순서 정보를 모델에 포함시키기 위해 위치 인코딩을 사용합니다.
 - 단어의 위치에 따른 문맥적 의미를 모델이 이해할 수 있도록 돕습니다.
- ▶ Layered Architecture
 - 여러 개의 인코더와 디코더 층으로 구성되어 있으며, 각 층은 복잡한 패턴과 관계를 학습

LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ Transformer

인코더는 텍스트를
이해하는 데 뛰어납니다.

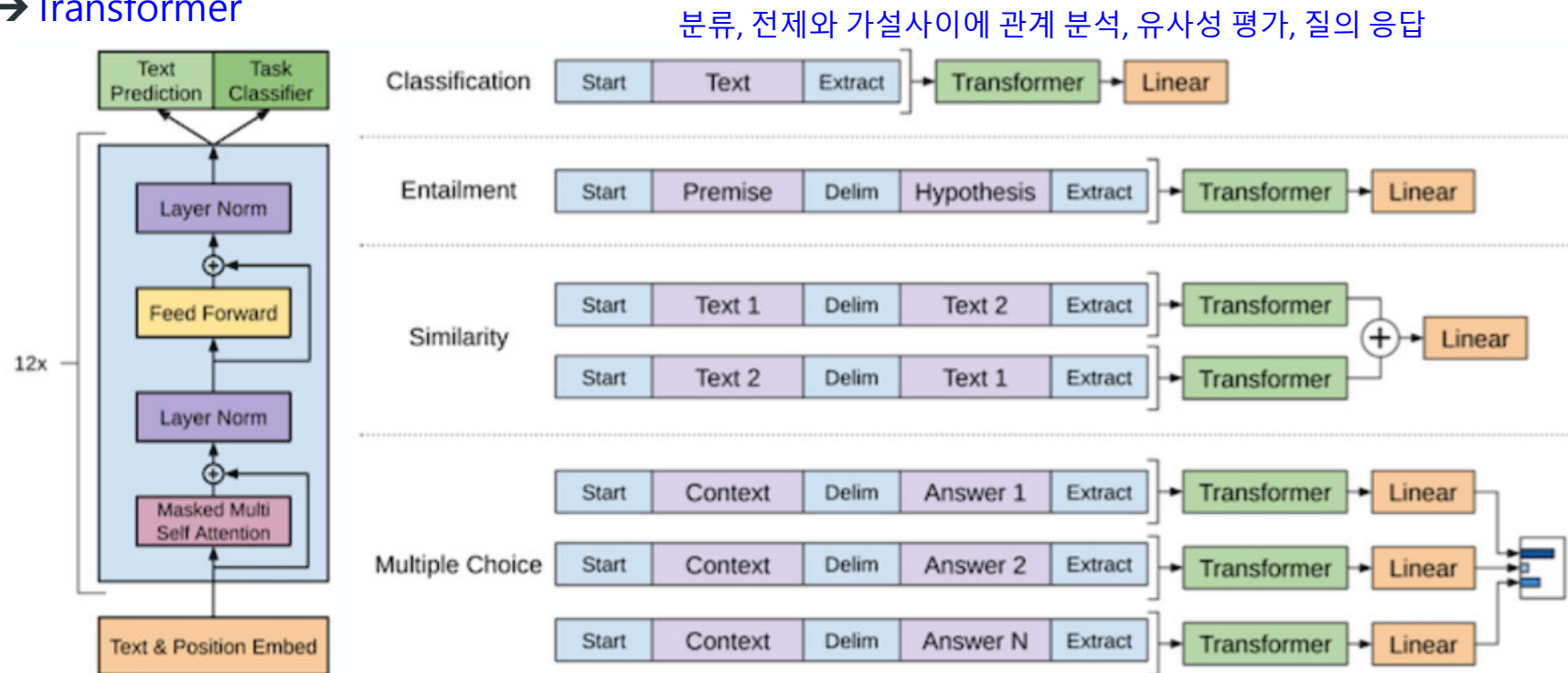


디코더는 텍스트를 생성하는데
뛰어납니다.

LLM(Large Language Model)

●LLM의 언어 처리 Task Model

→Transformer



Text & Position Embed : 텍스트 데이터와 해당 위치 정보를 포함하는 임베딩 벡터를 생성

12x Transformer Block :

- Masked Multi Self Attention: 입력 시퀀스 내의 각 요소가 서로 상호 작용하며 중요한 정보에 집중할 수 있도록 합니다.
- Layer Norm: 층 정규화를 통해 학습 과정을 안정화하고 더 빠르게 수렴하도록 돕습니다.
- Feed Forward: 각 위치에서 독립적으로 적용되는 신경망으로, 비선형 변환을 추가합니다.

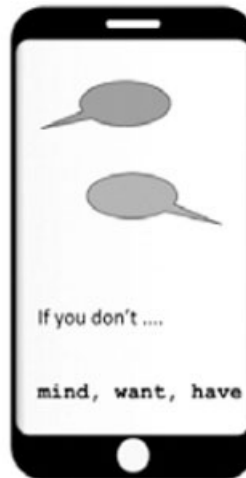
LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ 자기 회귀 언어 모델(Autoregressive Language Model)

- ▶ 문장에서 이전 토큰만을 기반으로 다음 토큰을 예측
- ▶ 알려진 어휘에서 주어진 문장의 바로 다음에 가장 가능성 있는 토큰을 생성하도록 모델에 요청
- ▶ Transformer Model의 Decoder
- ▶ Attention Head가 앞서 온 토큰만 볼 수 있도록 전체 문장에 mask가 적용
- ▶ 텍스트 생성에 이상적

예) GPT



자기회귀 언어 모델은 알려진 어휘에서 주어진 문장의 바로 다음에 가장 가능성 있는 토큰을 생성하도록 모델에 요청합니다.

LLM(Large Language Model)

● LLM의 언어 처리 Task Model

→ 자동 인코딩 언어 모델(Autoencoding Language Model)

- ▶ 손상된 버전의 입력 내용으로부터 기존 문장을 재구성하도록 훈련
- ▶ 알려진 어휘에서 문장의 어느 부분이든 누락된 단어를 채우도록 모델에 요청
- ▶ Transformer Model의 Encoder
- ▶ 전체 문장의 양방향 표현을 생성
- ▶ 마스크 없이 전체 입력에 접근할 수 있습니다
- ▶ 텍스트 생성과 같은 다양한 작업에 파인 튜닝 될 수 있습니다.
- ▶ 예) BERT



표지판에서 __ 하지 않으면, 벌금을 받게 될 것입니다.

자동 인코딩 언어 모델은 알려진 어휘에서 문장의 어느 부분이든 누락된 단어를 채우도록 모델에 요청합니다.

LLM(Large Language Model)

○ Large Language Model 이해

→ 거대 언어 모델(Large Language Model)

- ▶ 자기 회귀 이거나 자동 인코딩 또는 두 가지의 조합이 될 수 있는 언어 모델
- ▶ 인터넷의 텍스트, 책, 논문, 기사 등 **다양하고 방대한 양의 텍스트 데이터로부터 학습**
- ▶ **언어를 이해하고 생성하는데 특화되어 질문에 답변하고, 글을 작성하고, 대화를 나누는 생성적 작업 수행**
- ▶ **Transformer Architecture 기반**
- ▶ 특화된 분야에 정교하게 사용될 수 있도록 파인 튜닝 할 수 있습니다.
- ▶ 수천억 개 이상의 파라미터를 갖는 모델
- ▶ 텍스트 생성 및 분류와 같은 복잡한 언어 작업을 파인 튜닝이 필요 없을 만큼 높은 정확도로 수행
- ▶ 단계적으로 추론을 유도하여 복잡한 문제의 논리적 추론을 수행
- ▶ GPT-3.5 Turbo, GPT-4, 구글의 Bert, Gemini, PaLM2(Pathways Language Model2), LLaMa2

Small Language Model – 1500만 개 미만의 파라미터를 갖는 모델

LLM(Large Language Model)

Large Language Model 이해

→ Large Language Model 비교

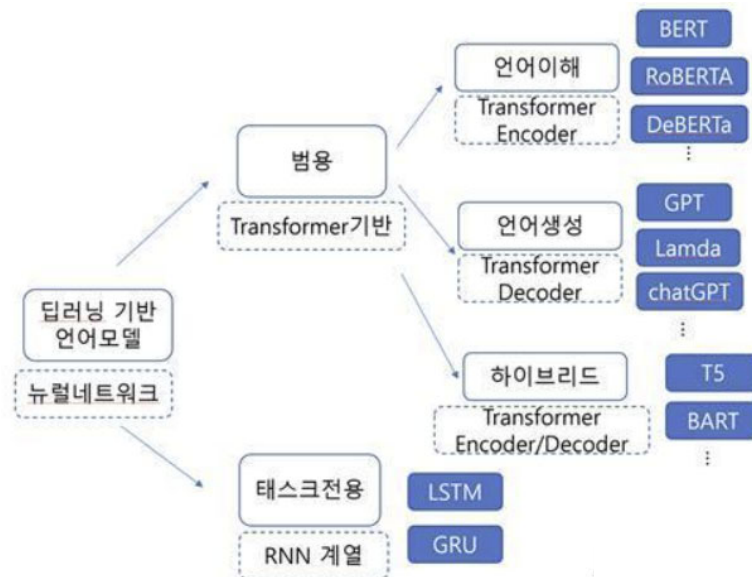
LLM	특성
BERT-Large	2018, Google AI, Transformer 기반의 양방향 모델 전체 문장 내에서 단어의 좌우, 즉, 양방향으로 문맥을 고려하면서 학습하기 때문에, 문장의 의미를 포괄적으로 이해함. 맥락 상 단어의 의미를 파악하는 것이 중요한 자연어 처리 작업에 강점을 지님. Fine-tuning을 통해 다양한 NLP 태스크에 적용 가능 문서 분류, 감성 분석, 개체명 인식, Q&A, 기계 번역 등의 분야에 활용
GPT	2019, OpenAI, Transformer 기반의 자기 회귀 모델 Transformer의 decoder 부분만 중첩된 구조의 모델 일방향으로 나아가면서 다음 단어를 예측하는 방식으로 학습하기 때문에, 순차적으로 일관되게 문장을 생성함. 인간처럼 일관되고 연관성이 높은 언어를 구사하여 대화형 작업에 강점을 지님 큰 규모의 데이터셋에서 미세조정 없이도 강력한 성능을 발휘, 일반화 능력이 뛰어남 문장 완성, 요약, 대화형 챗봇, 창작 글쓰기(소설, 시 등), 프로그래밍 코드 작성 등의 분야에 활용
T5	2019, Google Research Transformer 기반, 모든 NLP 문제를 텍스트 생성 문제로 변환하여 처리 파인튜닝 없이 한 번에 여러 작업을 해결하는 데 잠재력을 보인 최초의 LLM 기계 번역, 요약, 텍스트 분류 등의 분야에 활용

LLM(Large Language Model)

Large Language Model 이해

→ Large Language Model 비교

LLM	특성
RoBERTa	2019, Facebook AI, BERT의 개선된 버전 BERT보다 더 긴 훈련 시간, 더 큰 데이터셋, 더 큰 배치 사이즈로 성능 개선 텍스트 분류, 자연어 이해, 질문응답 등의 분야에 활용
ELECTRA	2020, Google Research, Transformer 기반, Discriminator와 Generator로 구성 기존 BERT보다 훈련 효율성을 높이며, 낮은 계산 비용으로 성능을 향상 감정 분석, 텍스트 분류 등



ChatGPT 모델 이해와 활용



소목차

- GPT 모델 구조와 학습
- 문장 생성과 텍스트 완성
- 다양한 활용 사례
- 고급 활용과 주의 사항

OpenAI API

● OpenAI

- Elon Musk, Sam Altman, Greg Brockman 등의 창업자들에 의해 2015년에 설립
- 자연어 처리, 강화학습, 생성 모델링 등의 기술과 알고리즘을 개발하는 데 중점을 둠
- 자연어 처리 API
- <https://platform.openai.com>

OpenAI API 주요 기능 :

텍스트 생성

이미지 생성

임베딩 생성

파인튜닝

모델링

음성 텍스트 변환(STT)

openai의 ChatGPT API

● OpenAI API

- 자연어 처리 API
- 애플리케이션에 GPT-4, GPT-3.5의 기능을 통합할 수 있습니다.
- OpenAI API 라이브러리가 지원하는 프로그래밍 언어는 Python, Node.js
- <https://platform.openai.com/docs/libraries>
- 적용분야
 - ▶ 텍스트 생성: 기사, 블로그, 이메일 작성, 스토리텔링 등
 - ▶ 질문 응답: FAQ 자동화, 학습 도우미, 고객 지원 챗봇
 - ▶ 번역 및 요약: 텍스트 번역, 문서 요약
 - ▶ 코드 생성 및 디버깅: 코드 생성, 코드 해석, 오류 수정
 - ▶ 창의적 작업: 시나리오 작성, 캐릭터 생성, 게임 시뮬레이션

ChatGPT 모델 이해와 활용

● ChatGPT 모델 이해

→ GPT (Generative Pre-trained Transformer)

▶ Transformer의 디코더(Decoder) 아키텍처를 기반으로 설계

- Encoder는 입력 문장을 종합적으로 이해하고 고차원 표현으로 변환하는 데 적합
- Decoder는 이미 주어진 문맥에서 새로운 텍스트를 생성하는 데 더 적합

▶ 자기회귀적 모델 (Autoregressive)

- 이전의 입력 토큰들을 사용해 다음 토큰을 예측

▶ Causal Masking (인과적 마스킹)

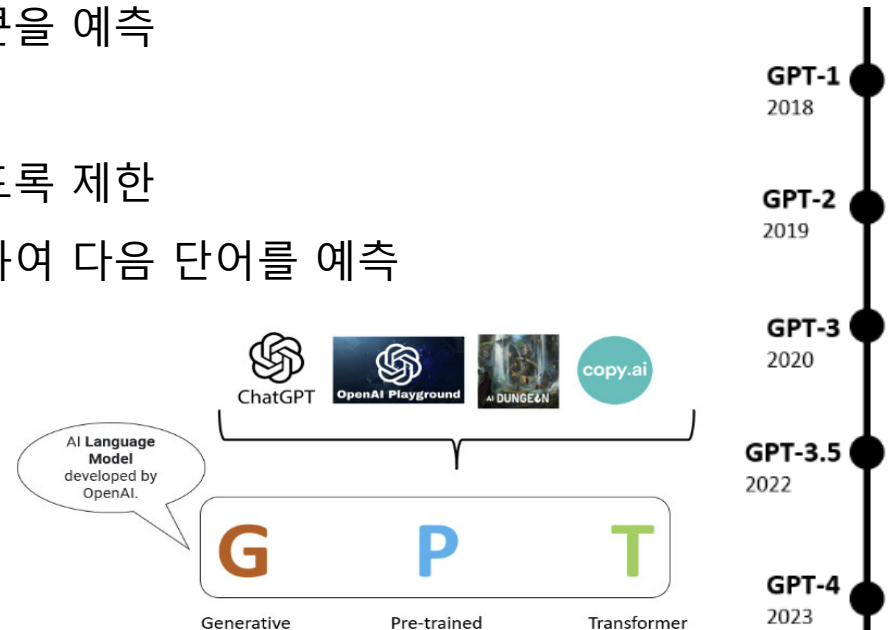
- 입력 시퀀스에서 미래 정보를 보지 않도록 제한
- 모델은 현재 시점 이전의 정보만 이용하여 다음 단어를 예측

▶ 단방향(순방향) Attention

Generative : 다음 단어를 예측, 생성 (언어 모델)

Pre-trained : 많은 양의 데이터를 사전에 훈련

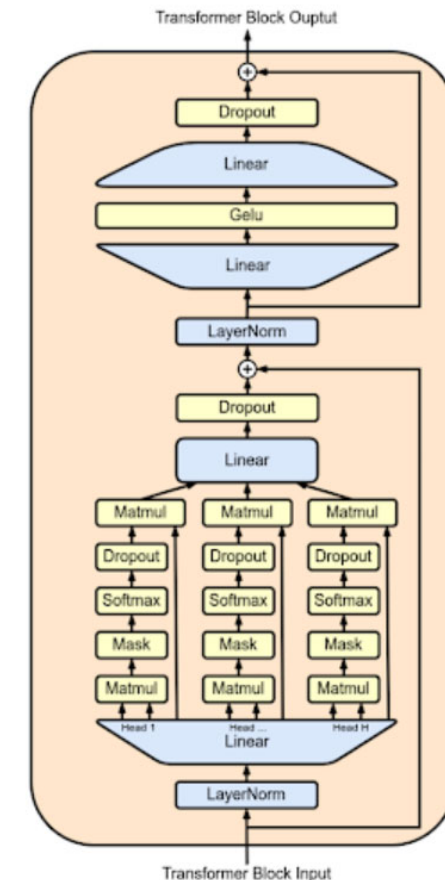
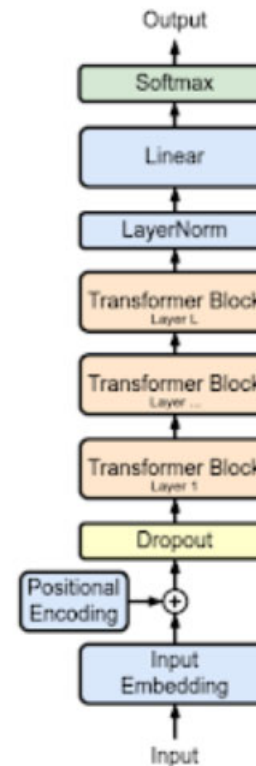
Transformer : 신경망에 Encoder-Decoder



GPT 모델 구조와 학습

● GPT 모델 구조

- Attention Mechanism(Self-Attention)을 활용하여 입력간의 관계를 학습
- Autoregressive Model 모델 : 자신이 생성한 텍스트를 다시 자신의 입력으로 받아 이를 바탕으로 다시 출력 텍스트를 생성



ChatGPT 모델 이해와 활용

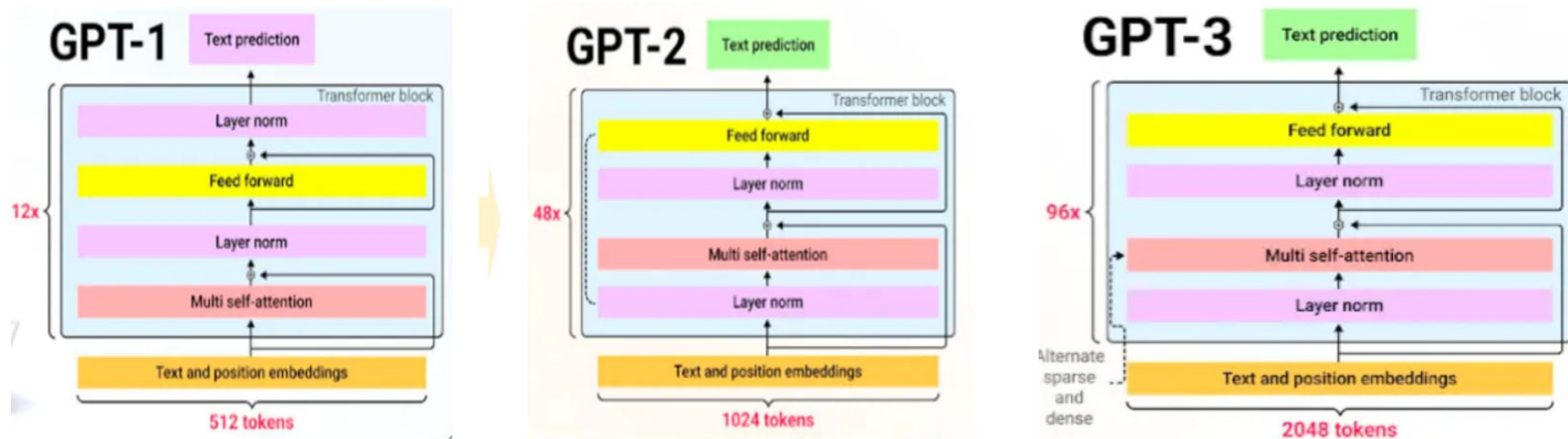
ChatGPT 모델 이해

→ GPT-1 모델 구조

- ▶ Transformer의 디코더 12개를 쌓아올린 구조
- ▶ Transformer의 디코더 중에서 Masked Self-attention과 Feed Forward 네트워크만으로 구성

→ GPT-2 모델 구조

- ▶ GPT-1에 비해 Layer Normalization 위치가 변경
- ▶ 모델 성능을 개선하기 위해 task conditioning, Zero-Shot Learning, Zero Shot Task Transfer를 사용



ChatGPT 모델 이해와 활용

● ChatGPT 모델 이해

→ GPT-3 모델 구조

- ▶ GPT-2와 동일한 모델 구조를 사용
- ▶ GPT-3 파라미터는 125M과 같은 small 버전부터 175B 버전까지 존재함
- ▶ 특정 가중치가 할당된 Common Crawl, WebText2, Books1, Books2 및 Wikipedia 말뭉치의 혼합에 대해 학습
- ▶ zero-shot 및 few-shot 세팅에서 downstream NLP tasks를 잘 수행

→ GPT-3.5 모델 (InstructGPT)

- ▶ 정책을 준수하도록 강제함으로써 AI가 인간의 가치와 일치하는 가이드라인 내에서 동작하도록 학습되었다.
- ▶ fine-tuning 단계는 GPT-3 모델에 RLHF (Reinforcement Learning from Human Feedback) 개념을 도입하였다

Task Conditioning :

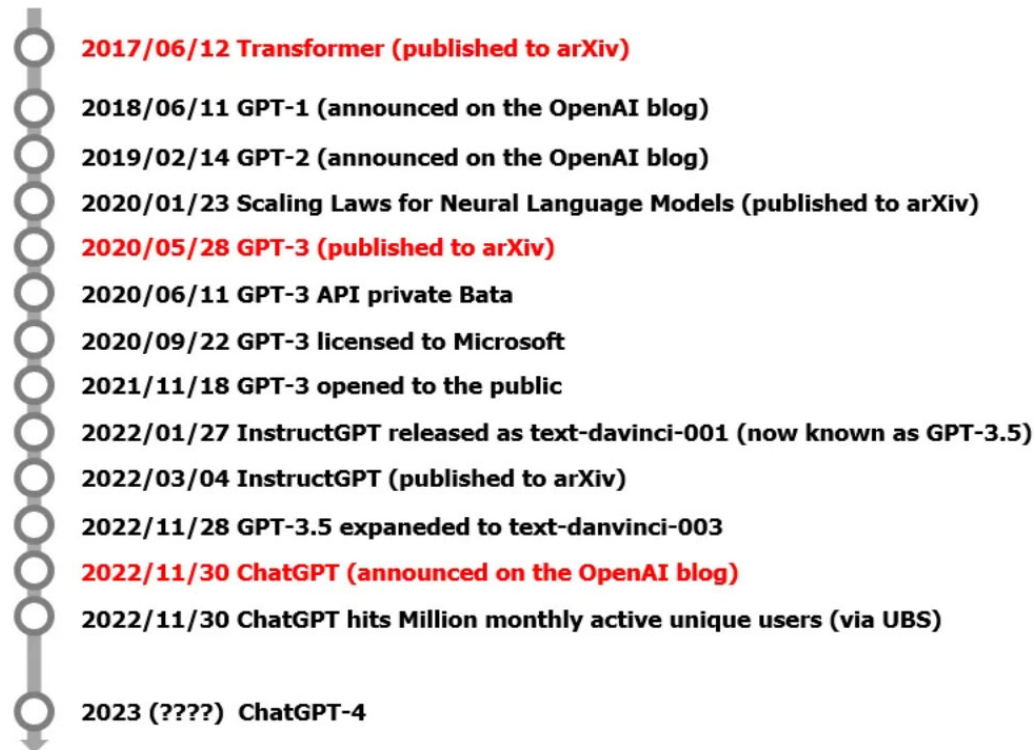
특정 작업에 대한 정보를 입력 시퀀스에 포함시켜 모델이 이를 인식하고 그에 맞게 반응하도록 유도합니다.

ChatGPT 모델 이해와 활용

● ChatGPT 모델 이해

→ ChatGPT 모델 구조

- ▶ Azure 슈퍼컴퓨터에서 학습된 GPT-3.5 모델을 fine-tuning한 모델로 훨씬 더 엄격한 가이드 라인에서 작동하며, 많은 규칙을 강제하고 있다.



ChatGPT 모델 이해와 활용

○ GPT 모델 Layer

→ Input Embedding Layer

- ▶ 입력 텍스트를 고정된 크기의 벡터로 변환
- ▶ GPT는 BPE(Byte Pair Encoding) 같은 토큰라이저를 사용하여 단어를 더 작은 단위로 분해
- ▶ 토큰 임베딩과 위치 임베딩(순서 정보가 유지되도록)을 포함

→ Transformer Block (Decoder Block)

- ▶ 입력 시퀀스 내의 모든 토큰 간의 관계를 모델링하여 각 토큰의 문맥적 의미를 파악
- ▶ **Multi-Head Self-Attention Mechanism** : 여러 개의 어텐션 헤드를 사용하여 다양한 표현을 동시에 학습하고, 각 토큰이 다른 모든 토큰과 어떻게 관련되는지를 평가합니다.
- ▶ **Position-wise Feed-Forward Neural Networks** : 각 위치의 토큰에 대해 독립적으로 작동하는 두 개의 선형 변환과 하나의 비선형 활성화 함수(ReLU)로 구성
 - 모델의 표현력을 향상시킵니다.
 - Feed-Forward Layer는 Attention 출력 결과를 고차원 공간에서 추가로 변환
 - 각 계층에 활성화 함수(ReLU 등)가 포함됩니다

ChatGPT 모델 이해와 활용

● GPT 모델 Layer

- ▶ **Residual Connections and Layer Normalization** : 각 서브 레이어(Attention과 Feed-Forward Network)의 출력에 대해 레이어 정규화를 수행하고, 각 서브 레이어의 입력에 대한 잔차 연결을 추가하여 깊은 네트워크에서도 안정적인 학습을 도모합니다. 각 계층의 출력이 입력에 더해져 정보 손실을 방지하고, 학습 안정성을 높입니다

→ Output Layer

- ▶ 선형 레이어로 구성되며, 소프트맥스 함수를 적용하여 각 토큰의 발생 확률을 계산
- ▶ 높은 확률을 가진 단어가 출력으로 생성

ChatGPT 모델 이해와 활용

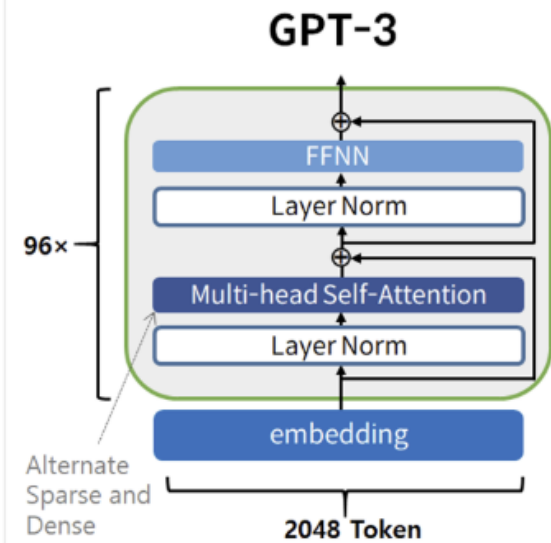
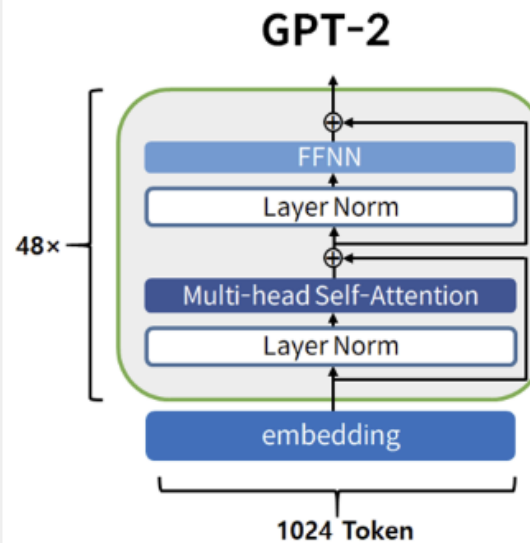
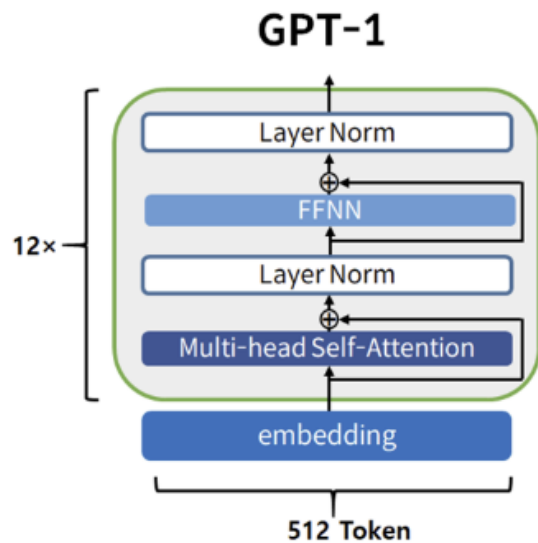
● ChatGPT 모델의 발전

모델	출시연도	파라미터 & 토큰	특징
GPT-1	2018	1억 1,700만 개 512개 토큰 처리 가능 768 은닉층	초기 버전, 단순한 문장 완성 및 질의응답 등의 기능을 수행 12개의 Decoder 층을 가진 트랜스포머 아키텍처를 사용 학습 방식: Unsupervised pre-training with unlabeled data and Supervised fine-tuning with labeled data 특정 task를 위해 매번 Fine-tuning 필요하며, 학습 데이터에 민감
GPT-2	2019	약 15억 개 1024개 토큰 처리 가능 1600 은닉층	학습 데이터: WebText(800만개의 문서, 40GB 용량) 학습 방식: Unsupervised Pre-training and Zero shot Learning 메타 러닝(meta learning)의 일종인 in-context-learning(사전학 습 모델에 task에 대한 텍스트 인풋을 삽입) 48개의 Decoder의 층
GPT-3	2020	약 1,750억 개 2048개 토큰 처리 가능 12288 은닉층	CommonCrawl(Web, e-book, wiki 등 753.4GB) 데이터로부터 5000억 단어 이상의 데이터 세트를 사용하여 훈련된 모델 거대 언어 모델(Large Language Model)의 가능성을 보여준 모 델, 96개의 Decoder의 층 학습 방식: Unsupervised Pre-training and Zero shot, One shot, Few shot Learning 사람처럼 글 작성, 코딩, 번역, 요약 가능 예시를 통해 간접적으로 모델에 지시

ChatGPT 모델 이해와 활용

ChatGPT 모델의 발전

모델	출시 연도	파라미터 수	특징
GPT-3.5	2022	약 1,750억 개	인간 피드백 기반 강화학습(Reinforcement Learning with Human Feedback, RLHF) 적용으로 답변 정확도와 안정성 급증 직접적으로 대화 형태로 지시
GPT-4	2023	약 1조 7000억개	모델 구조/크기, 하드웨어 정보, 데이터 및 모델 학습 방법 비공개 이미지 입력도 받을 수 있는 멀티모달(multimodal) 제공



ChatGPT 모델 이해와 활용

● GPT-1 모델 이해

→ GPT-1 (2018)

- ▶ Labeling이 되어있지 않은 다량의 말뭉치 데이터를 활용하여 사전 훈련
- ▶ 사전 훈련시키기 위하여 BooksCorpus 데이터셋이 활용됨
- ▶ 12 계층의 Transformer Decoder 구조
- ▶ 최초로 텍스트를 읽고 질문에 답변을 할 수 있는 모델로 인정
- ▶ 다양한 서로 다른 Task(자연어 이해, 추론, 분류, Q&A, NER, 감정분석 등)들에 대해서 Zero-Shot 성능을 보여줌
- ▶ 텍스트 기반의 데이터를 좀 더 자연스러운 방식으로 컴퓨터가 이해할 수 있도록 함
- ▶ 생성 언어 모델링이 사전 훈련 아이디어와 결합하면 일반적인 AI의 모습을 보여줄 수 있다는 점을 증명함

ChatGPT 모델 이해와 활용

● GPT-2 모델 이해

→ GPT-2(2019)

- ▶ 15억 개의 파라미터를 보유
- ▶ GPT-1에 비해서 약 10배 더 큰 데이터셋을 활용
- ▶ OpenAI에 의해 구축된 WebText 데이터셋을 학습
- ▶ 사전 훈련된 GPT-2 모델은 다른 추가적인 지도 파인 튜닝이 없이 unsupervised pre-training 만을 통해 zero-shot으로 down-stream task를 수행할 수 있는 범용 언어 모델
- ▶ 총 4개의 다른 파라미터(117M, 345M, 762M, 1.5B)를 가진 모델을 학습하였을 때 파라미터가 커질수록 perplexity가 낮아지는 것을 확인할 수 있었다.

Perplexity : 언어 모델이 얼마나 잘 작동하는지를 측정하는 지표

주어진 텍스트 데이터에 대해 모델이 만들어낸 예측들이 얼마나 '혼란스러운지'를 나타내는 수치

Downstream task : 언어 모델이 사전 훈련된 후 실제로 적용되는 구체적인 작업 (감정 분석, 질문 응답, 요약 작성)

perplexity 가 낮을수록, 모델은 downstream tasks에서 더 나은 성능을 보일 가능성이 높습니다

ChatGPT 모델 이해와 활용

● GPT-3 모델 이해

→ GPT-3 (2020)

- ▶ 약 1750억 개의 파라미터를 가지는 모델
- ▶ 96 decoders, 2048 tokens 생성
- ▶ 학습시키기 위해서 5000억 개의 단어가 포함된 말뭉치를 사용(Common Crawl)
- ▶ GPT-3 모델의 성능으로 모델 파라미터 크기가 클수록 정확도가 향상되고, Few-shot에 따른 성능은 예시가 많을수록 정확도가 향상된다는 것을 확인
- ▶ 특정 가중치가 할당된 5개의 서로 다른 말뭉치의 혼합에 대해 학습 (753GB)
- ▶ 96 decoders, 2048 tokens 생성
- ▶ 기본적인 수식 처리, 코드 구현 등의 태스크를 처리하여, GPT-3는 NLP 태스크 뿐 아니라 다양한 비즈니스 활동을 신속하고 정확하게 보조할 수 있게 되었다

ChatGPT 모델 이해와 활용

● GPT-3.5 모델 이해

→ GPT-3.5 (2022)

- ▶ InstructGPT
- ▶ GPT-3 기반이지만 정책을 준수하도록 강제함으로써 AI가 인간의 가치와 일치하는 가이드라인 내에서 동작하도록 학습되었다.
- ▶ GPT-3을 기본 모델로 GPT-3와 동일한 pre-trained 데이터셋에 피드백 기반 강화학습 (Reinforcement Learning with Human Feedback, RLHF)를 fine-tuning에 도입 적용하여 답변 정확도와 안정성을 높임
- ▶ GPT-3.5는 단어, 문장, 다양한 웹 컴포넌트 사이의 관계들을 학습

ChatGPT 모델 이해와 활용

● In-Context Learning과 Fine-Tuning

→ In-Context Learning:

- ▶ 특별한 학습 과정 없이, 주어진 입력 내에서 직접 문맥 정보를 활용하여 작업을 수행하는 방식
- ▶ 모델이 주어진 예시들을 바탕으로 새로운 입력에 대한 적절한 반응을 생성
- ▶ 입력된 예시의 질과 양에 따라 모델의 성능이 크게 영향을 받습니다.

→ Fine-Tuning:

- ▶ 사전 훈련된 모델을 특정 작업에 맞게 추가적으로 학습시키는 과정
- ▶ 모델은 초기에 학습된 일반적인 지식을 바탕으로, 더 구체적이고 특화된 작업을 더 효과적으로 수행할 수 있습니다.
- ▶ 사전 훈련된 모델의 파라미터를 특정 작업의 데이터에 맞게 조정합니다.
- ▶ 특정 작업에 대해 모델의 정확도와 효율성을 크게 개선합니다.

ChatGPT 모델 이해와 활용

● In-Context Learning과 Fine-Tuning



The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



ChatGPT 모델 이해와 활용

● In-Context Learning

→ Zero-shot Learning

- ▶ 모델이 한 번도 학습하지 않은 새로운 작업이나 개념을 해결하는 능력
- ▶ 생성 AI에게 필요한 데이터나 설명을 덧붙이지 않고 답변을 생성하게 하는 방법
- ▶ 모델의 사전 학습된 능력을 바탕으로 결과를 예측하며, 가중치 업데이트(gradient update)는 수행되지 않습니다

→ One-shot Learning

- ▶ 몇모델은 단일 예시를 기반으로 작업을 추론합니다.
- ▶ 가중치 업데이트는 여전히 수행되지 않습니다.

→ Few-shot Learning

- ▶ 여러 개의 예시를 통해 힌트를 주면서 답변을 생성하는 방법
- ▶ 모델이 새로운 작업을 학습할 때, 소량의 데이터(몇 개의 예제) 만으로도 문제를 해결하는 능력
- ▶ 가중치 업데이트 없이 제공된 문맥 정보만 활용됩니다

ChatGPT 모델 이해와 활용

● GPT-4 모델 이해

→ GPT-4(2023)

- ▶ **Multimodal Model** 및 초거대 언어 모델
- ▶ 텍스트와 이미지를 동시에 이해하고 생성 가능
- ▶ ChatGPT Plus에 탑재되어 있다.
- ▶ 공공 데이터 및 써드파티 공급자로부터 라이선싱 받은 데이터들을 활용
- ▶ OpenAI의 정책을 준수하기 위하여 인간과 AI의 입력을 기반으로 하는 강화 학습을 활용하여 학습이 되었다.
- ▶ 문맥 윈도우의 크기가 8192, 32868 토큰까지 늘어났다.

ChatGPT 모델 이해와 활용

● ChatGPT 모델 학습 방법

→ 사전 학습 (Pre-training)

- ▶ 대규모 텍스트 데이터셋에서 학습하여 언어 패턴, 구문, 문법 등을 사전 학습(Pre-training)합니다
- ▶ 다음 단어를 예측하는 언어 모델링(objective) 수행
- ▶ Cross-Entropy Loss를 사용하여 예측 단어와 실제 단어 간의 차이를 최소화합니다

→ 미세 조정 (Fine-tuning)

- ▶ 사전 학습된 모델을 특정 작업(예: 번역, 요약, 질문 응답)에 맞게 추가 학습합니다
- ▶ 작은 데이터셋에서도 높은 성능을 발휘하도록 설계되었습니다
- ▶ Fine-tuning은 일반적으로 지도 학습(Supervised Learning)으로 수행됩니다

→ 인간 피드백을 통한 강화 학습 (RLHF)

- ▶ 강화 학습과 인간 피드백을 결합하여 모델의 출력을 더 자연스럽고 유용하게 만듭니다.
- ▶ 사람의 평가 데이터를 활용해 보상 모델(Reward Model) 학습
- ▶ 보상 모델을 사용해 강화 학습(RL)을 수행하여 출력 품질을 최적화

openai의 ChatGPT API

● ChatGPT의 API 연동

→ GPT-3.5-turbo와 GPT-4o-mini 비교

특징	GPT-3.5-turbo	GPT-4o-mini
모델 크기	표준 GPT-3.5 모델	작고 경량화된 모델
응답 속도	매우 빠름	빠름
비용	저렴 (비용 효율적)	상대적으로 저렴
문맥 이해	기본적인 문맥 이해	향상된 문맥 이해
응답 품질	높은 품질의 자연어 처리	GPT-4의 일부 기능 포함
적합한 사용 사례	대화형 AI, 고객 서비스 등	저사양 환경, 빠른 응답 필요 시

* 모델 설정값 :

gpt-4

gpt-4-32k (32k 토큰 컨텍스트 윈도우 지원)

gpt-3.5-turbo

gpt-3.5-turbo-16k (16k 토큰 컨텍스트 윈도우 지원)

ChatGPT 모델 이해와 활용

● GPT-3.5-turbo와 GPT-4o-mini 엔진 비교

→GPT-3.5-turbo

- ▶ GPT-3.5 모델의 빠르고 경제적인 버전
- ▶ 주로 대화형 AI 및 실시간 응답이 필요한 애플리케이션에 사용
- ▶ 빠른 응답 속도와 낮은 비용
- ▶ 대화형 상호작용에 최적화되어 있으며, 자연스러운 대화를 지원

→GPT-4o-mini

- ▶ GPT-4의 경량화 버전
- ▶ 더 적은 리소스를 소모하면서도 GPT-4의 성능을 어느 정도 유지함
- ▶ 주로 저사양 환경이나 빠른 응답을 요구하는 애플리케이션에 적합
- ▶ 경량화된 구조로, 더 빠른 응답 속도를 제공
- ▶ GPT-4의 고급 기능을 일부 포함하여, 문맥 이해와 생성 능력이 개선됨

ChatGPT 모델 이해와 활용

● GPT-3.5-turbo와 GPT-4o-mini 엔진 비교

→ GPT-3.5-turbo 사용 사례

- ▶ 고객 지원 챗봇, 가상 비서 등 대화형 응용 프로그램.
- ▶ 콘텐츠 생성, 문서 요약 등 다양한 자연어 처리 작업.
- ▶ 높은 응답 품질이 요구되는 애플리케이션

→ GPT-4o-mini

- ▶ 저사양 장치나 모바일 앱에서 실시간 응답이 필요한 경우.
- ▶ 빠른 프로토타입 개발이나 테스트 환경에서의 사용.
- ▶ 비용 효율적인 솔루션이 필요한 경우

→ 선택 기준

- ▶ 경량화가 중요하고 빠른 응답이 필요한 경우, GPT-4o-mini가 적합
- ▶ 대화형 AI와 같은 보다 복잡한 자연어 처리 작업이 필요하다면 GPT-3.5-turbo 선택

ChatGPT 모델 이해와 활용

○ GPT 활용

→ 고객 서비스

- ▶ 자동 응답 시스템 - 고객의 질문에 신속하게 답변하고, 문제를 해결
고객 대기 시간을 줄이고, 효율적인 서비스 제공이 가능
- ▶ FAQ 처리 - 자주 묻는 질문에 대한 자동 응답을 통해 고객 지원 팀의 부담을 줄여 줌

→ 교육

- ▶ 개인 튜터링 - 학생들이 질문을 하거나 특정 주제에 대해 도움을 요청할 수 있음
ChatGPT는 이해를 돕기 위한 설명과 예제를 제공 함
- ▶ 학습 자료 생성 - 수업 자료, 퀴즈, 요약 등을 자동으로 생성하여 교육자들에게 유용한 도구로 활용 됨
- ▶ 학생들이 자기 주도적으로 학습할 수 있는 환경을 조성하는 데 기여

→ 콘텐츠 생성

- ▶ 블로그 및 기사 작성 - 주제에 대한 글을 작성하거나 아이디어를 제공하여 작가나 블로거의 작업을 지원
- ▶ 소셜 미디어 콘텐츠 - 마케팅 팀이 소셜 미디어 포스트를 작성하거나 캠페인 아이디어를 생성하는 데 도움을 줄 수 있음

ChatGPT 모델 이해와 활용

○ GPT 활용

→ 프로그래밍 및 기술 지원

- ▶ 코드 작성 및 디버깅
- ▶ 기술 문서 작성 - API 문서, 사용자 매뉴얼 등을 자동 생성, 개발자 문서화 작업 지원

→ 개인 비서

- ▶ 일정 관리 - 사용자의 일정이나 할 일을 관리, 알림을 설정하는 등의 역할을 수행
- ▶ 정보 검색 - 사용자가 필요로 하는 정보나 자료를 검색하고 제공

→ 의료 및 상담

- ▶ 의료 정보 제공 - 기본적인 건강 정보나 질병에 대한 설명을 제공
상담과 같은 초기 진단의 지원 역할
- ▶ 정신 건강 지원 - 사용자와 대화를 통해 정서적 지지나 상담을 제공하는 데 활용

→ 문서 요약 및 분석

- ▶ 대량의 문서를 요약, 분석 및 필요한 정보를 빠르게 얻고, 중요한 결정을 내리는 데 활용
- ▶ 리서치 회사는 ChatGPT를 사용하여 방대한 양의 연구 보고서를 요약하고, 주요 포인트를 도출하여 연구원들에게 제공

ChatGPT 모델 이해와 활용

○ GPT 활용

→ 게임 및 엔터테인먼트

- ▶ 사용자와의 대화를 통해 스토리를 전개하는 대화형 게임에서 NPC의 역할을 수행
- ▶ 사용자와의 협업을 통해 이야기나 캐릭터를 창작하는 데 도움을 줄 수 있음

→ 번역 및 언어 학습

- ▶ 여러 언어 간의 번역을 지원하여 사용자에게 다국어 커뮤니케이션을 도움
- ▶ 국제적인 업무 수행 능력을 향상
- ▶ 문법, 어휘, 발음에 대한 질문에 답변하고, 연습 문제를 제공하여 학습을 지원

→ 법률 자문

- ▶ 기본적인 법률 정보 제공, 계약서 초안 작성, 법률 문서 검토 등의 작업에 활용
- ▶ 법률 서비스의 접근성을 높이고, 법률 비용을 절감하는 데 기여

→ 금융 상담

- ▶ 고객에게 개인화된 금융 상담 서비스를 제공
- ▶ 고객의 금융 지식을 향상하고, 더 나은 금융 결정을 내리는 데 도움이 됩니다

ChatGPT 모델 이해와 활용

● ChatGPT 의 한계

→ 정보의 정확성

▶ 불확실한 정보 제공

ChatGPT는 학습한 데이터에 기반하여 응답을 생성함
최신 정보나 특정 세부사항에 대한 정확성을 보장할 수 없음
2023년 이후의 사건이나 데이터는 반영되지 않을 수 있음

▶ 허위 정보 생성

일부 경우, 사실이 아닌 정보를 생성할 수 있으며, 이는 사용자에게 혼란을 줄 수 있음

→ 문맥 이해의 한계

▶ 긴 대화에서의 문맥 유지

ChatGPT는 긴 대화에서 문맥을 완벽하게 유지하지 못할 수 있음
이전의 대화 내용을 잊거나 잘못 이해할 수 있어, 일관성이 떨어지는 응답을 할 가능성이 있음

▶ 모호한 질문 처리

질문이 애매하게 표현되거나 불명확할 경우, 적절한 답변을 제공하지 못할 수 있음

ChatGPT 모델 이해와 활용

● ChatGPT 한계

→ 감정 및 사회적 맥락 이해 부족

▶ 감정 인식의 한계

ChatGPT는 사용자 감정을 인식하거나 적절한 감정적 반응을 제공하는 데 한계가 있음
이는 인간과의 대화에서 중요한 요소인 감정적 지지를 제공하기 어려움을 의미함

▶ 사회적 규범 이해 부족

특정 문화적 맥락이나 사회적 규범을 이해하지 못해 부적절한 응답을 할 수 있음

→ 창의성의 한계

▶ 창의적 작업의 제약

ChatGPT는 기존의 데이터를 기반으로 한 응답을 생성
완전히 새로운 아이디어나 창의적인 솔루션을 제공하는 데 한계가 있음

▶ 모방적인 응답

생성하는 텍스트는 주어진 입력에 기반하여 만들어짐. 종종 기존의 패턴이나 스타일을 모방하게 됨

→ 사용자 맞춤화의 한계

▶ 개별 사용자에게 대한 이해 부족

ChatGPT는 각 사용자의 배경, 선호도, 특정 요구사항을 이해하고 맞춤화된 응답 제공에 한계 존재
동일한 질문에 대해 다양한 사용자에게 동일한 응답을 제공할 수 있음

ChatGPT 모델 이해와 활용

● ChatGPT 한계

→ 비즈니스 및 산업적 활용의 제약

▶ 법적 및 규제 문제

ChatGPT의 사용이 특정 산업에서 법적 또는 규제 문제를 일으킬 수 있음
이로 인해 상업적 활용에 제약이 있을 수 있음

▶ 전문 분야의 한계

특정 전문적인 분야(예: 의학, 법률)에서는 안정성과 정확성을 보장하기 어려움
전문가의 검토가 필요한 경우가 많음

→ 사용자와 개발자는 이러한 한계를 인식하고, AI의 사용을 적절히 조절해야 함

ChatGPT API

● ChatGPT의 API 연동

→ 요청 매개변수 설정

- ▶ `model` : 사용할 모델 지정. " gpt-3.5-turbo" 모델 사용
- ▶ `content` : ChatGPT에 전달할 입력 텍스트

OpenAI API 문서
(<https://platform.openai.com/docs/libraries>)

→ API 응답 처리

- ▶ API 요청에 대한 응답은 JSON 형식으로 반환 됨
- ▶ 응답에서 `response.choices[0].content`를 사용하여 생성된 텍스트를 추출

→ 다양한 요청 및 응답 처리

- ▶ 다양한 유형의 대화와 작업을 수행 가능

→ 예외 처리

- ▶ API 요청 중에 발생할 수 있는 예외 상황을 처리하는 코드 추가

→ 보안 및 비용 관리

- ▶ API 키를 보안 유지하고, 사용량을 모니터링하고 비용을 관리

→ 테스트 및 디버깅

- ▶ 요청과 응답을 테스트하고 디버깅하여 ChatGPT를 올바르게 통합

ChatGPT API

● ChatGPT의 API 연동

→ OpenAI 계정 및 API 키 생성

- ▶ OpenAI 웹사이트(<https://openai.com/blog/openai-api>)에 가입하고 로그인
- ▶ 대시보드에서 API 키를 생성하고 해당 키를 안전한 곳에 보관

→ 필요한 라이브러리 설치

- ▶ `pip install openai`

```
import openai
```

```
# API 키 설정
```

```
api_key = "YOUR_API_KEY_HERE"
```

ChatGPT API

● Model endpoint compatibility

→ <https://platform.openai.com/docs/models#model-endpoint-compatibility>

→ post 요청 endpoint url → <https://api.openai.com/v1/chat/completions>

Endpoint	Latest models
/v1/assistants	모든 GPT-4o (chatgpt-4o-latest 제외), GPT-4o-mini, GPT-4, GPT-3.5 Turbo 모델들이 지원됩니다. 검색 도구는 gpt-4-turbo-preview, gpt-3.5-turbo-1106 모델
/v1/audio/transcriptions	whisper-1
/v1/audio/translations	whisper-1
/v1/audio/speech	tts-1, tts-1-hd
/v1/chat/completions	GPT-4o(Realtime preview 제외), GPT-4o-mini, GPT-4, GPT-3.5 Turbo 모델들과 그들의 날짜가 명시된 릴리스들. chatgpt-4o-latest 다이내믹 모델. gpt-4o, gpt-4o-mini, gpt-4, 그리고 gpt-3.5-turbo의 파인 튜닝 버전들
/v1/completions (Legacy)	gpt-3.5-turbo-instruct, babbage-002, davinci-002
/v1/embeddings	text-embedding-3-small, text-embedding-3-large, text-embedding-ada-002
/v1/fine_tuning/jobs	gpt-4o, gpt-4o-mini, gpt-4, gpt-3.5-turbo

ChatGPT API

● Model endpoint compatibility

Endpoint	Latest models
/v1/moderations	text-moderation-stable, text-moderation-latest
/v1/images/generations	dall-e-2, dall-e-3
/v1/realtime (beta)	gpt-4o-realtime-preview, gpt-4o-realtime-preview-2024-10-01

ChatGPT API

● POST 요청 Assistants API 실습

- POST 요청을 `https://api.openai.com/v1/assistants` 엔드포인트에 보내는 것은 주로 새로운 AI 어시스턴트를 생성하는 데 사용됩니다.
- 어시스턴트는 다양한 기능을 수행할 수 있지만, 그 기능은 사용자가 설정하고 정의한 파라미터와 구성에 따라 달라집니다.

```
import requests
url = "https://api.openai.com/v1/assistants"
headers = {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json"
}
data = {
    "name": "Your Assistant Name",
    "description": "A brief description of what this assistant is for",
    "model": "gpt-3.5-turbo"
}
response = requests.post(url, headers=headers, json=data)
print(response.text)
```

ChatGPT API

● POST 요청 Assistants API 실습

→ 텍스트 요약 기능을 수행할 수 있는 Assistants 생성

```
import requests
url = "https://api.openai.com/v1/assistants"

headers = {
    "Authorization": "Bearer YOUR_API_KEY", # 여기에 실제 API 키를 입력하세요.
    "Content-Type": "application/json"
}

data = {
    "name": "Text Summarizer Assistant",
    "description": "An assistant designed to summarize text",
    "model": "gpt-3.5-turbo", # 요약 작업에 적합한 모델을 선택하세요.
    "instructions": "Summarize the text"
}

response = requests.post(url, headers=headers, json=data)
print(response.text)
```

ChatGPT API

● Create chat completion

- post 요청 URL `https://api.openai.com/v1/chat/completions`
- `client.chat.completions.create ()` : 채팅 세션을 시작
- `messages`: 대화 내역을 나타내는 메시지 리스트
 - ▶ 각 메시지는 메시지를 보내는 주체를 나타내는 `role`과 메시지의 내용 `content` 필드를 가집니다

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "developer", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello!"}
    ]
)

print(completion.choices[0].message)
```

ChatGPT API

● Create chat completion의 response

```
{
  "id": "chatcmpl-123",          #응답의 고유 식별자
  "object": "chat.completion",  #응답 객체의 유형, 채팅 완성에 대한 응답임을 표시
  "created": 1677652288,        #객체가 생성된 시간, 유닉스 타임스탬프 형식으로 제공
  "model": "gpt-4o-mini",       #응답을 생성하는 데 사용된 모델의 이름
  "system_fingerprint": "fp_44709d6fcb", #시스템의 특정 상태나 설정을 나타내는 지문
  "choices": [{                 #생성된 응답들을 포함하는 배열
    "index": 0,                 #선택된 응답의 인덱스
    "message": {
      "role": "assistant",      #응답을 보낸 역할
      "content": "\n\nHello there, how may I assist you today?", #실제 응답 내용
    },
    "logprobs": null,           #응답 생성 시 로그 확률 정보
    "finish_reason": "stop"     #응답 생성이 종료된 이유, "stop"은 모델이 자체적으로 응답을
                                #종료했음을 나타냅니다.
  }],
  "service_tier": "default",    #사용된 서비스 등급
}
```


ChatGPT API

● Create chat completion의 response

```
" "usage": {                                #요청 및 응답 생성에 사용된 토큰 수를 세부 내용 설명
  "prompt_tokens": 9,                        #프롬프트에 사용된 토큰 수
  "completion_tokens": 12,                  #응답에 사용된 토큰 수
  "total_tokens": 21,                      #전체 토큰 사용량
  "completion_tokens_details": {           #추가적인 세부사항
    "reasoning_tokens": 0,                 #추론 토큰
    "accepted_prediction_tokens": 0,       #수용된 예측 토큰
    "rejected_prediction_tokens": 0       #거부된 예측 토큰 수
  }
}
```

ChatGPT API

● Create chat completion

```
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "What's in this image?"},
                {
                    "type": "image_url",
                    "image_url": {
                        "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg",
                    }
                },
            ],
        },
    ],
    max_tokens=300,
)
print(response.choices[0])
```

ChatGPT API

● 스트리밍 방식으로 대화형 AI 모델의 응답 처리

스트리밍 옵션 모델의 응답을 실시간으로 여러 부분으로 나누어서 받을 수 있으며, 각 부분이 준비되는 즉시 처리할 수 있습니다.

특히 긴 대화나 실시간 인터랙션이 필요한 애플리케이션에 유용합니다.

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "developer", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello!"}
    ],
    stream=True #응답이 스트리밍 방식으로 전달
)
#응답이 준비되는 대로 바로 받아볼 수 있습니다.
# completion은 반복 가능한 객체로, 생성된 각 응답 조각(chunk)을 순회
for chunk in completion:
    print(chunk.choices[0].delta)
# delta는 모델이 생성한 변경 사항(응답의 일부분)을 포함합니다.
```

ChatGPT API

● 특정 도구(tools)를 활용하는 채팅 세션 생성

```
from openai import OpenAI
client = OpenAI()
tools = [ { "type": "function",
            "function": {
                "name": "get_current_weather",
                "description": "Get the current weather in a given location",
                "parameters": { "type": "object",
                                "properties": { "location": {"type": "string",
                                                            "description": "The city and state, e.g. San Francisco, CA",
                                                            "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]},
                                                            },
                                "required": ["location"],
                                },
                            },
            }
        ]
messages = [{"role": "user", "content": "What's the weather like in Boston today?"}]
completion = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tools,
    tool_choice="auto" )
print(completion)
```

ChatGPT API

● 특정 도구(tools)를 활용하는 채팅 세션 생성

GPT-4o 모델로부터 생성된 응답의 로그 확률(log probabilities) 정보도 함께 요청

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Hello!"}
    ],
    logprobs=True,          #답 생성 시 로그 확률 정보를 요청하는 옵션
    top_logprobs=2          #최대 몇 개의 상위 로그 확률을 반환할지 지정
)

print(completion.choices[0].message) #응답의 메시지 내용을 출력
print(completion.choices[0].logprobs) #응답의 로그 확률 정보를 출력
```

ChatGPT API

● 텍스트 생성 실습

→ <https://platform.openai.com/docs/guides/completions>

- ▶ 자유 형식의 텍스트 문자열인 프롬프트를 입력으로 사용
- ▶ 모델에 전달하고 싶은 텍스트를 하나의 연속된 문자열로 제공
- ▶ 특정한 질문에 대한 답변, 글 작성, 코드 생성 등 다양한 텍스트 생성 작업에 적합
- ▶ 단발성 질의응답 또는 특정 문제에 대한 해결책 제시에 사용됨
- ▶ OpenAI Python SDK의 일부로, 주로 GPT-3 또는 GPT-3.5와 같은 모델을 이용한 자연어 처리 작업에 사용됩니다

ChatGPT API

● 텍스트 생성 실습

→ **프롬프트 엔지니어링** : 모델로부터 올바른 출력을 얻기 위해 프롬프트를 제작하는 과정

모델에 정확한 지시사항, 예시 및 필요한 맥락 정보(모델의 훈련 데이터에 포함되지 않은 개인적이거나 전문적인 정보 등)를 제공함으로써 모델의 출력 품질과 정확성을 향상시킬 수 있습니다.

chat completions API에서는 모델에 대한 지시를 포함하는 메시지 배열을 제공함으로써 프롬프트를 만듭니다.

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)

response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt="커피 전문점을 위한 tagline 을 작성해주세요"
)
response_text = response.choices[0].text
print("응답 텍스트:", response_text.strip())
```

```
Completion(id='cmpl-AmzW401EBWTDCH7LOfUuyvuKt7Ib', choices=[CompletionChoice(finish_reason='length', index=0, logprobs=None, text='\n\n1. "따뜻한 마음으로  
품')], created=1736239576, model='gpt-3.5-turbo-instruct', object='text_completion', system_fingerprint=None, usage=CompletionUsage(completion_tokens=16,  
prompt_tokens=21, total_tokens=37, completion_tokens_details=None, prompt_tokens_details=None))
```

client.completions.create() 메서드는 openai.Completion 객체를 반환

ChatGPT API

Completion.create() 매개변수

→ <https://platform.openai.com/docs/guides/text-generation>

매개변수	설명
model	사용할 모델 이름 (예: "text-davinci-003", "text-curie-001")
prompt	입력 프롬프트 텍스트 생성의 시작점
temperature	출력 텍스트의 창의성 조정 (0~2, 기본값: 1) 낮은 값은 더 예측 가능하고 집중된 응답 생성 높은 값은 더 창의적이고 예측 불가능한 응답 생성 0에 가까운 값 : 모델이 가장 확률이 높은 토큰을 선택 정확성이나 예측 가능성이 중요한 작업에 사용 예: 코드 생성, 간단한 요약, 공식적 응답 1 (기본값) : 모델이 어느 정도 창의성과 다양성을 보여줍니다. 일반적인 텍스트 생성 작업에 적합 예: 스토리 생성, 마케팅 문구 작성 2에 가까운 값 : 모델이 더 무작위로 동작하며, 창의적이고 예상치 못한 응답을 생성 응답의 품질이 낮아질 수도 있지만, 실험적이고 창의적인 작업에 유용 예: 시 쓰기, 창의적 아이디어 생성
max_tokens	생성할 텍스트의 최대 길이(토큰 수)

ChatGPT API

Completion.create() 매개변수

매개변수	설명
frequency_penalty	이미 생성된 단어의 반복을 억제 값 범위: -2.0 ~ 2.0 -2.0에 가까운 값 : 단어 반복에 대한 제한이 거의 없음 모델이 동일한 단어를 자주 반복할 수 있습니다 예: 정형화된 텍스트 생성 0 (기본값) : 단어 반복에 특별한 제한이 없음 2.0에 가까운 값 : 반복된 단어 사용을 강하게 억제 응답이 다양해지지만, 비문법적 문장이 생성될 가능성도 있음
presence_penalty	새로운 단어 또는 아이디어의 도입을 증가 값 범위: -2.0 ~ 2.0 -2.0에 가까운 값 : 모델이 새로운 단어 도입을 억제합니다 주제에 충실하며, 보수적이고 반복적인 응답을 생성 0 (기본값) : 새로운 단어와 기존 단어 사이의 균형을 유지합니다 2.0에 가까운 값 : 모델이 새로운 단어와 주제를 적극적으로 도입하려 합니다 창의적인 작업에 적합하지만, 응답이 주제에서 벗어날 가능성도 있습니 다
top_p	확률 분포의 상위 p%에 해당하는 토큰만 사용 확률이 높은 단어만 고려하도록 제어

ChatGPT API

● Completion.create() 매개변수

매개변수	설명
n	생성할 결과 수 (기본값 : 1)
stream	진행 상황을 스트리밍으로 반환할지 여부 (기본값 : false)
logprobs	확률 분포의 상위 p%에 해당하는 토큰만 사용
echo	결과와 함께 프롬프트를 에코 백(기본값 : false)
stop	토큰 생성을 중지하는 문장. 최대 4개
best_of	서버 측에서 best_of 개 만큼의 결과를 생성해서 가장 좋은 결과를 반환(기본값 : 1)
logit_bias	지정한 토큰의 가능성 감소
user	최종 사용자 ID

ChatGPT API

● 질의 응답 실습

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)
prompt = "인공지능에 대해 알려주세요"
response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt=prompt,
    temperature=0,
    max_tokens=500
)

response_text = response.choices[0].text
print("응답 텍스트:", response_text.strip())
```

ChatGPT API

● 요약 실습

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)
prompt = """인공지능(Artificial Intelligence, AI)은 인간의 지능을 모방하거나
대체하는 기술을 말합니다. 인공지능은 컴퓨터 프로그램이나 시스템을 통해 인간의 학습,
추론, 문제 해결 등의 지능적인 작업을 수행할 수 있도록 만들어진 기술입니다. 인공지능은
크게 강한 인공지능과 약한 인공지능으로 나뉩니다. 강한 인공지능은 인간과 동일하거나 그
이상의 지능을 가지며, 모든 종류의 작업을 수행할 수 있습니다. 반면 약한 인공지능은
특정한 작업에만 특화된 지능을 가지며, 인간의 지능과는 차이가 있습니다.
인공지능은 다양한 분야에서 활용되고 있습니다. 예를 들어 음성 인식 기술을 이용한
인공지능 스피커, 이미지 인식 기술을 이용한 얼굴 인식 소프트웨어, 자율주행 자동차 등이
있습니다. 또한 인공지능은 의료, 금융, 교육 등 다양한 분야에서도 활용되고 있으며, 더
나은 서비스를 제공하기 위해 계속 발전하고 있습니다.
하지만 인공지능은 아직 완벽하지 않으며, 인간의 지능을 완전히 대체할 수는 없습니다.
따라서 인공지능을 개발하고 활용하는 과정에서 윤리적인 문제나 안전 문제 등을 고려해야
합니다. \n\n Summarize the above text. """
response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt=prompt,
    temperature=0,
    max_tokens=500
)
#prompt 문자열의 끝에 모델에게 텍스트 요약 작업을 명확하게 지시
response_text = response.choices[0].text
print("요약 텍스트:", response_text.strip())
```

ChatGPT API

● 번역 실습

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)
prompt = """ 한국어를 영어로 번역합니다.
            한국어 : 대한민국의 민주주의는 국민이 지킵니다.
            영어 : """
response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt=prompt,
    temperature=0,
)

response_text = response.choices[0].text
print("번역 텍스트:", response_text.strip())
```

ChatGPT API

Completion.create() 의 model

model	설명
text-davinci-003 text-davinci-002 text-davinci-001 text-davinci	GPT-3 계열중 가장 강력한 언어 모델 정교한 언어 생성과 복잡한 작업 처리 능력을 보유 자연어 처리(NLP) 전반에 걸쳐 사용 가능 복잡한 프롬프트를 처리하며, 고품질의 텍스트를 생성 긴 문장을 이해하고 일관된 응답을 제공 이야기 작성, 시 쓰기, 창의적 콘텐츠 생성에 탁월 논리적 추론이 필요한 작업, 고급 요약, 번역, 스토리 생성 작업에서 주로 사용 사용 비용이 높음
text-curie-003 text-curie-002 text-curie-001 text-curie	davinci보다 가벼운 모델로 간단한 질의응답, 기본 텍스트 생성에 적합 빠르고 효율적인 작업 수행에 적합 정확성과 속도 간의 균형을 제공 실시간 처리에 적합 비용이 저렴하여 대규모 작업에서 경제적 감정 분석, 중급 수준의 콘텐츠 생성, 대화형 AI 챗봇 작업에서 주로 사용

ChatGPT API

● Completion.create() 의 model

model	설명
gpt-4o	매우 높은 수준의 지능과 강력한 성능을 제공하지만, 토큰 당 비용이 더 높습니다.
gpt-4o-mini	더 빠르고 토큰 당 비용이 적습니다
o1	추론 모델, 결과를 반환하는 속도가 느리고 "생각"을 위해 더 많은 토큰을 사용하지만, 고급 추론, 코딩, 다단계 계획 수행이 가능합니다.

* model 파라미터 설정 가능한 값 : ('-001'이 붙지 않은 모델은 파인 튜닝 전 사전 학습된 모델)

* 사실 기반 정확하고 간결한 텍스트 :

temperature=0.2 frequency_penalty=0 presence_penalty=0

* 창의적이고 독창적인 텍스트 생성 (시, 창작 스토리) :

temperature=1.5 frequency_penalty=0.5 presence_penalty=1.5

* 중복을 피하면서 적절히 창의적인 텍스트 생성 :

temperature=1.0 frequency_penalty=1.0 presence_penalty=1.0

ChatGPT API

● Completion의 Response Attributes

응답(response) 속성	설명
id	생성된 응답의 고유 ID
object	객체 유형 ("chat.completion")
created	응답 생성 시점 (UNIX 타임스탬프)
model	사용된 모델 이름
choices	생성된 텍스트 리스트 text : 모델이 생성한 텍스트 index : 응답의 순서 logprobs : 선택된 토큰의 확률 (옵션) finish_reason : 응답 종료 이유 ("stop", "length", 등)
usage	API 호출에서 사용된 토큰 수 prompt_tokens: 입력 프롬프트에 사용된 토큰 수 completion_tokens: 생성된 응답의 토큰 수 total_tokens: 전체 토큰 수 (prompt_tokens + completion_tokens)

ChatGPT API

● Chat Completions API

→ <https://platform.openai.com/docs/guides/text-generation>

- ▶ 프롬프트를 통해 모델들은 코드, 수학 방정식, 구조화된 JSON 데이터, 시, 산문과 같은 거의 모든 종류의 텍스트 응답을 생성할 수 있습니다.
- ▶ 메시지 목록을 입력으로 사용합니다. 각 메시지는 역할(role)과 내용(content)을 가지며, 이는 대화형 세션을 구성하는 데 사용됩니다.
- ▶ 대화형 어플리케이션에 최적화되어 있으며, 사용자와의 연속적인 상호작용을 모델링할 수 있습니다.
- ▶ 대화의 흐름을 이해하고 그에 맞게 응답을 생성하는 데 유리합니다.

→ 텍스트 생성 요청 단계

1. 어떤 모델을 사용하여 응답을 생성할지 설정(선택하는 모델은 출력에 큰 영향을 미치며, 각 생성 요청의 비용에도 영향을 줍니다.)
2. 프롬프트 작성
3. 모델에 특정 유형의 출력을 요청하는 지시사항을 포함한 ChatGPT에 입력할 수 있는 사용자 메시지를 작성

ChatGPT API

● Chat Completion API

- Completion API를 사용한 텍스트 생성의 입출력은 '텍스트->텍스트' 인 반면 채팅의 입출력은 '채팅 메시지 리스트 -> 채팅 메시지'가 됩니다
- 채팅 메시지에는 'role'(역할)과 'content'(콘텐츠)가 포함됩니다.

role	설명
system	채팅 AI의 행동에 대한 지시
user	인간의 발화
assistant	AI의 발언

ChatGPT API

● ChatCompletion 클래스와 함수

클래스 및 함수	설명
ChatCompletion	OpenAI의 ChatGPT API에서 채팅 기반 작업을 수행하기 위한 클래스 모델과 대화를 주고받으며 텍스트 생성, 질의응답, 텍스트 완성 등 다양한 작업을 처리
ChatCompletion.create()	API 호출을 통해 모델로부터 응답을 생성 response["choices"][0]["message"]["content"] 객체 반환

```
import openai

openai.api_key = "api_key"
response = openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "당신은 유용한 조수입니다."},
        {"role": "user", "content": "AI가 무엇인가요??"}
    ],
    temperature=0.7,
    max_tokens=150,
)

print("Assistant Response:")
print(response.choices[0].message)
```

ChatGPT API

● ChatCompletion 클래스와 함수

→ <https://platform.openai.com/docs/guides/gpt/chat-completions-api>

매개변수	설명
model	사용할 모델 이름 (예: "gpt-3.5-turbo", "gpt-4").
messages	대화 기록을 관리하는 리스트. 각 메시지는 role(발화자)과 content(내용)으로 구성
role	모델의 동작 방식이나 역할을 정의 "system": 모델의 초기 지침 제공 "user": 사용자 입력 "assistant": 모델이 생성한 응답
temperature	텍스트 생성의 창의성 수준. (0~2 사이의 값, 기본값: 1)
max_tokens	응답에서 생성할 최대 토큰 수
top_p	확률 분포를 자르는 임계값, 샘플링의 다양성을 제어
frequency_penalty	이미 생성된 단어의 반복을 제어
presence_penalty	새로운 주제나 단어 사용 가능성을 제어
logit_bias	지정한 토큰의 가능성 감소
stop	토큰 생성을 중지하는 문장, 최대 4개
n	생성할 결과 수 (기본값 : 1)

ChatGPT API

● ChatCompletion 클래스 와 함수

응답(response) 속성	설명
id	생성된 응답의 고유 ID
object	객체 유형 ("chat.completion")
created	응답 생성 시점 (UNIX 타임스탬프)
model	사용된 모델 이름
choices	모델의 생성 결과 리스트 각 choice는 message와 finish_reason 속성으로 구성 message : 모델이 생성한 메시지 (role 및 content 포함) finish_reason : 응답 생성 종료 이유 (예: "stop", "length")
usage	API 호출에서 사용된 토큰 수 prompt_tokens: 입력 프롬프트에 사용된 토큰 수 completion_tokens: 생성된 응답의 토큰 수 total_tokens: 전체 토큰 수 (prompt_tokens + completion_tokens)

ChatGPT API

● 프로그램 코드 생성 실습

```
from openai import OpenAI
OPENAI_API_KEY = "sk-proj-7cJkcA....."

client = OpenAI(api_key=OPENAI_API_KEY)

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "developer", "content": "You are a helpful assistant."},
        {
            "role": "user",
            "content": """"매개변수의 값이 짝수인지 홀수인지 판별하는 파이썬 함수를
완성해주세요
def evenCheck(num) :""""
        ]
)

print(completion.choices[0].message)
print(completion.choices[0].message.content)
```

ChatGPT API

● Chat 대화 실습

```
import openai

context = [
    {"role": "system", "content": "당신은 유용한 조수입니다."},
    {"role": "user", "content": "지도학습은 무엇인가요?"}
]
# 첫 번째 질문
response = openai.chat.completions.create (
    model="gpt-4o-mini",
    messages=context,
    max_tokens=150
)
print( response.choices[0].message.content)
context.append({"role": "assistant", "content": response.choices[0].message.content})
# 두 번째 질문
context.append({"role": "user", "content": "지도 학습에 대해 예제 코드를 만들어 주세요"})
response = openai.chat.completions.create (
    model="gpt-4",
    messages=context,
    max_tokens=150
)
print("Assistant Response:")
print(response.choices[0].message.content)
```

ChatGPT API

● ChatCompletion 클래스 와 함수

→ ChatCompletion.create()의 장점과 한계

- ▶ 대화 문맥을 유지하며, 복잡한 대화 기반 작업에 적합.
- ▶ 다양한 파라미터를 통해 출력 텍스트 제어 가능.
- ▶ 간단한 API 호출로 강력한 자연어 생성 성능 제공.

- ▶ 높은 토큰 수 제한(4096~32,768 토큰)을 초과할 경우 문맥 손실
- ▶ 반복적인 대화에서 모델이 불필요한 정보 반복 가능
- ▶ 응답 시간 및 비용이 사용량에 따라 증가

ChatGPT API

● Completion 클래스의 system role 예시

- ➔ 특정 상황이나 요구에 맞게 행동하도록 AI 시스템(예를 들어 챗봇이나 대화형 인공지능)의 역할을 설정하는 지침
 - ▶ 예의 바르고 격려적인 방식으로 응답하도록 지시합니다.
 - ▶ 상세하고 기술적인 답변을 제공하도록 요구합니다.
 - ▶ 모든 응답에서 객관성을 유지하도록 합니다.
 - ▶ 셰익스피어 작품의 등장인물처럼 응답하도록 지시합니다.
 - ▶ 100단어를 넘지 않는 간단한 답변을 요구합니다.

"You are a friendly and helpful assistant. Please respond in a polite and encouraging manner."

"You are an expert in computer science and should provide detailed, technical answers."

"Please remain neutral and unbiased in your responses."

"Respond in the style of a Shakespearean character."

"Please provide concise answers. No longer than 100 words."

"You are a personal trainer helping someone who is new to fitness. Keep your advice simple and easy to follow."

"Provide thorough explanations with examples whenever a question requires a detailed answer."

"Offer supportive and comforting responses to users who may be feeling upset or stressed."

ChatGPT API

● Edit 클래스 와 함수

클래스 및 함수	설명
Edit	사용자 입력 텍스트를 기반으로 텍스트를 수정하는 데 사용 문서 교정, 코드 리팩토링, 콘텐츠 개선 등 다양한 작업에 활용
Edit.create()	사용자 입력 텍스트를 기반으로 모델이 제안한 수정 사항을 반환 model (필수) : 사용할 모델의 이름을 지정 일반적으로 text-davinci-edit-001 또는 code-davinci-edit-001을 사용합니다 input (선택) : 편집을 수행할 원본 텍스트 입력이 없으면 빈 문자열로 간주됩니다. instruction (필수) : 텍스트에서 수행할 작업을 설명하는 지시문 예: "Fix the grammar" 또는 "Convert this into a formal tone" temperature (선택) : 생성된 결과의 창의성 수준을 조정 값 범위: 0에서 1 (낮은 값은 더 결정적, 높은 값은 더 창의적), 기본값: 1 top_p (선택) : 생성된 결과에서 높은 확률을 가진 토큰의 누적 확률을 기반으로 출력을 제한 값 범위: 0에서 1. 기본값: 1

ChatGPT API

● Edit 클래스 와 함수

매개변수	설명
model	사용할 모델 이름 (예: "text-davinci-edit-001", code-davinci-edit-001).
input	편집할 텍스트
instruction	편집 방법
temperature	텍스트 생성의 창의성 수준. (0~2 사이의 값, 기본값: 1)
top_p	확률 분포를 자르는 임계값, 샘플링의 다양성을 제어 temperature와 동시에 변경하는 것은 권장하지 않음
n	생성할 결과 수 (기본값 : 1)

```
import openai
openai.api_key = "API_KEY"

response = openai.Edit.create(
    model="text-davinci-edit-001",
    input="The cat was jump on the table.",
    instruction="Fix the grammar",
    temperature=0.5
)

print(response["choices"][0]["text"])
```

ChatGPT API

● Edit 클래스의 instruction 예시

문법 및 철자 교정 :

"Correct grammar and spelling errors." (문법 및 철자 오류 수정)

"Fix typos and improve the clarity." (오타자를 수정하고 명확성을 개선합니다)

스타일 및 톤 조정 :

"Make the text more formal." (텍스트를 더 형식적으로 만듭니다.)

"Rewrite this in a casual tone." (이 글을 캐주얼한 톤으로 다시 쓰십시오.)

"Add a professional touch to the text." (텍스트에 전문적인 감각을 더하세요)

요약 및 구조화 :

"Summarize this text in two sentences." (이 텍스트를 두 문장으로 요약하세요)

"Organize the content into bullet points." (콘텐츠를 총알 모양으로 구성합니다.)

문장 재작성 :

"Rewrite the text to make it more concise." (텍스트를 더 간결하게 다시 작성합니다.)

"Expand this into a detailed explanation." (자세한 설명으로 확대하세요)

번역 :

"Translate this text into French." (이 텍스트를 프랑스어로 번역합니다)

"Convert this text to English from German." (이 텍스트를 독일어에서 영어로 변환합니다.)

창의적인 작업 :

"Reimagine this as a poem." (이것을 시로 재구성하세요)

"Make this text sound like it was written in the 18th century." (이 텍스트를 18세기에 쓰여진 것처럼 보이게 하세요.)

코드 관련 작업 :

"Refactor this Python code for readability." (파이썬 코드를 리팩터화해 가독성을 확보하세요)

"Add comments to the code explaining each step." (각 단계를 설명하는 코드에 댓글을 추가합니다)

"Fix syntax errors in this JavaScript code." (이 자바스크립트 코드의 구문 오류 수정)

ChatGPT API

● ChatCompletion으로 문장 교정 실습

```
import openai

OPENAI_API_KEY = "sk-pr9dq4vApXF27cJkcA...."
openai.api_key = OPENAI_API_KEY

with open('./data/sample.txt', 'r', encoding='utf-8') as file:
    file_content = file.read()

response = openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[ {"role": "system", "content": "You are a creative writer."},
               {"role": "user", "content": f"Correct the grammar: {file_content}"} ],
    temperature=0.7,
    max_tokens=150,
    top_p=1.0,
    frequency_penalty=0.0,
    presence_penalty=0.0
)

print("Original Text:\n", file_content)
print(response.choices[0].message.content)
```

ChatGPT API

● Image generation

- 텍스트 프롬프트를 기반으로 처음부터 이미지를 생성합니다 (DALL·E 3 및 DALL·E 2 사용 가능)
- 기존 이미지의 일부 영역을 새 텍스트 프롬프트를 기반으로 이미지의 편집된 버전을 생성합니다 (DALL·E 2만 해당)
- 기존 이미지의 변형을 생성합니다 (DALL·E 2만 해당)
 - ▶ DALL·E 3를 사용할 때는 이미지 크기를 1024x1024, 1024x1792 또는 1792x1024 픽셀로 설정할 수 있습니다.
 - ▶ 이미지는 표준 품질로 생성되지만, DALL·E 3를 사용하는 경우 품질을 "hd"로 설정하여 세부 사항을 강화할 수 있습니다. 정사각형, 표준 품질 이미지는 생성 속도가 가장 빠릅니다.
 - ▶ DALL·E 3로 한 번에 1개의 이미지를 요청할 수 있습니다(병렬 요청을 통해 더 많이 요청 가능)하거나 DALL·E 2를 사용하여 n 매개변수로 한 번에 최대 10개의 이미지를 요청할 수 있습니다.

ChatGPT API

● Image 클래스 와 함수

클래스 및 함수	설명
Image	텍스트 프롬프트를 기반으로 이미지를 생성, 제공된 이미지와 마스크를 기반으로 특정 영역을 편집, 입력된 이미지에서 다양한 변형 버전을 생성 작업을 수행하는 클래스
Image.create()	텍스트 기반 이미지 생성 작업을 수행 prompt (필수) : 생성하려는 이미지의 텍스트 설명 n : 생성할 이미지의 개수 (기본값: 1, 최대값: 10). size : 이미지 크기 (기본값: "1024x1024", 지원되는 값: "256x256", "512x512", "1024x1024") response_format : 반환 형식 ("url" 또는 "b64_json", 기본값: "url") user (선택) : 요청을 추적하기 위한 고유 사용자 식별자

ChatGPT API

● Image 클래스 와 함수

→ <https://platform.openai.com/docs/guides/images>

매개변수	설명
prompt	이미지 설명. 최대 길이 1000자
size	생성하는 이미지 크기(256x256/512x512/1024x1024, 기본값1024x1024)
response_format	생성하는 이미지 형식(url/b64_json)
user	최종 사용자 ID
n	생성할 이미지 수 (1~10개, 기본값 : 1)
mask	마스크 이미지, 편집 영역에 투명 색상을 지정. 4MB 미만의 정사각형 PNG(RGBA)
image	편집할 이미지. 4MB 미만의 정사각형 PNG(RGBA)

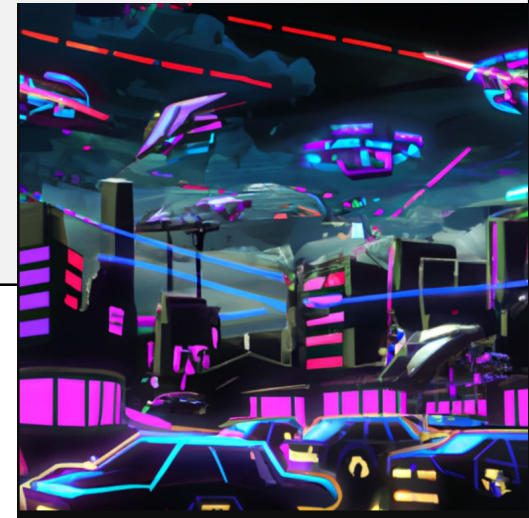
ChatGPT API

◉ Image generation 실습

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)

response = client.images.generate(
    model="dall-e-2",
    prompt="A futuristic cityscape with flying cars and neon lights",
    size="512x512",
    quality="standard",
    n=1,
)

print(response.data[0].url)
```



<https://oaidalleapiprodscus.blob.core.windows.net/private/org-pBx3BsTH1IAVWp18cVjEfghb/user-ZWbFV7MmPg4jWJPBZUBCpaje/img-Bm7qVOAv5b020uogQ4qQ6LTF.png?st=2025-01-07T23%3A29%3A20Z&se=2025-01-08T01%3A29%3A20Z&sp=r&sv=2024-08-04&sr=b&rscd=inline&rsct=image/png&skoid=d505667d-d6c1-4a0a-bac7-5c84a87759f8&sktid=a48cca56-e6da-484e-a814-9c849652bcb3&skt=2025-01-08T00%3A18%3A00Z&ske=2025-01-09T00%3A18%3A00Z&sks=b&skv=2024-08-04&sig=LUnWJnqqeYdEB1QIMC6naeInTOPLhZDVvPF%2Bn3gALc8%3D>

ChatGPT API

● masked Image edit 실습

클래스 및 함수	설명
Image.create_edit()	기존 이미지를 편집하거나 특정 부분을 변경 image(필수) : 편집할 이미지 파일(PNG 형식). mask(필수) : 변경할 영역을 지정하는 마스크 이미지(PNG 형식). prompt(필수): 이미지에 반영할 텍스트 설명. n, size, response_format, user

```
from openai import OpenAI
client = OpenAI()

response = client.images.edit(
    model="dall-e-2",
    image=open("sunlit_lounge.png", "rb"),
    mask=open("mask.png", "rb"),
    prompt="A sunlit indoor lounge area with a pool containing a flamingo",
    n=1,
    size="1024x1024",
)

print(response.data[0].url)
```

제품 이미지를 수정하여 새로운 스타일 적용 작업에 활용

ChatGPT API

● Image Variation 실습

클래스 및 함수	설명
Image.create_variation()	주어진 이미지의 다양한 변형 버전을 생성 image(필수) : 변형할 이미지 파일(PNG 형식) n, size, response_format, user

```
from openai import OpenAI
client = OpenAI()

response = client.images.create_variation(
    model="dall-e-2",
    image=open("corgi_and_cat_paw.png", "rb"),
    n=1,
    size="1024x1024"
)

print(response.data[0].url)
```

실험적인 디자인 및 브랜딩 작업에 활용

ChatGPT API

○ Tokenizer

- LLM에서는 텍스트를 토큰이라는 최소 단위로 분할해서 처리합니다.
- **tiktoken**은 OpenAI 모델과 사용할 수 있는 빠른 BPE 토크나이저 라이브러리입니다
 - ▶ <https://github.com/openai/tiktoken>
- BPE(Byte Pair Encoding) 특성
 - ▶ 역변환이 가능하고 손실이 없어 원본 텍스트로 토큰을 다시 변환할 수 있다.
 - ▶ 토크나이저의 훈련 데이터에 없는 임의의 텍스트에서도 작동함
 - ▶ 텍스트를 압축(토큰 시퀀스는 원본 텍스트에 해당하는 바이트보다 짧으며 각 토큰은 평균적으로 약 4바이트에 해당함)
 - ▶ 모델이 일반적인 부분 단어를 인식할 수 있도록 합니다. (예 : "ing"는 영어에서 흔한 부분 단어이므로 BPE 인코딩은 종종 "encoding"을 "encod"와 "ing"와 같은 토큰으로 분리)

ChatGPT API

● Tokenizer 인코딩 방식

→ LLM의 인코딩은 텍스트가 토큰으로 분할되는 규칙으로 모델마다 사용하는 인코딩이 다름

Encoding Method	설명
Byte-Pair Encoding (BPE)	데이터 압축 기법으로 사용되었지만, 자연어 처리에서 자주 등장하는 바이트 쌍을 반복적으로 병합하여 보다 큰 블록을 형성합니다. GPT 시리즈에서 주로 사용됩니다.
WordPiece	구글의 번역 시스템에서 사용된 방식 자주 등장하는 문자의 조합을 식별하고, 이를 단일 토큰으로 결합하여 어휘 사전을 구성BERT 및 관련 모델에서 널리 채택
Unigram Language Model Tokenizer	토큰의 확률적 언어 모델을 기반으로 하여 가장 가능성이 높은 토큰을 선택합니다. 주로 SentencePiece 라이브러리에서 구현되어 있으며, 다양한 언어에 대해 효과적으로 작동할 수 있도록 설계되었습니다.
Character-Level Tokenization	텍스트를 개별 문자로 분리합니다. 언어의 구조적 복잡성을 최대한 단순화시키지만, 모델이 더 많은 데이터를 처리해야 하기 때문에 계산 비용이 더 높을 수 있습니다.
Subword Regularization	훈련 중에 입력 문장의 토큰화 방법을 무작위로 변형하여, 모델이 다양한 토큰화 패턴에 적응하도록 합니다. 기계 번역과 같은 분야에서 모델의 일반화 능력을 향상시킴

ChatGPT API

● Tokenizer 인코딩 모델

Encoding Model	설명
cl100k_base	"cl100k" 시리즈에 속하는 기본 설정 큰 언어 모델이나 복잡한 언어 처리 작업을 위해 설계됨 "100k"는 사용되는 토큰의 수나 다양성을 의미함 텍스트를 토큰으로 변환하는 데 필요한 패턴, 순위, 특수 토큰 등의 정보를 포함함
p50_base	"p50" 시리즈의 기본 인코딩 설정 특정 도메인이나 용도에 맞춰 최적화된 텍스트 인코딩 방식 "p50"은 토큰의 범위나 모델의 특화된 용도를 나타냄
gpt2	OpenAI의 GPT-2 언어 모델의 인코딩 모델 BPE(Byte Pair Encoding)를 사용하여 텍스트를 토큰화 자주 사용되는 문자나 문자열을 기반으로 텍스트를 효율적으로 압축하고, 모델이 언어의 구조를 더 잘 학습할 수 있게 돕습니다.

ChatGPT API

● titoken 라이브러리 주요 함수

클래스와 함수	설명
get_encoding(name)	인코딩 이름을 매개변수로 전달하면, 해당 인코딩 설정에 따라 초기화된 Encoding 객체를 반환
Encoding 클래스	텍스트를 토큰으로 변환하는 데 필요한 모든 설정을 포함합니다. 특정 패턴, 특수 토큰, 병합 가능한 순위 등을 설정하여 인스턴스화할 수 있습니다. 기본 제공 인코딩 외에 사용자 정의 인코딩을 생성할 수 있다.
encode(text)	텍스트 문자열을 입력 받아 해당 인코딩 방식에 따라 토큰의 시퀀스로 변환
decode(tokens)	토큰의 시퀀스를 입력으로 받아 원래의 텍스트 문자열로 복원 토큰화된 데이터를 다시 사람이 읽을 수 있는 형태로 변환
add_special_token(token, id)	새로운 특수 토큰을 인코딩 설정에 추가합니다. 특수 토큰은 주로 텍스트 내에서 특별한 의미를 가지는 문자열을 처리할 때 사용되며, 고유한 식별자(id)와 함께 등록됩니다.

ChatGPT API

● Tokenizer 실습

```
#pip install tiktoken

import tiktoken
#인코딩 모델 로드
enc = tiktoken.get_encoding("cl100k_base")
#인코딩 실행
tokens= enc.encode("Hello World")
print(len(tokens))
print(tokens)

#디코딩 실행
print(enc.decode(tokens))
#분할된 상태로 디코딩을 실행
print(enc.decode_tokens_bytes(tokens))
```

▶ 한국어 토큰나이저 인코딩 실습

ChatGPT API

● Embedding

- Embedding은 자연어 처리 및 머신러닝 분야에서 클러스터링, 추천 등 데이터 분석에 활용
- 텍스트를 벡터(부동소수점 배열) 표현으로 변환
 - ▶ 벡터 표현은 유사한 의미를 가진 단어나 문장은 벡터 거리가 가깝고, 유사하지 않은 의미를 가진 단어나 문장은 벡터 거리가 멀어지도록 설계되어 있습니다
- <https://platform.openai.com/docs/guides/embeddings>

활용 분야 :

- * 검색 : 쿼리 문자열과의 연관성에 따라 결과의 순위를 매김
- * 클러스터링 : 텍스트의 연관성을 기준으로 그룹화
- * 추천 : 텍스트의 연관성을 기준으로 추천
- * 이상치 탐지 : 유사도가 거의 없는 이상치 타지
- * 다양성 측정 : 유사도 분포 분석
- * 분류 : 텍스트가 가장 유사한 라벨에 따라 분류

ChatGPT API

● Embedding 클래스 와 함수

model	설명
text-embedding-3-small	높은 효율성을 지닌 소형 임베딩 모델 text-embedding-ada-002에 비해 성능이 향상됨 다중 언어 검색 벤치마크(MIRACL)에서 평균 점수가 31.4%에서 44.0%로 상승, MTEB 평균 점수가 61.0%에서 62.3%로 향상되었습니다. 이전 text-embedding-ada-002 모델 대비 5배 저렴하며, 1,000 토큰당 \$0.00002 효율적인 성능과 비용 절감을 동시에 달성하여, 다양한 애플리케이션에 적합 소규모 프로젝트나 비용을 중시하는 사용자에게 적합하도록 설계
text-embedding-3-large	큰 용량과 더 높은 계산 자원을 사용하여, 더 정교하고 깊이 있는 텍스트 분석을 제공 최대 3,072차원의 임베딩을 생성 MIRACL에서 평균 점수가 54.9%, 영어 작업 벤치마크(MTEB)에서 평균 점수가 64.6%로, 이전 모델보다 우수한 성능을 보임 1,000 토큰당 \$0.00013 임베딩 차원을 조절하여 성능과 비용의 균형을 맞출 수 있음
text-embedding-ada-002	

ChatGPT API

● Embedding 클래스 와 함수

클래스 및 함수	설명
Embedding	주어진 입력 텍스트에 대해 고차원 벡터 표현(임베딩)을 생성 베딩을 사용하면 텍스트를 고차원 벡터 공간에서 의미론적으로 유사한 텍스트끼리 가까운 위치에 배치 입력 텍스트의 의미와 구조를 벡터로 변환하여 컨텍스트를 캡처 유사도 비교, 검색 최적화, 추천 엔진, 군집화, 주제 분석 등 다양한 응용 사례에 사용됩니다.
Embedding.create()	주어진 입력 텍스트에 대해 임베딩을 생성 model : 사용할 모델의 이름(예: "text-embedding-3-small"). input : 임베딩을 생성할 텍스트 또는 텍스트 리스트. 임베딩 결과와 메타데이터 반환 user : 선택적 매개변수로, 사용자 정의 데이터를 제공할 때 사용 **kwargs : 추가적인 선택적 매개변수들을 키워드 인자로 전달 (특정 API 기능을 설정하거나 구성할 때 사용)

ChatGPT API

● Embedding 실습

```
import openai
from openai import OpenAI

## openai apikey 입력
OPENAI_API_KEY = "sk-proj-hbsi47ndwg_2l1zuYAaOp9ODzrIP8P_lebAfJdVE9WAXeGz0whJkcA....."

client = openai.OpenAI(api_key=OPENAI_API_KEY)

text = "영화 안중근"
response = client.embeddings.create(
    input=text,
    model="text-embedding-3-small"
)

embedding_result = response.data[0].embedding
print(len(embedding_result))
print(embedding_result)
```

▶ `response = requests.post('https://api.openai.com/v1/embeddings', headers=headers, json=json_data)` 를 사용한 REST API의 채팅 완성 엔드포인트를 사용 한 embedding 실습

ChatGPT API

● Embedding 기반 유사도 검색 실습

→ 페이스북이 개발한 오픈소스 벡터 데이터베이스 Faiss를 이용해 유사도 검색을 수행

- ▶ pip install faiss-cpu
- ▶ pip install faiss-gpu
- ▶ pip install faiss-gpu[cuda101]
- ▶ pip install faiss-gpu[cuda110]

```
import openai
from openai import OpenAI

## openai apikey 입력
OPENAI_API_KEY = "sk-proj-hbsi47ndwg_2l1zuYAa0p9ODzEM 4vApXF27cJkcA....."

client = openai.OpenAI(api_key=OPENAI_API_KEY)

in_text = "오늘은 눈이 오지 않아서 다행입니다"

response = client.embeddings.create(
    input=in_text,
    model="text-embedding-3-small"
)
```

ChatGPT API

● Embedding 기반 유사도 검색 실습

→ 페이스북이 개발한 오픈소스 벡터 데이터베이스 Faiss를 이용해 유사도 검색을 수행

```
import numpy as np
in_embeds = [ record.embedding for record in response.data ]
ndarray_embeds = np.array(in_embeds).astype("float32")
#임베딩을 다양한 수치 계산과 머신러닝 라이브러리에서 효율적으로 사용하기 위해서 np.array로 변환
target_texts = [
    "좋아하는 음식은 무엇인가요?",
    "어디에 살고 계신가요?",
    "아침 전철은 혼잡하네요",
    "오늘안 날씨가 화창합니다",
    "요즘 경기가 좋지 않습니다."
]
response2 = client.embeddings.create(
    input=target_texts,
    model="text-embedding-3-small"
)

target_embeds = [ record.embedding for record in response2.data ]
ndarray_embeds2 = np.array(target_embeds).astype("float32")
import faiss
#Faiss의 인덱스 생성
index = faiss.IndexFlatL2(1536)
```

ChatGPT API

● Embedding 기반 유사도 검색 실습

→ 페이스북이 개발한 오픈소스 벡터 데이터베이스 Faiss를 이용해 유사도 검색을 수행

```
#인덱스에 추가하는 임베딩은 numpy의 float32여야 합니다.  
index.add(ndarray_embeds2)  
#유사도 검색 수행  
D, I = index.search(ndarray_embeds, 1)  
print(D)      #D (Distances): 쿼리 벡터와 해당 가장 가까운 이웃들 간의 거리를 포함한 배열  
print(I)      #I (Indices): 쿼리 벡터와 가장 가까운 이웃 벡터의 인덱스를 포함한 배열  
print(target_texts[I[0][0]])
```

faiss.IndexFlatL2는 FAISS 라이브러리에서 제공하는 인덱스 객체로, 임베딩 벡터들 간의 유클리디안 거리(L2 distance)를 기반으로 유사도 계산을 수행할 수 있게 해줍니다.
faiss.IndexFlatL2는 고차원 공간에서 벡터 간의 거리를 빠르게 계산하고, 가장 유사한 벡터들을 찾는데 사용됩니다.
L2 거리는 두 벡터 간의 직선 거리를 계산하며, 이 거리가 작을수록 두 벡터는 서로 더 유사하다고 간주됩니다.

ChatGPT API

● Audio Generation

- 텍스트 본문의 요약을 음성 오디오로 생성하기 (텍스트 입력, 오디오 출력)
- 녹음에서 감정 분석 수행하기 (오디오 입력, 텍스트 출력)
- 모델과의 비동기 음성 대 음성 상호작용 수행하기 (오디오 입력, 오디오 출력)
- HTTP 클라이언트를 사용하여 REST API를 사용하거나, OpenAI의 공식 SDK를 사용할 수 있습니다.
- GPT-4o-audio-preview 모델은 텍스트 질문에 대해 오디오 형식으로 응답을 생성합니다.
 - ▶ 텍스트 입력 → 텍스트 + 오디오 출력
 - ▶ 오디오 입력 → 텍스트 + 오디오 출력
 - ▶ 오디오 입력 → 텍스트 출력
 - ▶ 텍스트 + 오디오 입력 → 텍스트 + 오디오 출력
 - ▶ 텍스트 + 오디오 입력 → 텍스트 출력
- base64 모듈은 바이너리 데이터와 ASCII 문자열 간의 인코딩 및 디코딩을 처리합니다.
- <https://platform.openai.com/docs/guides/audio>

ChatGPT API

🔴 Audio output from model 실습

```
import base64
from openai import OpenAI

client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o-audio-preview",
    modalities=["text", "audio"],
    audio={"voice": "alloy", "format": "wav"},
    messages=[
        {
            "role": "user",
            "content": "Is a golden retriever a good family dog?"
        }
    ]
)

print(completion.choices[0])

wav_bytes = base64.b64decode(completion.choices[0].message.audio.data)
with open("dog.wav", "wb") as f:
    f.write(wav_bytes)
```

ChatGPT API

🔴 Audio input to model 실습

```
import base64
import requests
from openai import OpenAI
client = OpenAI()
# Fetch the audio file and convert it to a base64 encoded string
url = "https://openaiassets.blob.core.windows.net/$web/API/docs/audio/alloy.wav"
response = requests.get(url)
response.raise_for_status()
wav_data = response.content # 바이너리 형태의 오디오 데이터 응답
# 오디오 데이터를 Base64 문자열로 인코딩합니다.
encoded_string = base64.b64encode(wav_data).decode('utf-8')
completion = client.chat.completions.create(
    model="gpt-4o-audio-preview",
    modalities=["text", "audio"],
    audio={"voice": "alloy", "format": "wav"},
    messages=[ { "role": "user",
                  "content": [ { "type": "text",
                                "text": "What is in this recording?" },
                              { "type": "input_audio",
                                "input_audio": { "data": encoded_string,
                                                  "format": "wav" } }
                            ]
                }, ]
)
print(completion.choices[0].message)
```

ChatGPT API

● Text to speech

- TTS 모델을 활용하여 텍스트 기반 콘텐츠를 음성으로 변환
- <https://platform.openai.com/docs/guides/text-to-speech>
- 다양한 언어로 제공하며, 스트리밍 기능을 통해 즉각적인 오디오 피드백을 사용자에게 제공
 - ▶ 6개의 내장된 목소리를 제공 (alloy, ash, coral, echo, fable, onyx, nova, sage and shimmer)
- 실시간 오디오 스트리밍을 지원합니다.
- 지원되는 출력 형식 기본 응답 형식은 "mp3"이지만, "opus", "aac", "flac", "pcm"과 같은 다른 형식도 사용할 수 있습니다.
- 지원되는 언어 TTS 모델은 일반적으로 Whisper 모델의 언어 지원을 따릅니다.

ChatGPT API

● Text to speech 실습

```
from pathlib import Path  #파일 시스템 경로를 객체 지향적으로 다루기 위한 클래스
from openai import OpenAI

client = OpenAI()
#상위 디렉토리에서 "speech.mp3"라는 이름의 파일 경로를 설정
speech_file_path = Path(__file__).parent / "speech.mp3"
response = client.audio.speech.create(
    model="tts-1",
    voice="alloy",
    #음성으로 변환할 텍스트를 입력
    input="Today is a wonderful day to build something people love!",
)
response.stream_to_file(speech_file_path) #변환된 음성 데이터를 스트림으로 받아 지정된 경로에
파일로 저장
```

ChatGPT API

● Speech to text

- 'large-v2 Whisper 모델'을 기반으로 'transcriptions'(오디오 파일의 내용을 듣고 그 내용을 텍스트로 변환)과 'translations' (오디오를 듣고 그 내용을 다른 언어로 변환) 제공
- 파일 업로드는 현재 최대 25MB로 제한되어 있으며, 지원하는 입력 파일 유형에는 mp3, mp4, mpeg, mpga, m4a, wav, webm이 포함됩니다.
- 기본적으로 Whisper API는 제공된 오디오를 텍스트로 transcript 합니다
 - ▶ 비디오 편집에 단어 수준의 정밀도를 가능하게 하며, 개별 단어에 연결된 특정 프레임의 제거를 허용합니다.
- timestamp_granularities[] 파라미터를 사용하면 더 구조화되고 타임스탬프가 찍힌 JSON 출력 형식을 활성화할 수 있습니다
- Whisper API는 25MB 이하의 파일만 지원합니다. 만약 25MB보다 큰 오디오 파일이 있다면, 파일을 25MB 이하로 나누거나 압축된 오디오 형식을 사용해야 합니다
 - ▶ PyDub이라는 오픈 소스 파이썬 패키지를 사용하여 오디오를 분할
- <https://platform.openai.com/docs/guides/speech-to-text>

ChatGPT API

● Speech to text Transcriptions 실습

```
from openai import OpenAI
client = OpenAI()

audio_file= open("/path/to/file/audio.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file
)
print(transcription.text)
```

```
from openai import OpenAI
client = OpenAI()

audio_file = open("/path/to/file/speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file,
    response_format="text"
)

print(transcription.text)
```

ChatGPT API

● Speech to text Translations 실습

```
from openai import OpenAI
client = OpenAI()

audio_file = open("/path/to/file/german.mp3", "rb")
transcription = client.audio.translations.create(
    model="whisper-1",
    file=audio_file,
)
print(transcription.text)
```

```
from openai import OpenAI
client = OpenAI()

audio_file = open("/path/to/file/speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    file=audio_file,
    model="whisper-1",
    response_format="verbose_json",
    timestamp_granularities=["word"]
)
print(transcription.words)
```

ChatGPT API

● Moderation

- 텍스트나 이미지가 잠재적으로 해로운지 여부를 확인하는 데 사용할 수 있는 도구
- 콘텐츠를 필터링하거나 문제를 일으키는 사용자 계정에 개입하는 등의 시정 조치를 취할 수 있습니다.
- Moderation 엔드포인트는 무료로 사용할 수 있습니다.
- omni-moderation-latest 모델 : 모든 스냅샷은 더 많은 분류 옵션과 멀티 모달 입력을 지원합니다.
- text-moderation-latest (Legacy) 모델 : 오직 텍스트 입력만 지원하고 입력 분류가 더 적은 구형 모델입니다.
- <https://platform.openai.com/docs/guides/moderation>

ChatGPT API

● Moderation 실습

```
from openai import OpenAI
client = OpenAI()

response = client.moderations.create(
    model="omni-moderation-latest",
    input="...text to classify goes here...",
)
print(response)
```

```
from openai import OpenAI
client = OpenAI()
response = client.moderations.create(
    model="omni-moderation-latest",
    input=[ {"type": "text", "text": "...text to classify goes here..."},
            { "type": "image_url",
              "image_url": { "url": "https://example.com/image.png",
                             # can also use base64 encoded image URLs
                             # "url": "data:image/jpeg;base64,abcdefg..." }
            }, ],
)
print(response)
```

ChatGPT API

○ 추론 모델

- o1 시리즈 모델들은 복잡한 추론을 수행하기 위해 강화 학습으로 훈련된 새로운 대규모 언어 모델입니다.
- o1 모델들은 대답하기 전에 생각하며, 사용자에게 응답하기 전에 긴 내부 사고 과정을 거칩니다.
- o1 모델 : 세계에 대한 광범위한 일반 지식을 사용하여 어려운 문제에 대해 추론하도록 설계되었습니다.
- o1-mini 모델 : o1의 더 빠르고 저렴한 버전으로, 광범위한 일반 지식이 필요하지 않은 코딩, 수학, 과학 작업에서 특히 능숙합니다.
- o1 및 o1-mini 모델은 각각 200,000개와 128,000개의 토큰에 대한 컨텍스트 윈도우를 제공합니다.
- 최대 출력 토큰 한도는 o1: 최대 100,000 토큰, o1-mini: 최대 65,536 토큰
- <https://platform.openai.com/docs/guides/reasoning>

ChatGPT API

● reasoning 실습

```
from openai import OpenAI
client = OpenAI()
prompt = """
Write a bash script that takes a matrix represented as a string with
format '[1,2],[3,4],[5,6]' and prints the transpose in the same format.
"""

response = client.chat.completions.create(
    model="o1",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)
print(response.choices[0].message.content)
```

- Code Refactoring
- o1 모델에 파이썬 프로젝트 계획 및 제작을 요청
- o1 모델에 기초 과학 연구와 관련된 질문하기

ChatGPT API

● Fine-Tuning

- LLM을 특정한 작업이나 상황에 맞게 조금 더 훈련시키는 과정
- 전이 학습의 한 형태로 모델을 특정 분야나 작업에 최적화시키기 위해 추가적인 학습을 시키는 과정
- 데이터는 '질문-답변' 세트 형식으로 준비
- <https://platform.openai.com/docs/guides/fine-tuning/preparing-your-dataset>
- 사전 학습된 가중치를 초기화 값으로 사용하며, 상대적으로 적은 데이터를 사용해 특정 목표를 달성하도록 조정
- 데이터의 품질, 모델의 크기, 작업의 특성에 따라 적절한 파인튜닝 기법을 선택하면, 높은 성능과 효율성을 얻을 수 있습니다.

사전 학습(Pre-training)

대규모 텍스트 데이터를 사용해 모델이 언어의 일반적인 패턴, 문법, 상식 등을 학습
비지도 학습 방식으로 주로 수행

GPT, BERT

ChatGPT API

○ FineTuning

- 사전 학습된 모델을 기반으로 개별 작업에 맞게 추가 학습을 하는 것
- 파인 튜닝은 few-shot learning을 개선하여 프롬프트에 담을 수 있는 것보다 훨씬 많은 예제로 훈련함으로써, 다양한 작업에 대해 더 나은 결과를 달성할 수 있게 합니다.
- 모델이 파인 튜닝되면, 프롬프트에 많은 예제를 제공할 필요가 없고, 비용을 절약하고 요청의 지연 시간을 낮출 수 있습니다.
- <https://platform.openai.com/docs/guides/fine-tuning>

파인 튜닝 단계 :

1. 훈련 데이터 준비 및 업로드
2. 새로운 파인 튜닝된 모델 훈련
3. 결과 평가 및 필요 시 1단계로 돌아감
4. 파인 튜닝된 모델 사용

- ▶ OpenAI의 텍스트 생성 모델을 파인 튜닝하면 특정 애플리케이션에 더 적합하게 만들 수 있지만, 이는 시간과 노력을 신중하게 투자해야 합니다.
- ▶ openai에서는 우선 프롬프트 엔지니어링, 프롬프트 체이닝(복잡한 작업을 여러 프롬프트로 나눔), 함수 호출과 같은 방법으로 좋은 결과를 얻으려 시도하는 것을 권장합니다.
- ▶ openai에서는 프롬프트 엔지니어링 가이드는 파인 튜닝 없이 성능을 향상시키기 위한 가장 효과적인 전략과 전술에 대한 배경 정보를 제공합니다.

ChatGPT API

● Fine-Tuning 주요 단계

→ 데이터 준비

- ▶ 작업에 맞는 고품질 데이터 수집
- ▶ 데이터 레이블링(필요 시), 전처리(클리닝, 토큰화 등) 수행

→ 모델 준비

- ▶ 사전 학습된 LLM 모델 로드(GPT, BERT, T5 등)
- ▶ 원하는 작업에 따라 모델 아키텍처를 조정(필요 시)

→ 하이퍼파라미터 설정

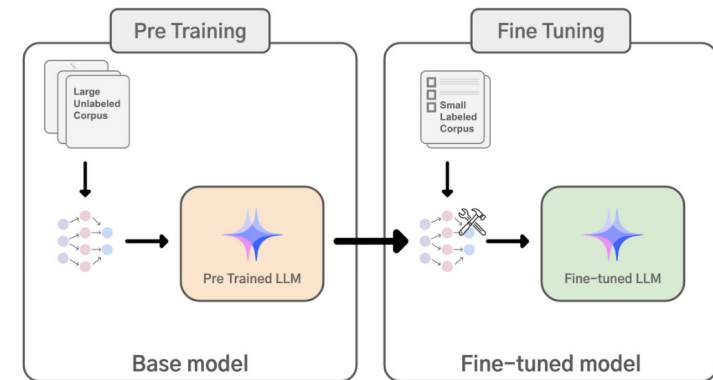
- ▶ 학습률, 배치 크기, 에폭 수 등 설정

→ 학습

- ▶ 도메인 데이터셋을 사용하여 모델을 추가 학습
- ▶ 기존의 사전 학습 가중치를 유지하면서 새로운 데이터에 맞게 업데이트

→ 평가 및 테스트

- ▶ 검증 데이터셋을 사용하여 모델 성능 평가.
- ▶ 오버피팅 방지(예: Early Stopping 사용)



ChatGPT API

● Fine-Tuning 기법

→ Full Fine-tuning

- ▶ 모델 전체 가중치를 업데이트
- ▶ 대규모 데이터와 컴퓨팅 자원이 필요

→ Partial Fine-tuning

- ▶ 모델의 일부 계층만 학습
- ▶ 예: 상위 계층만 업데이트, 하위 계층(기본 언어 이해 계층)은 고정

→ Prompt Tuning / Prefix Tuning

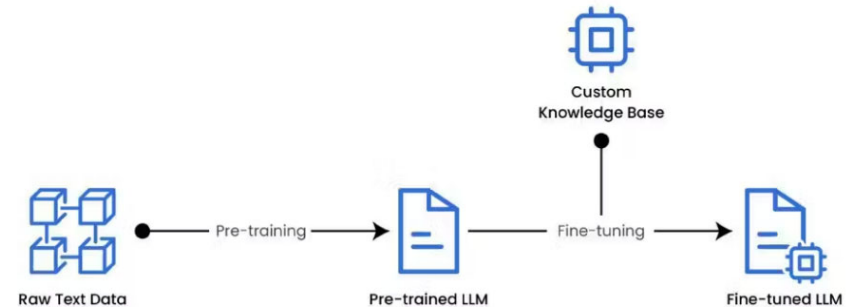
- ▶ 입력 프롬프트를 조정하여 모델 출력을 변경
- ▶ 대규모 모델에서 효율적인 파인튜닝 방식

→ Adapter Tuning

- ▶ 모델에 소규모 네트워크(Adapter)를 추가하고, Adapter만 학습
- ▶ 기존 모델의 가중치는 변경하지 않음

→ LoRA (Low-Rank Adaptation)

- ▶ 모델의 파라미터를 효율적으로 업데이트하기 위해 저차원 매개변수 학습



ChatGPT API

○ FineTuning

→ 파인 튜닝이 가능한 모델 :

- ▶ gpt-4o-2024-08-06
- ▶ gpt-4o-mini-2024-07-18
- ▶ gpt-4-0613
- ▶ gpt-3.5-turbo-0125
- ▶ gpt-3.5-turbo-1106
- ▶ gpt-3.5-turbo-0613
- ▶ 이미 파인 튜닝된 모델을 다시 파인 튜닝할 수 있습니다. (추가 데이터를 획득한 경우 이전 훈련 단계를 반복하지 않고 사용하기에 유용합니다.)

→ 파인 튜닝이 결과를 향상시킬 수 있는 몇 가지 일반적인 사용 사례 :

- ▶ 스타일, 톤, 형식 또는 기타 질적인 측면 설정
- ▶ 원하는 출력을 생성하는 신뢰성 향상
- ▶ 복잡한 프롬프트를 따르지 못하는 오류 수정
- ▶ 특정 방식으로 많은 엡지 케이스 처리
- ▶ 프롬프트로 명확하게 표현하기 어려운 새로운 기술이나 작업 수행

ChatGPT API

● FineTuning

- ➔ 파인 튜닝을 위한 데이터 준비 :
 - ▶ JSONL (JSON Lines) 형태의 데이터
 - ▶ 파일 안에는 prompt, completion의 형태로 구성
 - ▶ prompt에는 원하는 프롬프트 구성
 - ▶ Completion은 원하는 답 형태로 구성
- ➔ 모델을 파인 튜닝하기 위해 최소 10개의 예제를 제공해야 합니다.
- ➔ gpt-4o-mini 및 gpt-3.5-turbo 모델의 경우 50개에서 100개의 훈련 예제로 파인 튜닝할 때 분명한 개선을 확인할 수 있습니다.

ChatGPT API

● FineTuning 실습

→ jsonl 형식의 학습 데이터 준비

```
{"messages": [{"role": "system", "content": "Marv는 사실적인 정보를 제공하면서도 냉소적인 챗봇입니다."}, {"role": "user", "content": "프랑스의 수도가 무엇인가요?"}, {"role": "assistant", "content": "파리죠, 마치 모르는 사람이 있을 것 같네요."}]}
```

```
{"messages": [{"role": "system", "content": "Marv는 사실적인 정보를 제공하면서도 냉소적인 챗봇입니다."}, {"role": "user", "content": "'로미오와 줄리엣'을 쓴 사람은 누구인가요?"}, {"role": "assistant", "content": "오, 그냥 윌리엄 셰익스피어라는 사람이죠. 들어보셨나요?"}]}
```

→ jsonl format error Check

▶ https://cookbook.openai.com/examples/chat_finetuning_data_prep

→ jsonl 학습 데이터를 검증한 후에는 파인 튜닝 작업에 사용하기 위해 파일을 업로드

```
client = OpenAI(api_key=OPENAI_API_KEY)

client.files.create(
    file=open("./data/toy_chat_fine_tuning_15.jsonl", "rb"),
    purpose="fine-tune"
)

FileObject(id='file-UsGUFsvH8wRdbGVToHLrEo', bytes=4999, created_at=1736228398, filename='toy_chat_fine_tuning_15.jsonl', status='processed', status_details=None)
```

ChatGPT API

● FineTune 클래스 와 함수

클래스 및 함수	설명
FineTune	Fine-tuning API와 상호작용하여 모델의 학습, 상태 모니터링, 결과 확인 등을 쉽게 수행할 수 있도록 설계된 클래스 fine-tuning 관련 작업 생성, 상태 확인, 취소 등을 관리 Fine-tuning 학습 작업이 실행 중일 때 작업 상태와 진행 상황을 추적 학습이 완료된 후 생성된 모델을 활용할 수 있도록 정보를 제공
FineTune.create()	Fine-tuning 작업을 생성 training_file : 학습 데이터가 포함된 파일의 ID. validation_file : (선택 사항) 검증 데이터가 포함된 파일의 ID model : 기본 모델 이름(e.g., "davinci", "curie") n_epochs : 학습을 반복할 횟수(기본값: 4)
list()	모든 Fine-tuning 작업을 나열합니다 Fine-tuning 작업 목록(각 작업의 상태, 생성 시간 등 포함) 반환
retrieve(file_id)	특정 Fine-tuning 작업의 세부 정보를 가져옵니다 fine_tune_id: 조회할 작업의 고유 ID 작업의 세부 정보(상태, 결과 모델 ID 등) 반환
delete(file_id)	진행 중인 Fine-tuning 작업을 취소 fine_tune_id: 취소할 작업의 고유 ID
download(file_id)	서버에서 파일을 다운로드

ChatGPT API

● FineTune 클래스 와 함수

클래스 및 함수	설명
events()	Fine-tuning 작업과 관련된 이벤트 로그를 가져옵니다 fine_tune_id: 조회할 작업의 고유 ID 해당 작업의 이벤트 기록(학습 상태 업데이트, 오류 메시지 등)

model	parameter	특성 및 용도
ada	3.5억	가장 작은 모델로, 빠른 응답이 필요하고 비용 효율이 중요한 작업에 적합
babbage	13억	중간 크기의 모델로, 더 나은 성능이 요구되는 작업에 사용
curie	67억	더 큰 모델로, 복잡한 작업이나 더 높은 정확도가 필요한 경우에 적합
davinci	1750억	가장 큰 모델로, 최고 수준의 성능과 창의성이 요구되는 작업에 사용
gpt-3.5-turbo	비공개	챗봇과 같은 대화형 응용 프로그램에 최적화된 모델로, 높은 성능과 효율성을 제공
gpt-4	비공개	최신 모델로, 다양한 작업에서 뛰어난 성능을 발휘하며, 더 복잡한 언어 이해와 생성이 가능

ChatGPT API

● FineTuning 실습

→ 파인 튜닝 작업 생성

```
client = OpenAI(api_key=OPENAI_API_KEY)

try:
    job = client.fine_tuning.jobs.create(
        training_file="file-UsGUFSvH8wRdbGVToHLrEo",
        model="gpt-4o-mini-2024-07-18"
    )
    print("Fine-tuning job created successfully:", job.id)
except Exception as e:
    print("Error creating fine-tuning job:", str(e))
```

Fine-tuning job created successfully: ftjob-Nq0qtqaOeEWJE5LuHCrGe631

```
#Fine-Tuning Job 상태 검색
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)
client.fine_tuning.jobs.retrieve("ftjob-Nq0qtqaOeEWJE5LuHCrGe631")
```

```
FineTuningJob(id='ftjob-Nq0qtqaOeEWJE5LuHCrGe631', created_at=1736229338, error=Error(code=None, message=None, param=None), fine_tuned_model='ft:gpt-4o-mini-2024-07-18:personal::AmwwVm72', finished_at=1736229681, hyperparameters=Hyperparameters(n_epochs=6, batch_size=1, learning_rate_multiplier=1.8), model='gpt-4o-mini-2024-07-18', object='fine_tuning.job', organization_id='org-pBx3BsTHlIAVWp18cVjEfghb', result_files=['file-9DE64MdmXFURzdSivbj12T'], seed=1230904425, status='succeeded', trained_tokens=5736, training_file='file-UsGUFSvH8wRdbGVToHLrEo', validation_file=None, estimated_finish=None, integration_logs=[], user_provided_suffix=None, method={'type': 'supervised', 'supervised': {'hyperparameters': {'n_epochs': 6, 'batch_size': 1, 'learning_rate_multiplier': 1.8}}})
```

ChatGPT API

● FineTuning 실습

→ 파인 튜닝된 모델 사용하기

```
from openai import OpenAI
client = OpenAI(api_key=OPENAI_API_KEY)

completion = client.chat.completions.create(
    model="ft:gpt-4o-mini-2024-07-18:personal::AmwwVm72",
    messages=[
        {"role": "system", "content": "Marv는 사실적인 정보를 제공하면서도 냉소적인 챗봇입니다."},
        {"role": "user", "content": "인터넷 없이 살 수 있을까요?"}
    ]
)

print(completion.choices[0].message)
```

```
ChatCompletionMessage(content='물론이죠, 도전해보세요!', refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None)
```

ChatGPT API

● FineTune Task 관리

```
import openai
# Fine-tuning 작업 생성
response = openai.fine_tuning.jobs.create(
    training_file="file-abc123",
    model="davinci",
    n_epochs=4
)
fine_tune_id = response["id"]

# 작업 상태 확인
status = openai.FineTune.retrieve(fine_tune_id)["status"]
print(f"Fine-tuning 상태: {status}")

# 모든 작업 목록 가져오기
fine_tunes = openai.FineTune.list()
print(f"Fine-tuning 작업 목록: {fine_tunes}")

# 작업 취소
openai.FineTune.cancel(fine_tune_id)

# 이벤트 로그 확인
events = openai.FineTune.events(fine_tune_id)
print(f"이벤트 로그: {events}")
```

ChatGPT API

● File 클래스 와 함수

클래스 및 함수	설명
File	OpenAI API에 파일을 업로드, 나열, 검색 및 삭제에 사용 모델에서 사용하는 데이터를 준비하거나 연동하는 데 중요한 역할을 수행 Fine-tuning(미세 조정)을 위해 학습 데이터와 검증 데이터를 업로드 및 관리하는 데 사용
File.create()	OpenAI 플랫폼에 파일을 업로드할 때 사용 file : 업로드할 파일 경로 purpose : 파일의 용도 ("fine-tune" 등)
list()	사용자가 업로드한 파일 목록을 반환
retrieve()	특정 파일의 세부 정보를 반환
delete(file_id)	특정 파일을 삭제

```
file = openai.File.create(file="data.jsonl", purpose="fine-tune")  
print(file["id"]) # 업로드된 파일 ID
```


ChatGPT API

● Fine-Tuning 성공 사례

- GPT-3 기반으로 OpenAI가 다양한 작업에 맞게 API에서 파인튜닝을 제공
 - ▶ 특정 고객 서비스 응답 생성
- BERT, RoBERTa, GPT-2 등을 다양한 NLP 작업(텍스트 분류, 요약 등)에 맞게 파인튜닝
 - ▶ Hugging Face
 - ▶ 금융 뉴스 분석, 법률 문서 분류
- 다양한 텍스트 생성 작업에 맞게 파인튜닝
 - ▶ Google T5
 - ▶ 질의 응답 시스템, 텍스트 요약
- BERT를 생의학 도메인 텍스트에 맞게 파인튜닝
 - ▶ 의료 논문 분석, 유전자 이름 인식

ChatGPT API

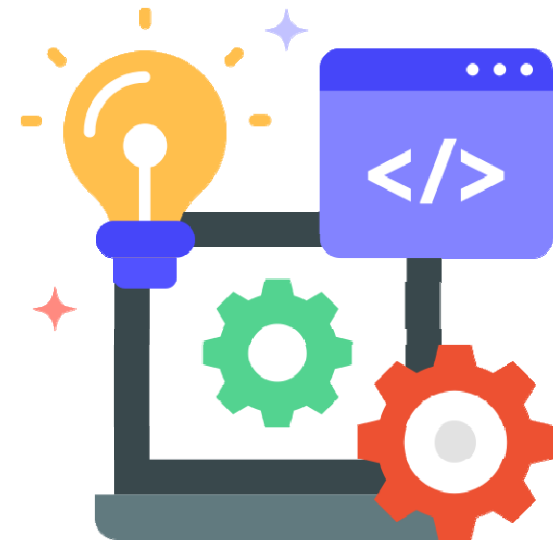
● Fine-Tuning의 한계

→ Fine-tuning의 한계

- ▶ 고품질의 레이블링된 데이터가 필요
- ▶ 높은 학습 비용과 시간이 필요
- ▶ 소량의 데이터로 파인튜닝 시 모델이 특정 데이터에 과적합될 수 있음
- ▶ 모델의 범용성 저하

Appendix

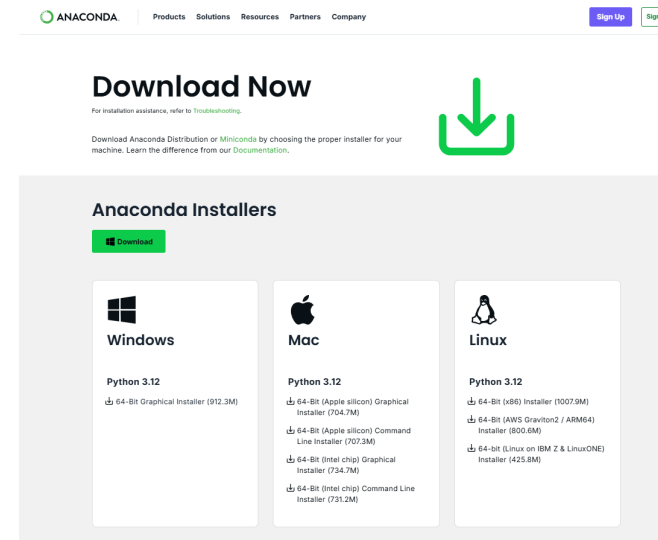
- 환경 구성
- OpenAI API key 생성
-



환경 구성

● Anaconda

- Conda 패키지 관리자, Python
- 데이터 과학 및 머신러닝 작업에 필요한 대규모 라이브러리 세트 (예: NumPy, Pandas, Scikit-learn, TensorFlow, Matplotlib 등)
- 설치 파일 크기가 크고(약 3GB 이상), 설치 후에도 많은 라이브러리가 미리 포함되어 있어 저장 공간을 많이 차지함.
- 초기 설정이 간단하며, 주요 패키지가 미리 설치되어 있어 빠르게 사용할 수 있음
- 큰 설치 용량과 많은 불필요한 패키지로 인해 가벼운 작업에는 비효율적일 수 있음



환경 구성

○ Miniconda 설치

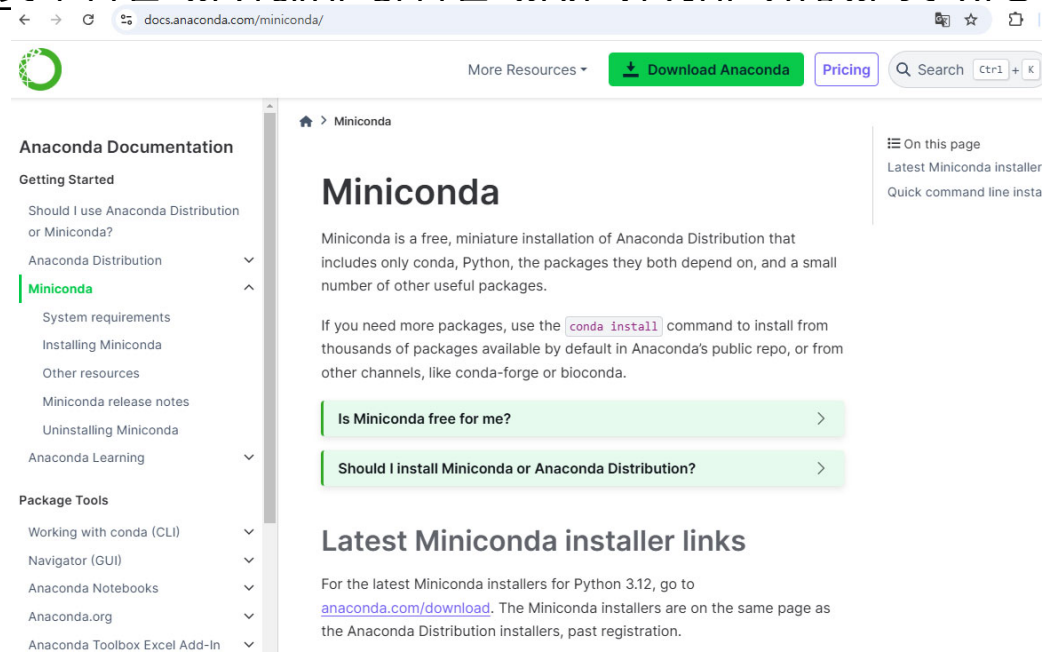
→ Miniconda 설치

▶ [Miniconda — Anaconda documentation](https://docs.anaconda.com/miniconda/)

▶ 설치 파일 크기가 작음 (약 50MB~100MB 정도)

▶ 원하는 패키지만 설치하여, 가볍고 사용자 정의된 환경을 만들고자 하는 사용자에게 적합

▶ 초기에 필요한 패키지와 도구를 스스로 설치해야 할 만큼 선택 사항이 아닌 필수인 것



환경 구성

● Miniconda 설치

→ 채널 추가

- ▶ 채널 확인 : `conda config --show channels`
- ▶ 채널 추가, 우선순위 변경 : `conda config --add channels conda-forge && conda config --set channel_priority strict`

→ 가상환경 생성

- ▶ `conda create -n 가상환경이름 python=3.12 -y`
- ▶ `conda info --envs`

→ 가상환경 활성화

- ▶ `conda activate 가상환경이름`

→ 필요한 패키지 설치

- ▶ `conda install notebook, seaborn, matplotlib -y`
- ▶ `conda install pandas, scikit-learn -y`

환경 구성

○ git 설치

→ <https://git-scm.com/download/win>

→ 64-bit Git for Windows Setup 다운로드

Download for Windows

[Click here to download](#) the latest (2.45.2) 64-bit version of Git for Windows. This is the most recent [maintained build](#). It was released **12 days ago**, on 2024-06-03.

Other Git for Windows downloads

Standalone Installer

[32-bit Git for Windows Setup.](#)

[64-bit Git for Windows Setup.](#)

Portable ("thumbdrive edition")

[32-bit Git for Windows Portable.](#)

[64-bit Git for Windows Portable.](#)

Using winget tool

Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

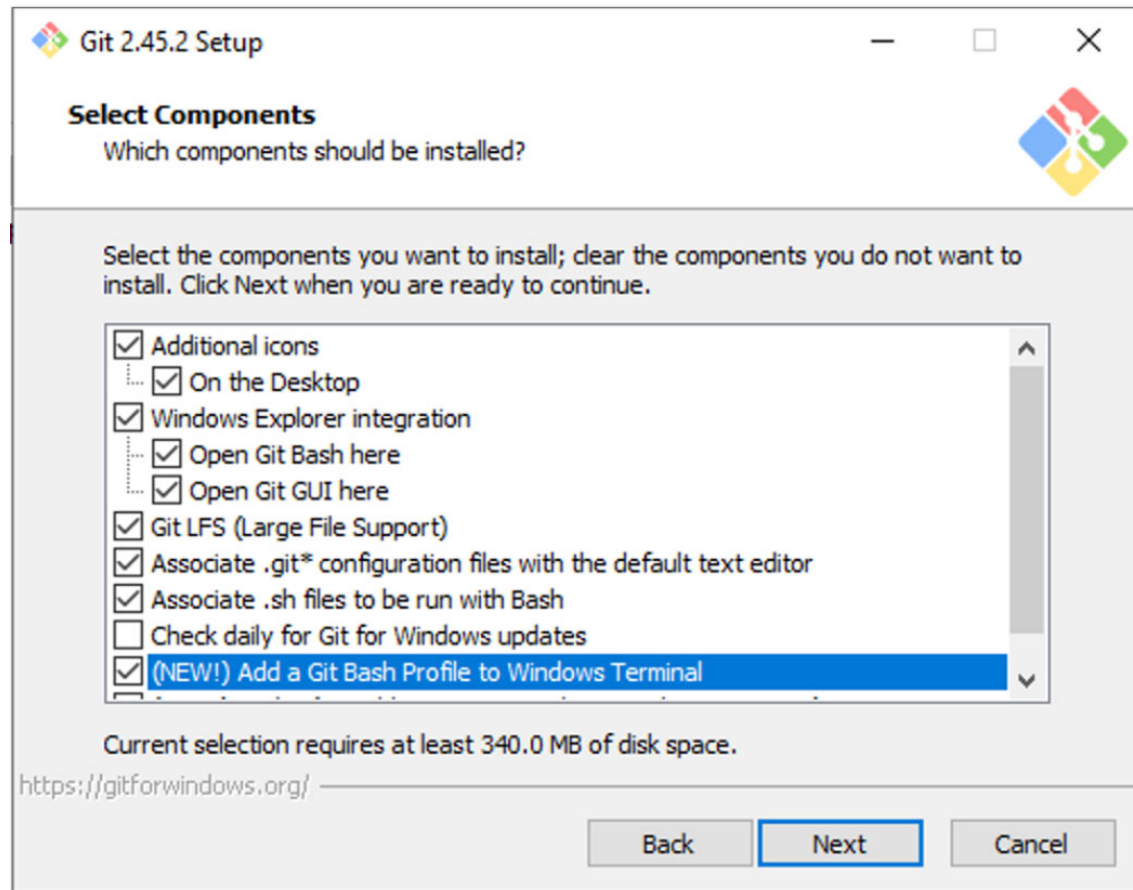
```
winget install --id Git.Git -e --source winget
```

The current source code release is version **2.45.2**. If you want the newer version, you can build it from [the source code](#).

환경 구성

○ git 설치

- 설치시 옵션 확인 후 진행
- 나머지는 전부 Next 버튼을 눌러 설치를 진행합니다.



환경 구성

○ git 설치

- Window 키 - PowerShell 을 반드시 관리자 권한으로 실행
- 아래의 명령어 "git" 을 입력하여 아래의 이미지 처럼 출력이 뜨는지 확인

```
Administrator: Windows PowerShell
PS C:\Users\Administrator> git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate] [-P | --no-pager] [--no-replace-objects] [--bare]
          [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
          [--config-env=<name>=<envvar>] <command> [<args>]

These are common Git commands used in various situations:


start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one


work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index


examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status


grow, mark and tweak your common history
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG


collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
PS C:\Users\Administrator>
```

환경 구성

● PowerShell Policy 적용

→ 먼저, Windows PowerShell 을 "관리자 권한으로 실행" 합니다.

→ 다음의 명령어를 입력하여 Policy 를 적용합니다.

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
```

→ 적용이 완료된 후 Windows PowerShell 을 꺾다가 껍니다.

→ Windows PowerShell 실행시 "관리자 권한으로 실행" 후 pyenv 설치

```
git clone https://github.com/pyenv-win/pyenv-win.git  
"$env:USERPROFILE\.pyenv"
```

환경 구성

● PowerShell Policy 적용

→ 환경변수 추가

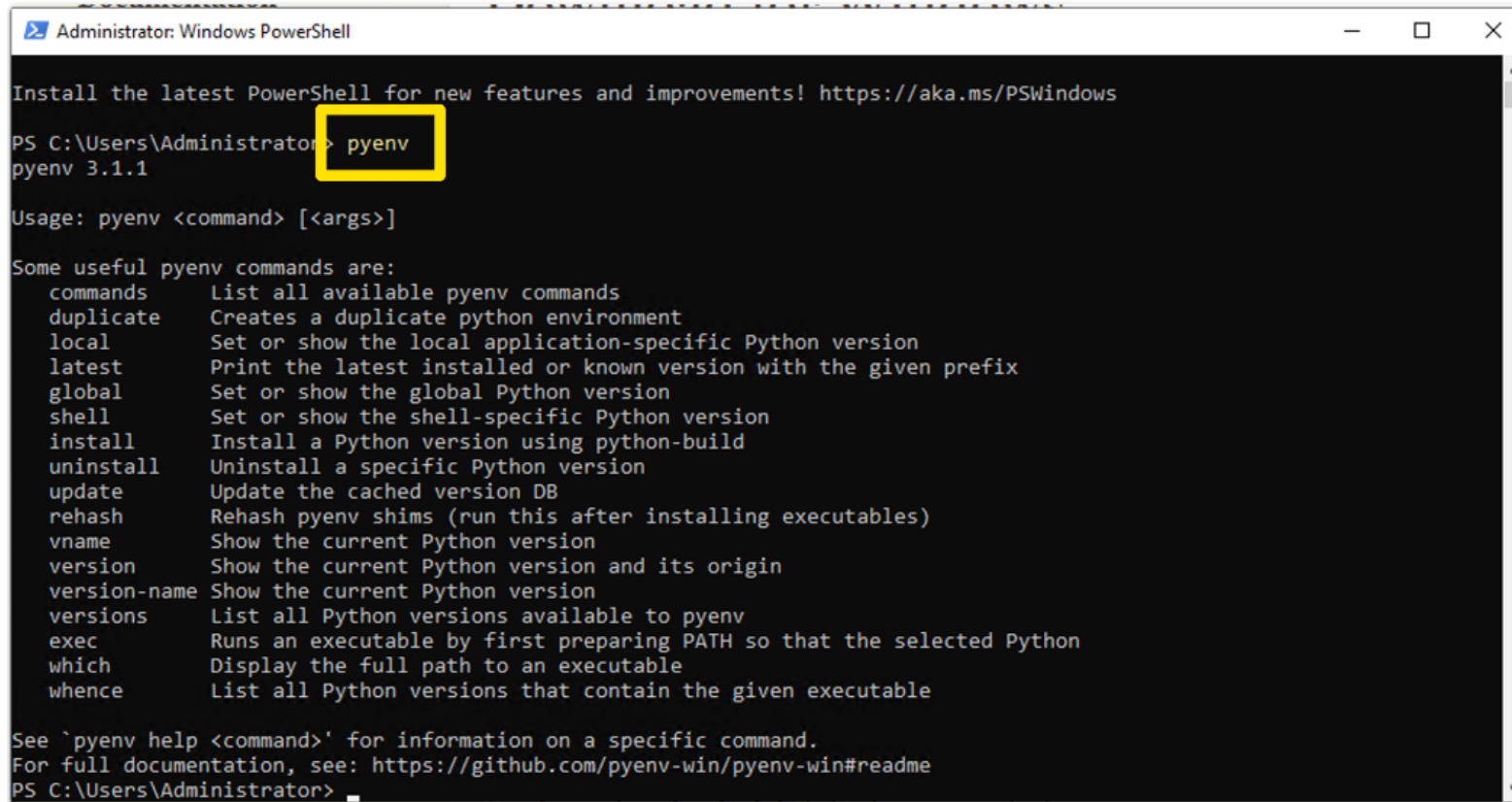
```
[System.Environment]::SetEnvironmentVariable('PYENV', $env:USERPROFILE +  
"\.pyenv\pyenv-win\","User")  
[System.Environment]::SetEnvironmentVariable('PYENV_ROOT', $env:USERPROFILE +  
"\.pyenv\pyenv-win\","User")  
[System.Environment]::SetEnvironmentVariable('PYENV_HOME', $env:USERPROFILE +  
"\.pyenv\pyenv-win\","User")
```

```
[System.Environment]::SetEnvironmentVariable('PATH', $env:USERPROFILE +  
"\.pyenv\pyenv-win\bin;" + $env:USERPROFILE + "\.pyenv\pyenv-win\shims;" +  
[System.Environment]::GetEnvironmentVariable('PATH', "User"), "User")
```

환경 구성

● PowerShell Policy 적용

→ 환경변수 추가



```
Administrator: Windows PowerShell

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> pyenv
pyenv 3.1.1

Usage: pyenv <command> [<args>]

Some useful pyenv commands are:
  commands      List all available pyenv commands
  duplicate      Creates a duplicate python environment
  local          Set or show the local application-specific Python version
  latest         Print the latest installed or known version with the given prefix
  global         Set or show the global Python version
  shell          Set or show the shell-specific Python version
  install        Install a Python version using python-build
  uninstall      Uninstall a specific Python version
  update         Update the cached version DB
  rehash         Rehash pyenv shims (run this after installing executables)
  vname          Show the current Python version
  version        Show the current Python version and its origin
  version-name   Show the current Python version
  versions       List all Python versions available to pyenv
  exec           Runs an executable by first preparing PATH so that the selected Python
  which          Display the full path to an executable
  whence         List all Python versions that contain the given executable

See `pyenv help <command>' for information on a specific command.
For full documentation, see: https://github.com/pyenv-win/pyenv-win#readme
PS C:\Users\Administrator>
```

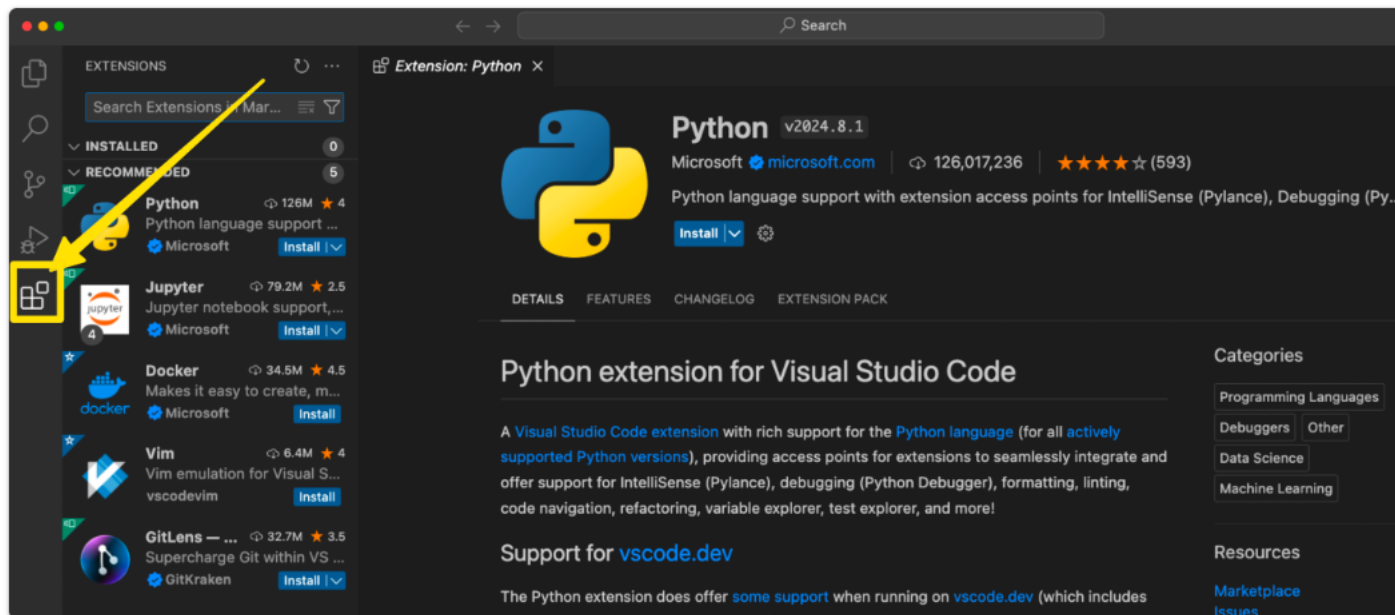
▶ pyenv install 3.11

▶ pyenv global 3.11

환경 구성

Visual Studio Code 설치

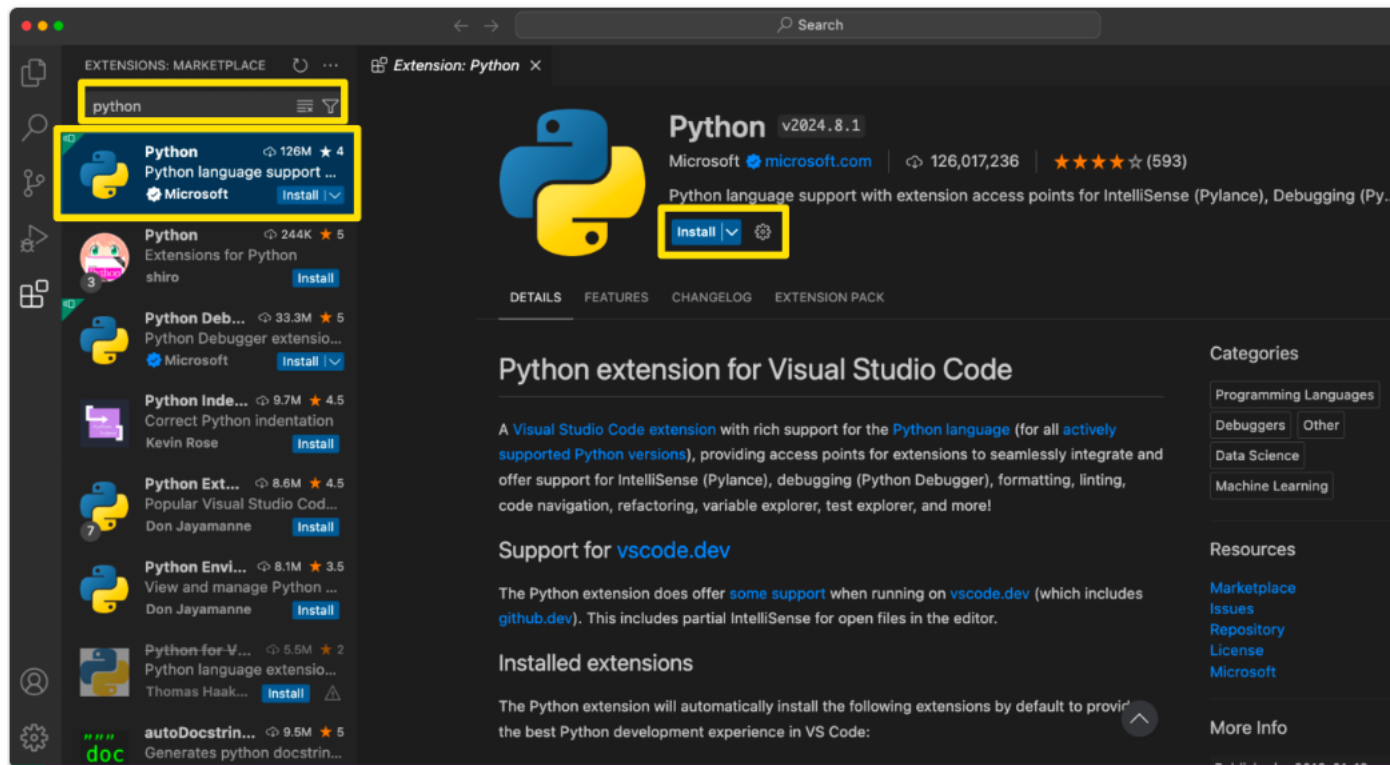
→ <https://code.visualstudio.com/download>



환경 구성

Visual Studio Code 설치

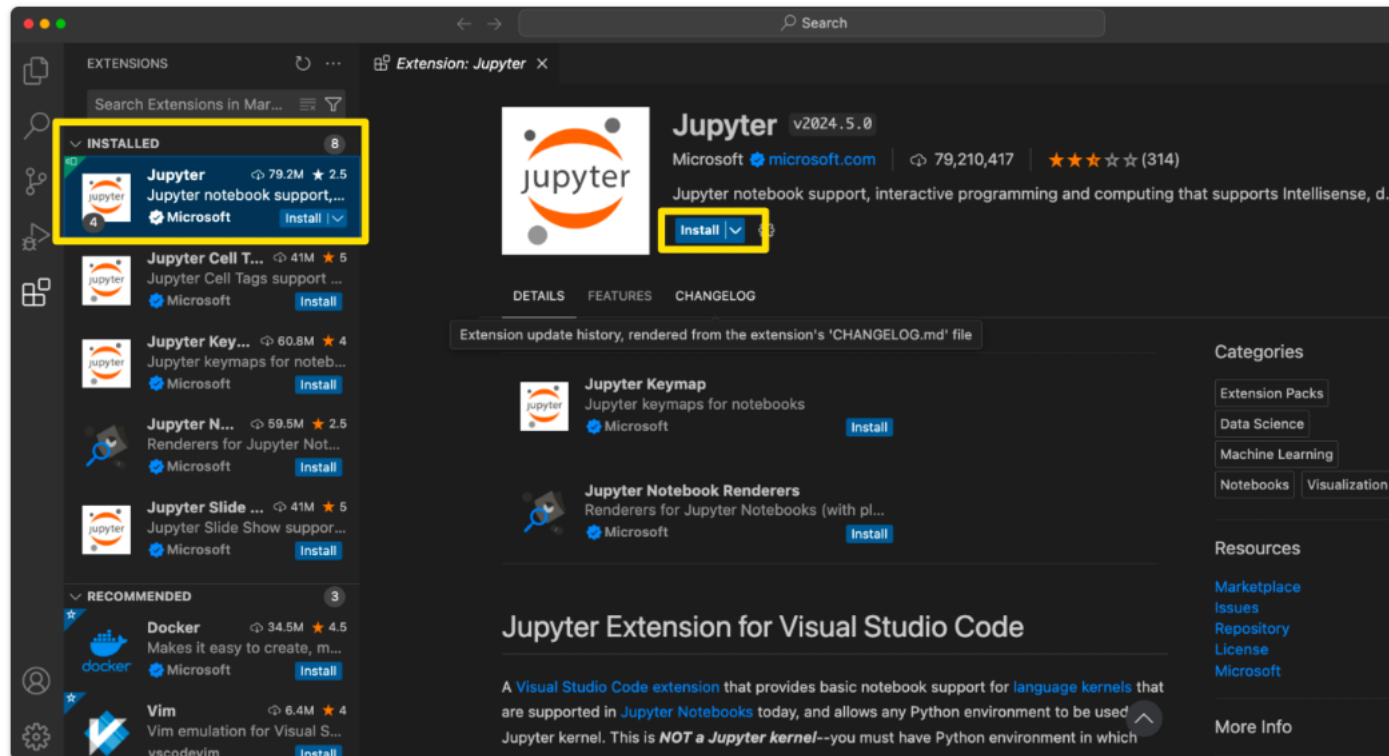
→ "python" 검색 후 설치



환경 구성

Visual Studio Code 설치

→ "jupyter" 검색 후 설치



OpenAI Key

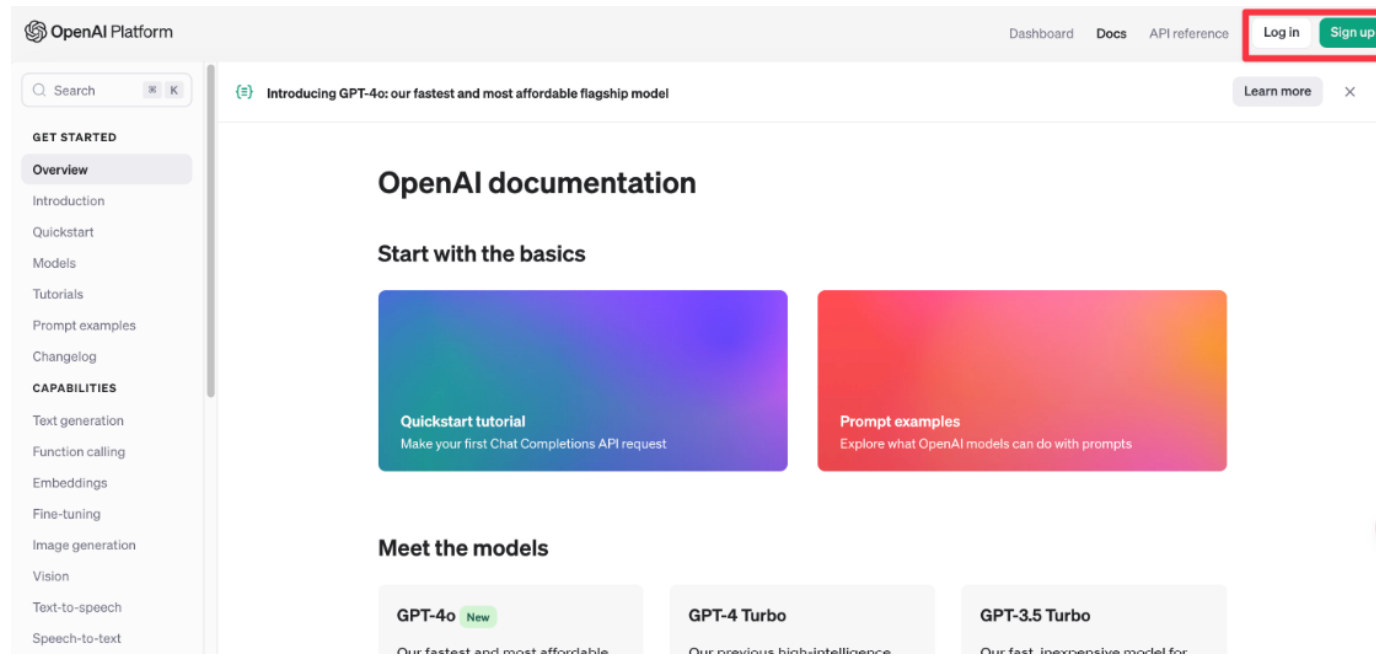
● OpenAI API key 생성

→ openai 패키지 설치

```
pip install openai
```

→ OpenAI API 웹사이트에 접속, 회원가입 또는 로그인

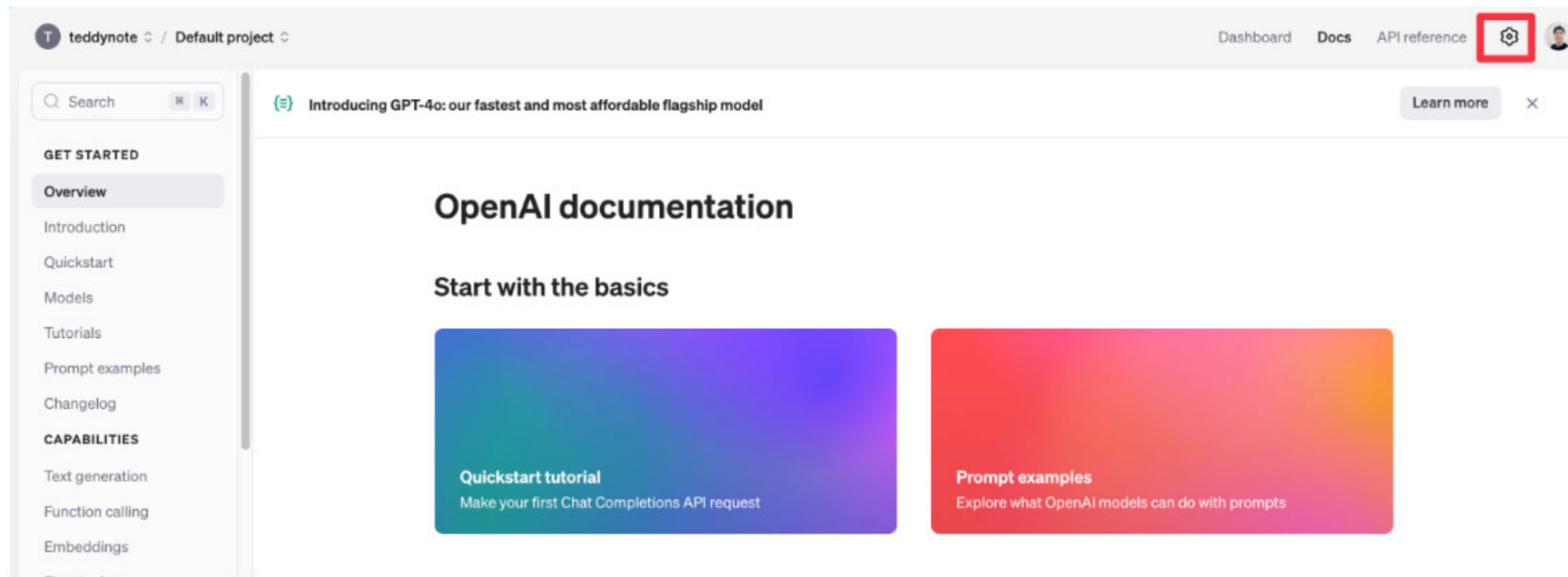
▶ <https://platform.openai.com/docs/overview>



OpenAI Key

● OpenAI API key 생성

→ 톱니바퀴(Setting) 를 눌러 설정으로 이동

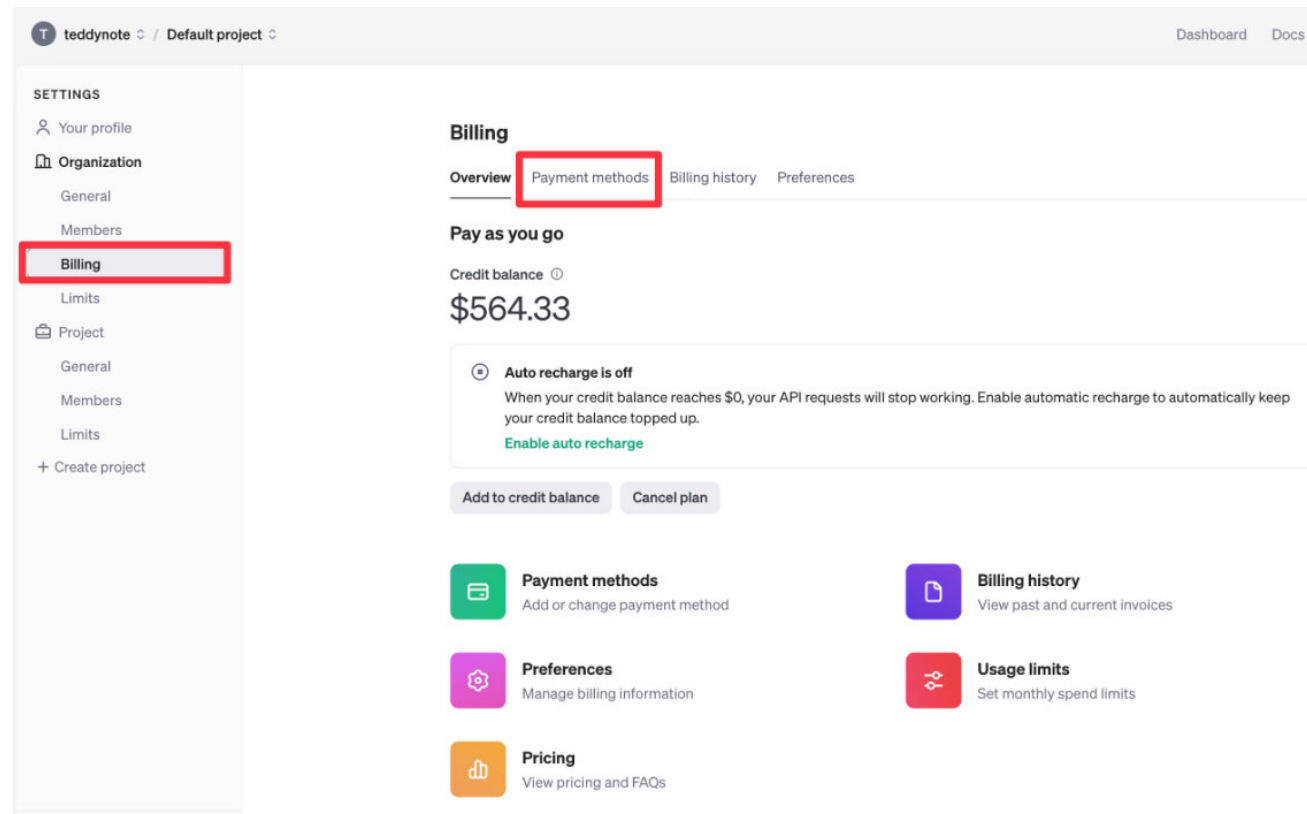


OpenAI Key

● OpenAI API key 생성

→ 왼쪽 "Billing" 메뉴에서 "Payment methods" 를 클릭하여 신용카드를 등록

▶ <https://platform.openai.com/settings/organization/billing/overview>

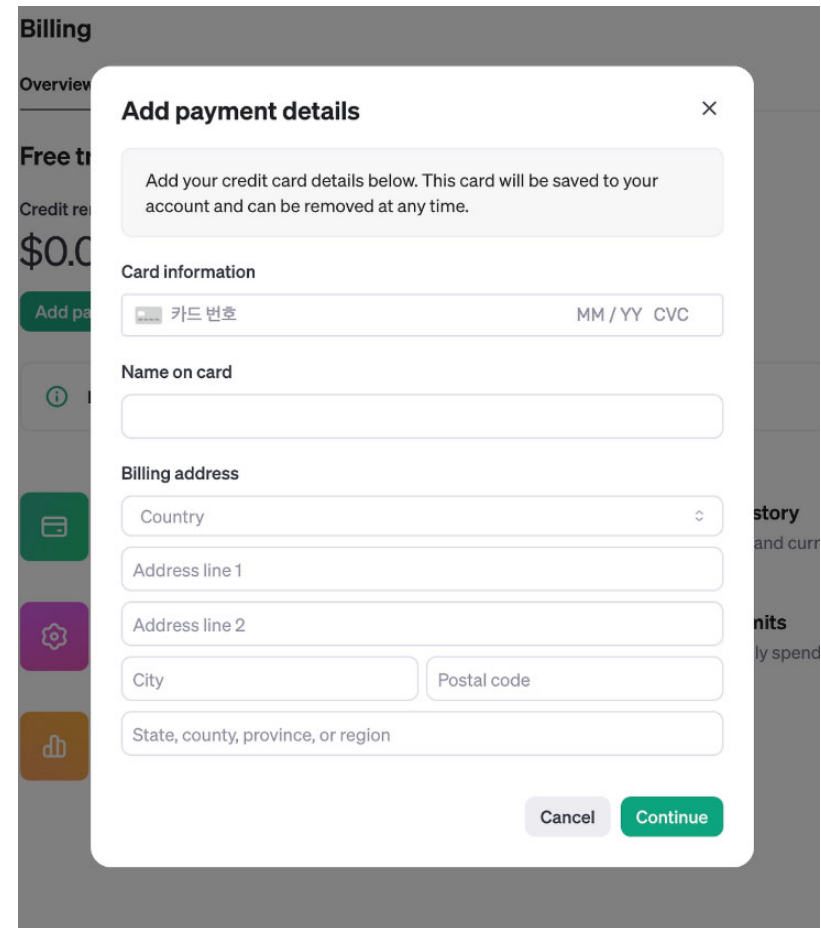
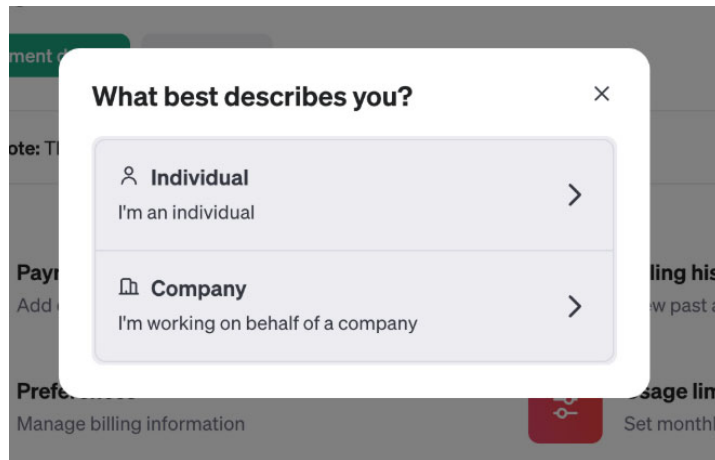


OpenAI Key

● OpenAI API key 생성

→ Add payment details 버튼을 클릭하여 카드를 등록 (Individual을 선택)

→ 영어 주소 입력 => <https://www.jusoen.com/>



OpenAI Key

🔴 OpenAI API key 생성

→ "Add to credit balance" 버튼을 눌러 사용할만큼의 미화(달러) 를 입력합니다.

The screenshot displays the OpenAI Billing interface. On the left, a sidebar menu lists settings categories: Your profile, Organization (General, Members), Billing (highlighted), and Limits. The main content area is titled 'Billing' and includes tabs for Overview, Payment methods, Billing history, and Preferences. Under 'Payment methods', a VISA card is shown as the default, with a 'Delete' button. Below this is an 'Add payment method' button. On the right, the 'Pay as you go' section shows a credit balance of \$564.33. A note indicates 'Auto recharge is off' with a link to 'Enable auto recharge'. At the bottom of this section, the 'Add to credit balance' button is highlighted with a red box, next to a 'Cancel plan' button. A bottom navigation bar contains icons and links for Payment methods, Billing history, Preferences, Usage limits, and Pricing.

OpenAI Key

● OpenAI API key 생성

→ 금액은 \$5 부터 추가가 가능합니다.

The screenshot displays the OpenAI 'Pay as you go' interface. A modal titled 'Add to credit balance' is centered on the screen. The modal contains a text input field for 'Amount to add' with the value '\$ 10'. Below the input field, a note states 'Enter an amount between \$5 and \$920' and a link for 'Model pricing'. The 'Payment method' section shows a 'VISA' card icon and a '+ Add payment method' link. At the bottom of the modal are 'Cancel' and 'Continue' buttons. The background interface shows a credit balance of '\$564.3', an 'Auto recharge' toggle, and navigation links for 'Billing history', 'Preferences', and 'Usage limits'.

Pay as you go

Credit balance (USD)

\$564.3

Auto recharge

When you use your credit, we can automatically recharge your credit balance.

Enable

Add to credit balance

Add to credit balance

Amount to add

\$ 10

Enter an amount between \$5 and \$920

Model pricing

Payment method

VISA

+ Add payment method

Cancel Continue

Billing history

View past and current billing history

Preferences

Manage billing information

Usage limits

Set monthly spending limit

OpenAI Key

● OpenAI API key 생성

→ 왼쪽의 "Limits" 탭에서 월간 사용한도를 설정할 수 있습니다.

- ▶ "Set a monthly budget": 월간 사용한도를 지정합니다. 이 금액에 도달하면 더이상 과금하지 않고 API 는 사용을 멈춥니다.
- ▶ "Set an email notification threshold": 이메일이 발송되는 요금을 지정할 수 있습니다. 이 금액에 도달하면 이메일이 발송됩니다.

The screenshot shows the OpenAI API settings page for a user named 'teddynote' under the 'Default project'. The left sidebar contains a 'SETTINGS' menu with options: 'Your profile', 'Organization' (with sub-items 'General', 'Members', 'Billing', and 'Limits' which is highlighted with a red box), 'Project' (with sub-items 'General', 'Members', and 'Limits'), and '+ Create project'. The main content area is titled 'Usage limits' and includes a description: 'Manage your API spend by configuring monthly spend limits. Notification emails will be sent to members of your organization with the "Owner" role. Note that there may be a delay in enforcing limits, and you are still responsible for any overage incurred.' Below this, the 'Usage limit' is set to '\$5,000.00'. Two configuration boxes are highlighted with red borders: 'Set a monthly budget' with a value of '\$200.00' and a 'Save' button, and 'Set an email notification threshold' with a value of '\$100.00'.

teddynote / Default project

Dashboard Docs API reference

SETTINGS

- Your profile
- Organization
 - General
 - Members
 - Billing
 - Limits
- Project
 - General
 - Members
 - Limits
- + Create project

Usage limits

Manage your API spend by configuring monthly spend limits. Notification emails will be sent to members of your organization with the "Owner" role. Note that there may be a delay in enforcing limits, and you are still responsible for any overage incurred.

Usage limit
The maximum usage OpenAI allows for your organization each month. [View current usage](#)

\$5,000.00

Set a monthly budget
If your organization exceeds this budget in a given calendar month (UTC), subsequent API requests will be rejected.

\$200.00

Save

Set an email notification threshold
If your organization exceeds this threshold in a given calendar month (UTC), an email notification will be sent.

\$100.00

OpenAI Key

● OpenAI API key 생성

→ 과금이 요구되는 API 요청은 거부하기 위해 예산설정(Set a monthly budget) 부분에 0을 입력하여 None으로 만들어 줍니다

Usage limits

Manage your API spend by configuring monthly spend limits. Notification emails will be sent to members of your organization with the "Owner" role. Note that there may be a delay in enforcing limits, and you are still responsible for any overage incurred.

Usage limit

The maximum usage OpenAI allows for your organization each month. [View current usage](#)

\$120.00

Set a monthly budget

If your organization exceeds this budget in a given calendar month (UTC), subsequent API requests will be rejected.

None

Set an email notification threshold

If your organization exceeds this threshold in a given calendar month (UTC), an email notification will be sent.

None

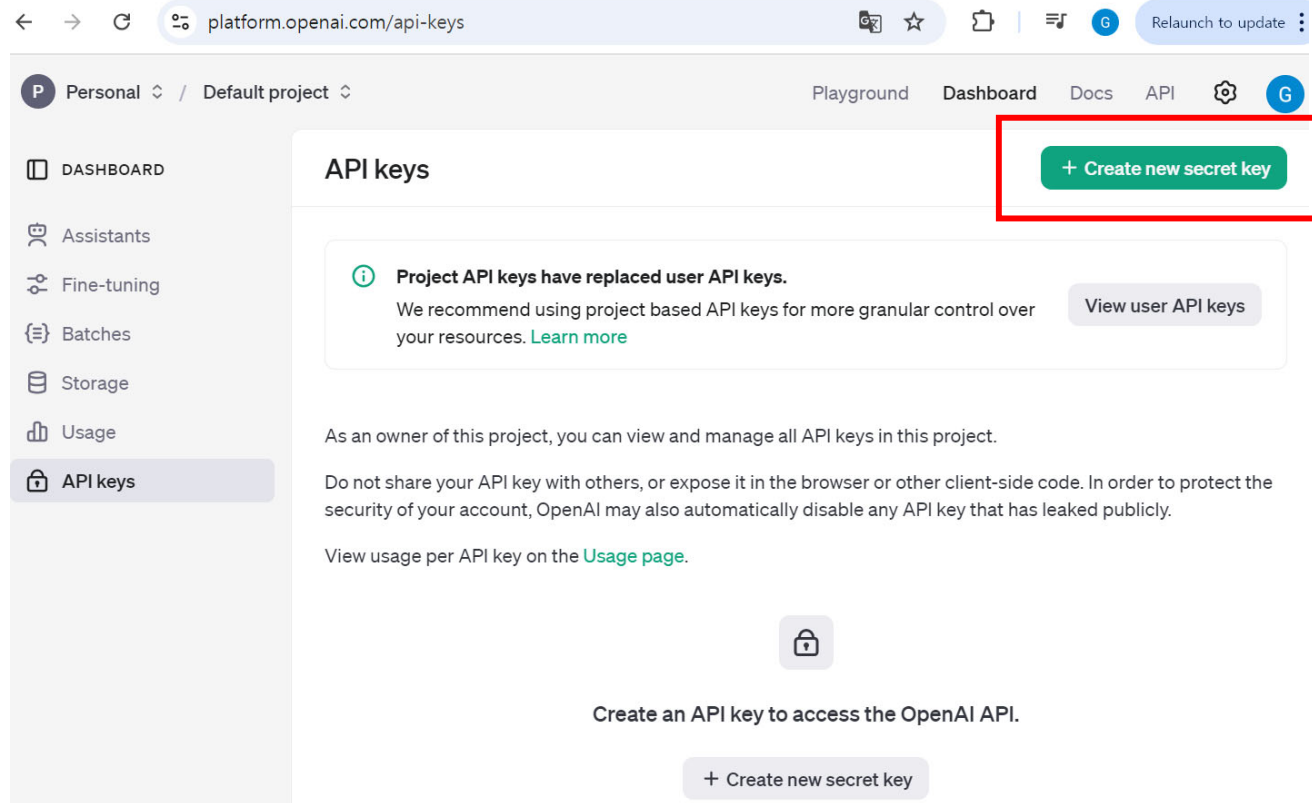
Save

OpenAI Key

● OpenAI API key 생성

→ API Key 관리 메뉴 로 접속합니다.

▶ <https://platform.openai.com/api-keys>



OpenAI Key

● OpenAI API key 생성

- 발급된 키는 생성했을 때 나타나는 창에서만 출력되고 후에는 다시 볼 수 없다고 경고메시지로 알려줍니다.
- 키를 생성하고 키를 복사해서 다른곳에 잘 저장해놓습니다

Create new secret key

Owned by

☒ You ☐ Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

My Test Key

Project

Default project

Permissions

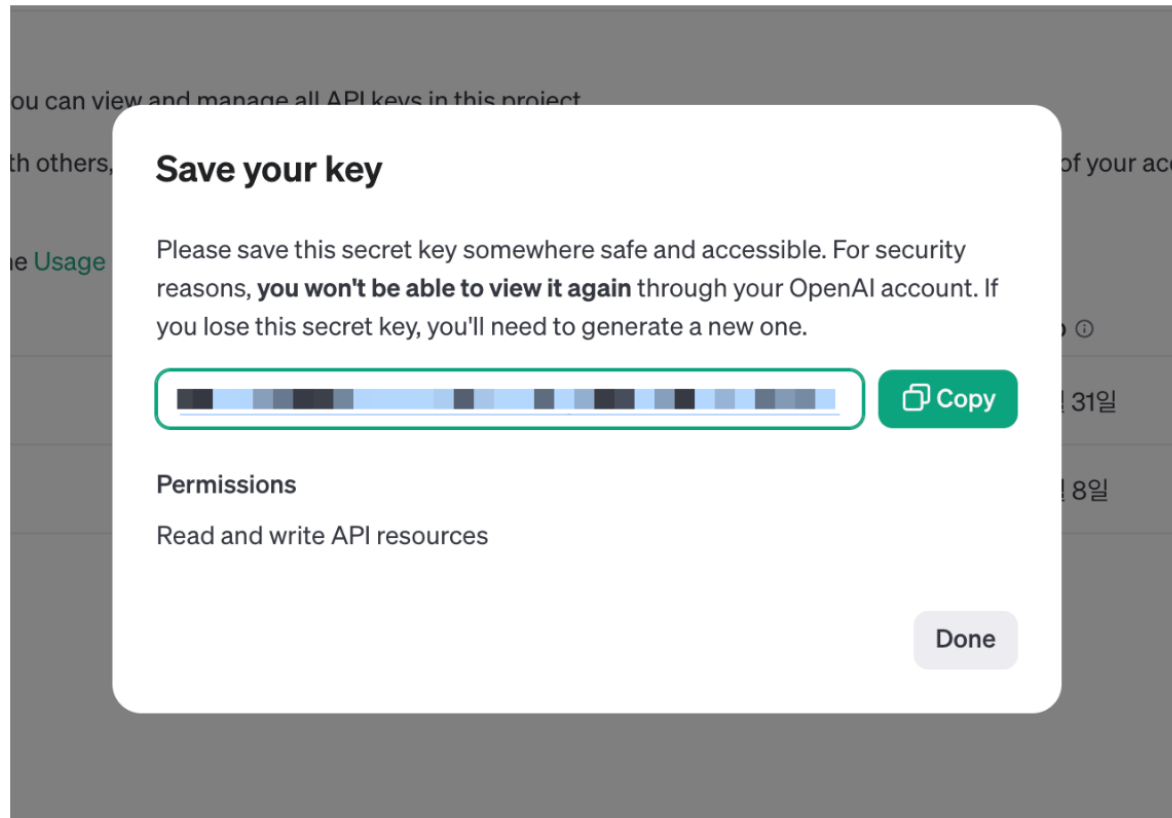
All Restricted Read Only

Cancel Create secret key

OpenAI Key

● OpenAI API key 생성

- 키가 유출되면 다른 사람이 내 API KEY 를 사용하여 GPT 를 사용할 수 있으며, 결제는 제 지갑에서 결제됩니다.
- 절대 키는 타인에게 공유하지 마시고, 안전한 곳에 보관 하세요



OpenAI Key

● OpenAI API key 생성

→ OpenAI API 사용량 확인

▶ <https://platform.openai.com/usage>

→ 결제수단 등록 안하고 API 요청하면 아래와 같이 에러 메시지가 발생합니다

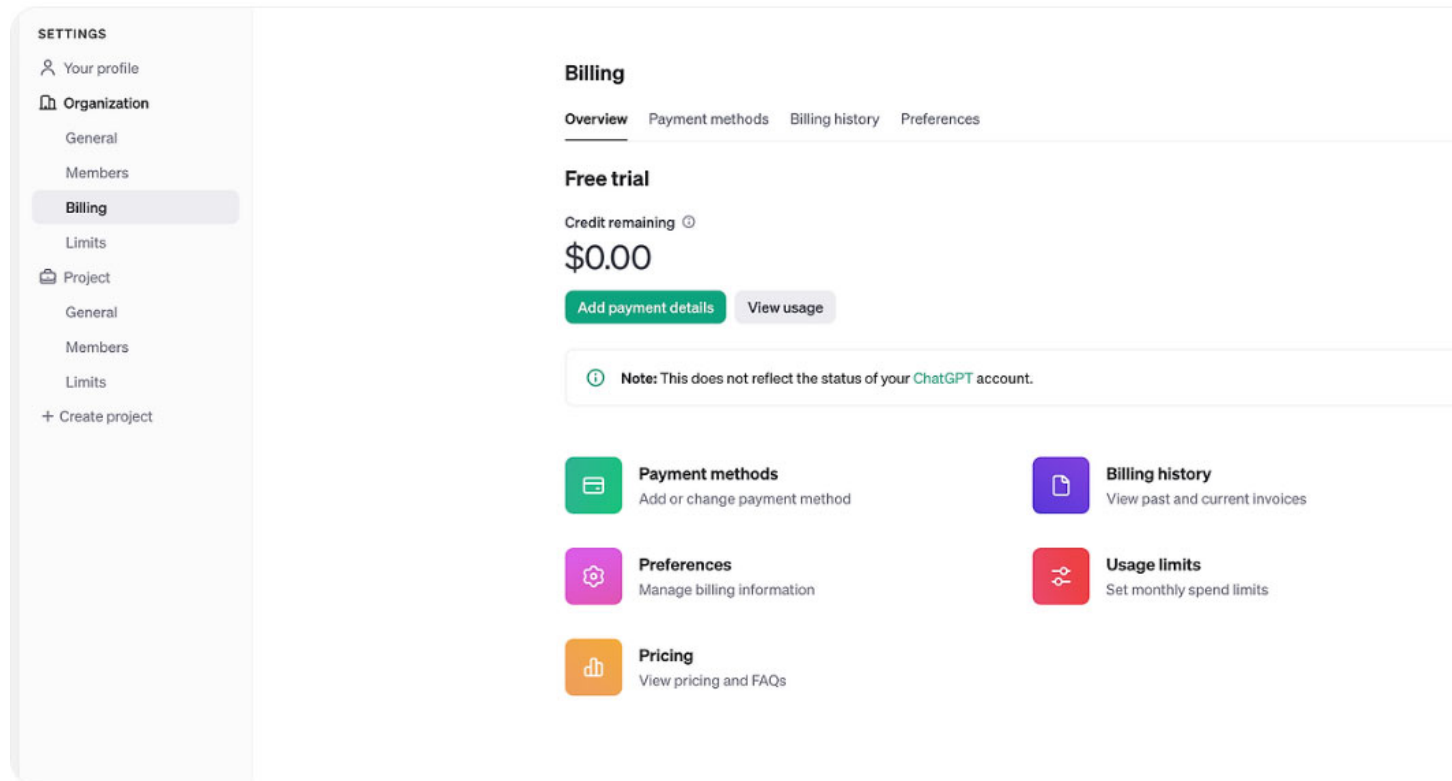
```
{
  "error": {
    "message": "You exceeded your current quota, please check your plan and billing details. For more information on this error, read the docs: https://platform.openai.com/docs/guides/error-codes/api-errors.",
    "type": "insufficient_quota",
    "param": null,
    "code": "insufficient_quota"
  }
}
```

OpenAI Key

● OpenAI API key 생성

→ 결제 수단 등록

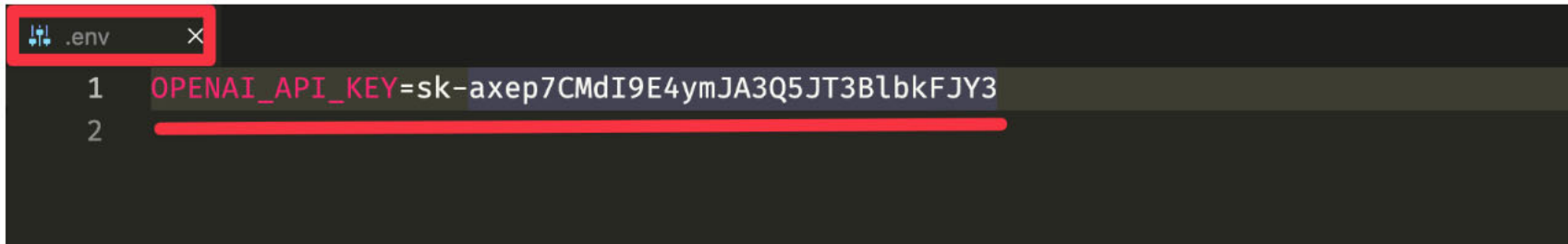
▶ 사용자 프로필 설정 페이지의 billing 메뉴로 이동



OpenAI Key

● .env 파일 설정

- 프로젝트 루트 디렉토리에 .env 파일을 생성합니다.
- .env 파일에 OPENAI_API_KEY=방금복사한 키를 입력 한 뒤 Ctrl + S 를 눌러 저장하고 파일을 닫습니다.



```
.env
1 OPENAI_API_KEY=sk-axep7CMdI9E4ymJA3Q5JT3BlbkFJY3
2
```

LangChain 업데이트

```
!pip install -r https://raw.githubusercontent.com/teddylee777/langchain-kr/main/requirements.txt
```

API KEY를 환경변수로 관리하기 위한 설정 파일

설치: pip install python-dotenv

```
from dotenv import load_dotenv
```

API KEY 정보로드

```
load_dotenv()
```

OpenAI Key

● .env 파일 설정

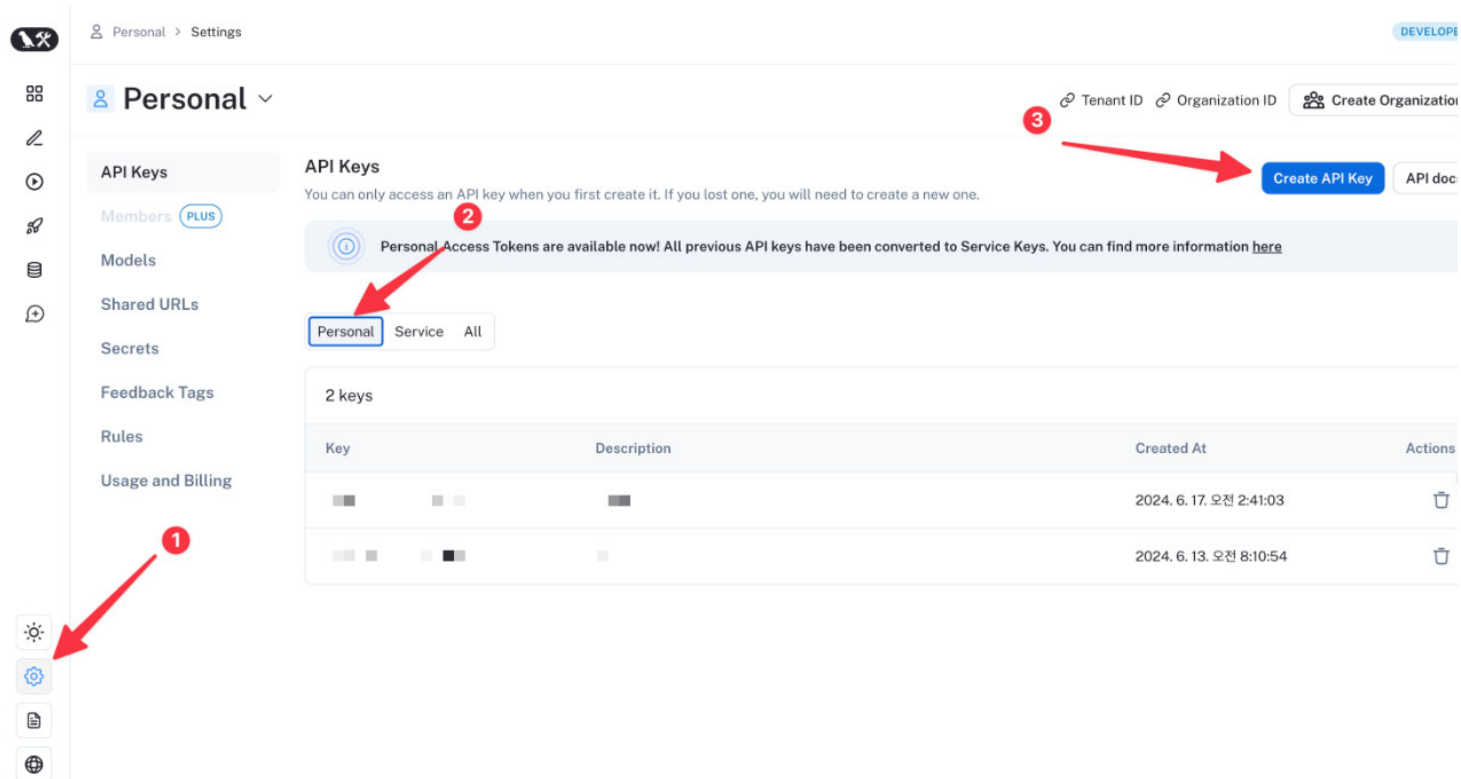
→ API Key 가 잘 설정되었는지 확인합니다.

```
import os  
  
print(f"[API KEY]\n{os.environ['OPENAI_API_KEY']}")
```

LangSmith API Key

LangSmith API Key 발급

- <https://smith.langchain.com/> 으로 접속하여 회원가입을 진행합니다.
- 가입후 이메일 인증하는 절차를 진행해야 합니다.
- 왼쪽 톱니바퀴(Settings) - 가운데 "Personal" - "Create API Key" 를 눌러 API 키를 발급 받습니다.

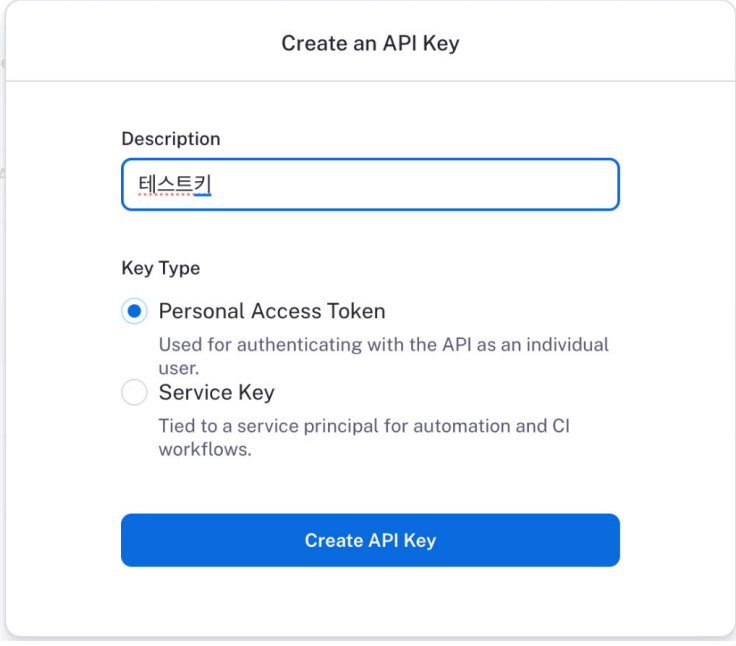


LangSmith API Key

● LangSmith API Key 발급

→ 생성한 키를 유출하지 않도록 안전한 곳에 복사해 두세요.

→



The screenshot shows a web form titled "Create an API Key". It has two main sections: "Description" and "Key Type". In the "Description" section, there is a text input field containing the Korean text "테스트키" (Test Key). In the "Key Type" section, there are two radio button options. The first option, "Personal Access Token", is selected and includes the description "Used for authenticating with the API as an individual user." The second option, "Service Key", is unselected and includes the description "Tied to a service principal for automation and CI workflows." At the bottom of the form is a blue button labeled "Create API Key".

Copy your new API key

 Copy

[A long string of characters representing the API key is shown, with some characters obscured by black boxes for security.]

I've saved the API key to a safe place

LangSmith API Key

LangSmith API Key 발급

→ <https://smith.langchain.com/onboarding?organizationId=befcfb5f-a055-492c-a8c5-0cfa2ad8a855&step=1>

1. **Generate API Key**

2. Install dependencies

Python

TypeScript

```
1 pip install -U langchain langchain-openai
```

Copy

3. Configure environment to connect to LangSmith.

Project Name

pr-damp-yak-93

```
1 LANGSMITH_TRACING=true
2 LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
3 LANGSMITH_API_KEY="<your-api-key>"
4 LANGSMITH_PROJECT="pr-damp-yak-93"
5 OPENAI_API_KEY="<your-openai-api-key>"
```

Copy

4. Run any LLM, Chat model, or Chain. Its trace will be sent to this project.

```
1 from langchain_openai import ChatOpenAI
2
3 llm = ChatOpenAI()
4 llm.invoke("Hello, world!")
```

Copy

LangSmith API Key

◉ .env 파일에 LangSmith 키 설정

- LANGCHAIN_TRACING_V2 : "true" 로 설정하면 추적을 시작합니다.
- LANGCHAIN_ENDPOINT : <https://api.smith.langchain.com> 변경하지 않습니다.
- LANGCHAIN_API_KEY : 이전 단계에서 발급받은 키 를 입력합니다.
- LANGCHAIN_PROJECT : 프로젝트 명 을 기입하면 해당 프로젝트 그룹으로 모든 실행 (Run) 이 추적됩니다.

```
OPENAI_API_KEY=sk-iiYAv7V7YXfhy79Dgkz3T3  
LANGCHAIN_TRACING_V2=false  
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com  
LANGCHAIN_API_KEY=ls__69f264b1b0774d55  
LANGCHAIN_PROJECT=랭스미스에_표기할_프로젝트명
```

Jupyter Notebook 혹은 코드에서 추적을 활성화

● .env 파일에 LangSmith 키 설정

→ 설정한 추적이 활성화 되어 있고, API KEY 와 프로젝트 명이 제대로 설정되어 있다면,

```
from dotenv import load_dotenv  
  
load_dotenv()
```

→ 프로젝트 명을 변경하거나, 추적을 변경하고 싶을 때는 아래의 코드로 변경할 수 있습니다.

```
import os  
  
os.environ["LANGCHAIN_TRACING_V2"] = "true"  
os.environ["LANGCHAIN_ENDPOINT"] = "https://api.smith.langchain.com"  
os.environ["LANGCHAIN_PROJECT"] = "LangChain 프로젝트명"  
os.environ["LANGCHAIN_API_KEY"] = "LangChain API KEY 입력"
```

langchain-teddynote

🔴 langchain-teddynote

→ langchain 관련 기능을 보다 더 편리하게 사용하기 위한 목적의 패키지

```
pip install langchain-teddynote
```

→ .env 파일에 LangSmith API 키가 설정 되어 있어야 합니다.(LANGCHAIN_API_KEY)

```
from langchain_teddynote import logging  
  
# 프로젝트 이름을 입력합니다.  
logging.langsmith("원하는 프로젝트명")
```

→ 추적을 원하지 않을 때는 다음과 같이 추적을 끌 수 있습니다.

```
from langchain_teddynote import logging  
  
# set_enable=False 로 지정하면 추적을 하지 않습니다.  
logging.langsmith("랭체인 튜토리얼 프로젝트", set_enable=False)
```

Serpapi Key 발급

● Serpapi Key 발급

→ <https://serpapi.com/manage-api-key>

The screenshot displays the SerpApi 'Api Key' management interface. On the left is a dark sidebar with the 'SerpApi' logo and an 'API DASHBOARD' section containing links: 'Your Account', 'Api Key' (highlighted in orange), 'Billing Information', 'Change Plan', 'Edit Profile', 'Extra Credits', 'Invoices', 'Manage Plan', 'Team', and 'Your Searches'. The main content area has a light blue header with a search bar and '0 / 100 searches' indicator. Below this, the 'Api Key' section features a key icon and the title 'Your Private API Key'. A light gray box contains the API key: `45ae94787d3be6441259dd586435f22b4019989f87897d44130b0ed2a70493b7`. A blue button labeled 'Regenerate API Key' is positioned below the key.

Huggingface Key 발급

○ Huggingface Key 발급

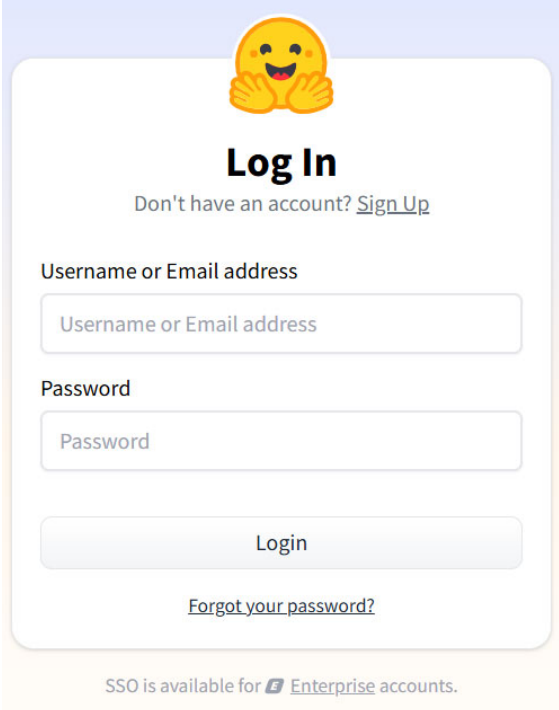
→ Hugging Face Hub

- ▶ 120k 이상의 모델, 20k의 데이터셋, 50k의 데모 앱(Spaces)을 포함하는 플랫폼
- ▶ 모든 것은 오픈 소스이며 공개적으로 이용할 수 있다

→ Huggingface

- ▶ 회원가입 (<https://huggingface.co>)
- ▶ 토큰 발급 (<https://huggingface.co/docs/hub/security-tokens>)
- ▶ LLM 의 READ 키를 복사

LLM 키: <https://huggingface.co/settings/tokens>



The image shows the Huggingface login interface. At the top is a yellow smiley face emoji with its hands clasped. Below it is the text 'Log In' in bold. Underneath is a link: 'Don't have an account? [Sign Up](#)'. There are two input fields: 'Username or Email address' and 'Password'. Below these is a 'Login' button. At the bottom is a link: '[Forgot your password?](#)'. At the very bottom, it says 'SSO is available for [Enterprise](#) accounts.'

● Anthropic API 𐄂

→ <https://console.anthropic.com/login?returnTo=%2F%3F>

● Google AI Studio API 키

→ <https://aistudio.google.com/app/apikey>

ChatGPT 모델 이해와 활용

● ChatGPT 크롬 확장 프로그램

→ 확장 프로그램 : ChatGPT 기능 확장

- ▶ [프롬프트 지니](#) : ChatGPT 상에서 자동 한영/영한 번역
- ▶ [ShareGPT](#) : 검색자료 공유하기 예: <https://chatgpt.com/share/670f23c0-8f30-8012-bfbf-6d94879d4e9b>
- ▶ [WebChatGPT](#) : 최신 정보로 결과 받기
- ▶ [AIPRM for ChatGPT](#) : 전문가가 만든 프롬프트를 제공
- ▶ [Sider](#) : 웹 문서 내용을 바로 요약 (블로그)

→ 확장 프로그램: GPT 기반 생산성 툴

- ▶ [ChatPDF](#) : PDF를 학습해서 PDF 문서 기반으로 요약, 질문, 콘텐츠 생성
- ▶ [Eightify](#) : 유튜브 비디오 내용을 바로 요약 (유튜브)

자연어 처리(NLP)의 응용 분야와 성공 사례

●NLP 응용 분야와 성공 사례

응용 분야	설명
문서 처리 및 정보 검색	검색 엔진, 문서를 자동으로 요약, 문서의 주제, 카테고리를 분류 Google Search SummarizeBot : 뉴스, 연구 논문 등을 자동 요약하여 정보 과부하 문제 해결
고객 서비스	실시간 자동 응답 챗봇, 감정 분석(고객 피드백 분석을 통해 긍정/부정 감정 이해) ChatGPT : 대화형 AI 모델로, 고객 지원, 학습 보조, 코드 디버깅 등 다양한 작업 지원 Zendesk : NLP를 활용하여 고객 서비스 요청을 자동으로 분류하고 라우팅
번역 및 언어 처리	한 언어에서 다른 언어로 텍스트 변환(기계 번역) 음성을 텍스트로 변환하고 다국어 지원(언어 인식 및 변환) Google Translate : NLP와 딥러닝 기술을 사용한 다국어 번역 서비스 DeepL : 문맥을 더 잘 이해하는 고품질 번역
음성 인식 및 합성	음성 텍스트 변환 및 텍스트 음성 변환(TTS) Amazon Alexa, Apple Siri : 음성 기반 가상 비서 Otter.ai : 회의 녹음 내용을 실시간 텍스트로 변환하여 저장

자연어 처리(NLP)의 응용 분야와 성공 사례

● NLP 응용 분야와 성공 사례

응용 분야	설명
의료 및 생명 과학	의료 기록 및 보고서에서 중요한 정보 추출(임상 문서 분석) 증상 기술을 분석하여 적절한 진단 제안(질병 진단 지원) IBM Watson Health : NLP를 통해 방대한 의료 데이터를 분석하여 의사에게 인사이트 제공 Mayo Clinic : 의료 기록 분석을 통해 환자 맞춤형 치료법 추천
금융 및 비즈니스	금융 뉴스 분석으로 시장 트렌드 예측 계약서 및 문서 자동 처리 Bloomberg Terminal : NLP로 금융 뉴스를 실시간 분석하여 투자 의사결정 지원 JP Morgan's COiN : 계약서 검토를 자동화하여 업무 효율성 향상
교육 및 학습	텍스트 작성 중 문법 및 스타일 오류 감지하고 교정 개인화된 학습 도우미 Grammarly : 텍스트의 문법, 스타일, 어조를 분석하고 제안 Duolingo : 다국어 학습에 NLP를 적용하여 개인 맞춤형 학습 제공

Q & A